

INFORMATIQUE • 1ÈRE C

MODULE 2 : ALGORITHMIQUE ET PROGRAMMATION

LEÇON N08

PROGRAMMATION EN HTML

DURÉE CONSEILLÉE : 3 H

DANS CETTE LEÇON

Cette leçon poursuit l'apprentissage du HTML en montrant comment présenter des données tabulaires, enrichir une page avec des images, sons et vidéos, et recueillir des informations grâce aux formulaires. Elle introduit également l'organisation professionnelle des dossiers d'un site et le rôle du CSS dans la mise en forme.

- À la fin de cette leçon, je sais définir ce qu'est un tableau HTML et identifier ses balises (table, tr, th, td)
- À la fin de cette leçon, je sais insérer un tableau structuré (avec en-têtes, fusion de cellules) dans une page web
- À la fin de cette leçon, je sais insérer une image, un son et une vidéo dans une page web avec les attributs adaptés
- À la fin de cette leçon, je sais créer un formulaire HTML complet (champs texte, boutons radio, cases à cocher, liste déroulante, boutons)
- À la fin de cette leçon, je sais choisir le bon type de champ de formulaire selon l'information à recueillir
- À la fin de cette leçon, je sais organiser le dossier d'un site web (arborescence, chemins relatifs)



TÉLÉCHARGE ONBUCH

Gratuit • Google Play
Tuteur IA, annales,
concours, résultats



PASSE À ONBUCH+

Tous les cours signés, Tuteur IA
illimité, fiches PDF — onglet
OnBuch+ de l'app

CE COURS SIGNÉ CONTIENT

COURS

complet, illustré, pas à pas

MÉTHODES

et exercices résolus

EXERCICES

type examen + corrigés

FICHE BILAN

l'essentiel à retenir



Signé • OnBuch+

Ludovic A.
CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+

SOMMAIRE

- | | |
|--|----------------------------------|
| 1. Les tableaux en HTML | plète |
| 2. Insertion des fichiers multimédias | 7. Activité d'intégration |
| 3. Les formulaires en HTML | 8. Méthodes et exercices résolus |
| 4. Organisation du dossier du site | 9. Exercices |
| 5. Contribution du CSS | 10. Corrigés des exercices |
| 6. Synthèse : construire une page com- | 11. Fiche bilan |

OBJECTIFS DE LA LEÇON

- À la fin de cette leçon, je sais définir ce qu'est un tableau HTML et identifier ses balises (table, tr, th, td)
- À la fin de cette leçon, je sais insérer un tableau structuré (avec en-têtes, fusion de cellules) dans une page web
- À la fin de cette leçon, je sais insérer une image, un son et une vidéo dans une page web avec les attributs adaptés
- À la fin de cette leçon, je sais créer un formulaire HTML complet (champs texte, boutons radio, cases à cocher, liste déroulante, boutons)
- À la fin de cette leçon, je sais choisir le bon type de champ de formulaire selon l'information à recueillir
- À la fin de cette leçon, je sais organiser le dossier d'un site web (arborescence, chemins relatifs)
- À la fin de cette leçon, je sais décrire la contribution du CSS à la présentation d'une page web
- À la fin de cette leçon, je sais construire une mini-page web combinant tableau, médias et formulaire

PRÉREQUIS

- Notion de site web, page web, navigateur et langage HTML (leçons précédentes du module)
- Structure de base d'un document HTML : balises html, head, body, titres et para-

graphes

- Notion de balise ouvrante/fermante et d'attribut (ex. : `<a href="...">`)
- Notion de fichier, dossier et arborescence (système d'exploitation)
- Utilisation élémentaire d'un éditeur de texte (Notepad++, Bloc-notes) et d'un navigateur

1 LES TABLEAUX EN HTML

Le lycée de Nkongsamba vient de clôturer son conseil de classe du premier trimestre. Le proviseur, très fier des résultats, convoque le club informatique : « Je veux publier sur le site du lycée les effectifs par série, les moyennes générales et les taux de réussite, présentés dans un tableau clair et lisible. J'aimerais aussi y voir la photo de la cérémonie des excellents et la vidéo de l'hymne national chanté ce jour-là. Et puisque nous y sommes, que les parents puissent s'inscrire en ligne à la réunion de rentrée ! »

Voilà une commande concrète et exigeante. Elle pose quatre questions : comment structurer des données chiffrées en **tableau** HTML ? Comment intégrer des fichiers multimédias ? Comment créer un formulaire d'inscription ? Et comment organiser proprement tous les fichiers du site ? C'est tout l'objet de cette leçon. Nous commençons par la première question : le tableau, outil roi de la présentation de données organisées.

1.1 Qu'est-ce qu'un tableau HTML ?

Intuition. Imagine le cahier de notes de ton professeur de mathématiques : les noms des élèves en lignes, les matières ou les évaluations en colonnes, et à chaque intersection une note. Cette organisation en lignes et colonnes permet de lire une information d'un seul coup d'œil : on suit la ligne de l'élève, on descend la colonne de la matière, et à l'intersection on trouve la donnée cherchée. Un tableau HTML fait exactement la même chose dans une page web : il présente des données tabularisées (tabulaires), c'est-à-dire des données qui ont naturellement une structure de grille.

DÉFINITION Tableau HTML

Un **tableau HTML** est un élément de page web qui présente des données organisées en lignes et en colonnes. Il est délimité par la balise conteneur `<table>...</table>`. À l'intérieur :

- chaque ligne est définie par la balise `<tr>` (table row) ;
- chaque cellule d'en-tête est définie par la balise `<th>` (table header) : son contenu s'affiche en gras et centré par défaut ;
- chaque cellule de données est définie par la balise `<td>` (table data) : son contenu s'affiche aligné à gauche par défaut.

ATTENTION Un tableau, c'est ligne par ligne

En HTML, on ne définit jamais les colonnes directement : on construit le tableau ligne par ligne. Chaque `<tr>` contient autant de cellules (`<th>` ou `<td>`) qu'il y a de colonnes. Le nombre de colonnes du tableau est donc déterminé par la ligne qui contient le plus de cellules. Si une ligne a moins de cellules que les autres, le tableau sera déformé.

1.2 Les balises de base : table, tr, th, td

Intuition. Construire un tableau HTML, c'est comme construire un mur : `<table>` est le terrain délimité, chaque `<tr>` est une rangée de briques, et chaque `<td>` ou `<th>` est une brique. On pose les rangées les unes après les autres, de haut en bas.

PROPRIÉTÉ Syntaxe de base d'un tableau

```
<table>
<tr>           <!-- premiere ligne : les en-tetes -->
  <th>Nom</th>  <!-- cellule d'en-tete : gras, centre -->
  <th>Moyenne</th>
</tr>
<tr>           <!-- deuxieme ligne : des donnees -->
  <td>MBARGA Paul</td>
  <td>14,5</td>
</tr>
</table>
```

Toute balise ouverte (`<table>`, `<tr>`, `<th>`, `<td>`) doit être refermée (`</table>`, `</tr>`, `</th>`, `</td>`), dans l'ordre inverse de l'ouverture : les cellules se ferment avant la ligne, les lignes avant le tableau.

Matière	Note	Coef.
Maths	15	4
Physique	13	3

`<table> ... </table>`

Correspondance visuelle entre les balises et le rendu : `<table>` englobe tout, chaque `<tr>` est une ligne, `<th>` les en-têtes (orange), `<td>` les données (bleu).

MÉTHODE Construire un tableau en 4 étapes

1. Dessiner la grille sur papier. Avant de toucher au clavier, trace le tableau au brouillon : combien de lignes ? combien de colonnes ? quelles cellules d'en-tête ? Cette étape évite 90 % des erreurs.
2. Écrire le conteneur. Ouvre et ferme immédiatement `<table>` et `</table>` : le squelette est prêt.

3. Écrire chaque ligne. Pour chaque ligne de ton brouillon, ajoute une paire `<tr>...</tr>`. La première ligne contient généralement les en-têtes en `<th>`.
4. Remplir les cellules. Dans chaque ligne, place les `<td>...</td>` dans l'ordre des colonnes, en vérifiant que chaque ligne a le même nombre de cellules.

1.3 Bordures : l'attribut border et le rôle du CSS

Par défaut, un tableau HTML s'affiche **sans aucune bordure** : les données flottent dans la page et deviennent illisibles. Historiquement (HTML ancien), on utilisait l'attribut `border` pour tracer les contours :

```
<table border="1"> <!-- bordure de 1 pixel : methode historique -->
```

À RETENIR Bordures : hier et aujourd'hui

L'attribut `border="1"` est obsolète : il mêle le contenu (HTML) et la présentation. Aujourd'hui, la règle d'or du web est la séparation des rôles : le HTML décrit la structure et le sens des données, le CSS (que nous étudierons en fin de leçon) gère toute la mise en forme : bordures, couleurs, espacements. Retiens néanmoins l'existence de `border`, car tu le rencontreras dans d'anciens sites et dans certains sujets.

1.4 Regroupement sémantique : `thead`, `tbody`, `tfoot` et `caption`

Intuition. Quand un tableau devient long (les 120 élèves de la 1^{re} C!), il est utile de distinguer clairement trois zones : la ligne de titres en haut, le corps des données au milieu, et une éventuelle ligne de synthèse (totaux, moyennes) en bas. HTML fournit des balises pour nommer explicitement ces zones.

DÉFINITION Balises de structure d'un tableau

- `<thead>` : regroupe la ou les lignes d'en-tête du tableau ;
- `<tbody>` : regroupe les lignes de données (le corps) ;
- `<tfoot>` : regroupe la ou les lignes de pied (totaux, synthèses) ;
- `<caption>` : donne un titre (une légende) au tableau, affiché par défaut juste au-dessus de lui. Elle se place juste après `<table>`.

Ces balises ne changent pas seulement l'organisation du code : elles aident les navigateurs, les moteurs de recherche et les lecteurs d'écran (utilisés par les personnes malvoyantes) à comprendre le tableau. C'est ce qu'on appelle la **sémantique** : le code dit ce que les données sont,

pas seulement comment elles s'affichent.

1.5 Fusion de cellules : colspan et rowspan

Intuition. Dans le bulletin de notes, la mention « Trimestre 1 » est souvent écrite une seule fois, à cheval sur plusieurs colonnes. De même, le nom d'un élève peut s'étendre sur plusieurs lignes. HTML permet ces **fusions de cellules** grâce à deux attributs.

PROPRIÉTÉ Attributs de fusion

- `colspan="n"` : la cellule s'étend horizontalement sur `n` colonnes ;
- `rowspan="n"` : la cellule s'étend verticalement sur `n` lignes.

Règle de cohérence : quand une cellule fusionne, les cellules qu'elle « recouvre » ne doivent pas être écrites. Une ligne contenant un `colspan="2"` aura donc une cellule de moins, et la ligne suivant un `rowspan="2"` aussi.

colspan="2"		C
D	E	F

La cellule orange occupe 2 colonnes

rowspan="2"	B	C
	E	F

La cellule orange occupe 2 lignes

Effet des fusions : à gauche, `colspan="2"` étire une cellule sur deux colonnes ; à droite, `rowspan="2"` l'étire sur deux lignes.

1.6 Exemple guidé : les moyennes de la 1^{re} C

Appliquons tout cela à la commande du proviseur : publier le tableau des moyennes par matière de la 1^{re} C, avec coefficients.

EXEMPLE Tableau des moyennes de la 1^{re} C (code complet commenté)

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Resultats 1ere C – Lycee de Nkongsamba</title>
</head>
<body>
  <h1>Conseil de classe – 1ere C</h1>

  <table border="1"> <!-- border historique : le CSS fera mieux -->
    <caption>Moyennes du 1er trimestre par matiere</caption>
```

```
<thead>
  <tr>
    <th>Matiere</th>
    <th>Moyenne / 20</th>
    <th>Coefficient</th>
  </tr>
</thead>

<tbody>
  <tr>
    <td>Mathematiques</td>
    <td>13,5</td>
    <td>4</td>
  </tr>
  <tr>
    <td>Physique-Chimie</td>
    <td>12,8</td>
    <td>3</td>
  </tr>
  <tr>
    <td>Informatique</td>
    <td>15,2</td>
    <td>2</td>
  </tr>
</tbody>

<tfoot>
  <tr>
    <!-- cette cellule s'étend sur 2 colonnes -->
    <td colspan="2">Moyenne generale de la classe</td>
    <td>13,4</td>
  </tr>
</tfoot>
</table>
</body>
</html>
```

Observe la ligne du `<tfoot>` : grâce à `colspan="2"`, la première cellule recouvre les colonnes « Matière » et « Moyenne », et la ligne ne contient donc que deux cellules au lieu de trois.

EXERCICE RÉSOLU 1 Ajouter une cellule fusionnée verticale

Énoncé. On veut regrouper les matières scientifiques : une cellule « Sciences » doit s'étendre sur les lignes Maths et Physique-Chimie, dans une nouvelle première colonne. Écris le `<tbody>` correspondant.

SOLUTION

Solution détaillée. On ajoute une colonne, donc chaque ligne aura une cellule de plus. La cellule « Sciences » porte `rowspan="2"` sur la première ligne ; elle n'est pas réécrite sur la

deuxième ligne, qu'elle recouvre déjà. La ligne « Informatique », hors regroupement, reçoit une cellule « Lettres et autres » (ou simplement « Autres »).

```
<tbody>
  <tr>
    <td rowspan="2">Sciences</td> <!-- recouvre 2 lignes -->
    <td>Mathematiques</td>
    <td>13,5</td>
    <td>4</td>
  </tr>
  <tr>
    <!-- PAS de cellule ici : rowspan la recouvre -->
    <td>Physique-Chimie</td>
    <td>12,8</td>
    <td>3</td>
  </tr>
  <tr>
    <td>Autres</td>
    <td>Informatique</td>
    <td>15,2</td>
    <td>2</td>
  </tr>
</tbody>
```

Vérification : chaque ligne compte bien 4 colonnes au total (la cellule fusionnée compte pour la ligne qu'elle recouvre).

ATTENTION L'erreur qui déforme tout le tableau

L'erreur la plus fréquente des débutants est d'oublier de fermer une balise `</td>` ou `</tr>`. Le navigateur « répare » alors le code à sa manière, et le résultat est imprévisible : cellules décalées, colonnes fantômes, tableau explosé. Deux réflexes de professionnel :

- indente ton code (chaque niveau décalé vers la droite) : une balise non fermée saute immédiatement aux yeux ;
- ferme chaque balise dès que tu l'ouvres, puis écris le contenu entre les deux.

Autre piège classique : écrire les cellules recouvertes par un `colspan` ou un `rowspan`, ce qui ajoute des colonnes en trop.

LESAVAIS-TU ? Les tableaux ont mis en page le web entier

Dans les années 1990, le CSS n'existait pas encore (ou était très peu supporté). Pour placer un menu à gauche et le texte à droite, les développeurs utilisaient... des tableaux invisibles (`border="0"`) ! Des sites entiers étaient emboîtés dans des grilles de cellules. C'était un détournement : un tableau est fait pour des données tabulaires, pas pour de la mise en page. Depuis l'arrivée du CSS, cette pratique est à bannir : on réserve `<table>` aux vraies données

en lignes et colonnes — comme nos résultats du conseil de classe — et on confie toute la mise en page au CSS, que nous découvrirons à la fin de cette leçon.

EXERCICE 1 À toi de jouer

Le proviseur veut un second tableau : « Effectifs et taux de réussite par série » (séries C, D, E ; colonnes : Série, Effectif, Admis, Taux). Dessine la grille sur papier, puis écris le code HTML complet avec `<caption>`, `<thead>`, `<tbody>` et un `<tfoot>` contenant une ligne « Total » utilisant `colspan`.

CORRIGÉ Exercice 1

```
<table border="1">
  <caption>Effectifs et taux de réussite par série</caption>
  <thead>
    <tr>
      <th>Serie</th>
      <th>Effectif</th>
      <th>Admis</th>
      <th>Taux</th>
    </tr>
  </thead>
  <tbody>
    <tr><td>C</td><td>120</td><td>96</td><td>80 %</td></tr>
    <tr><td>D</td><td>110</td><td>82</td><td>75 %</td></tr>
    <tr><td>E</td><td>45</td><td>30</td><td>67 %</td></tr>
  </tbody>
  <tfoot>
    <tr>
      <td colspan="3">Taux general de réussite</td>
      <td>76 %</td>
    </tr>
  </tfoot>
</table>
```

Points de vigilance : la ligne du `<tfoot>` ne contient que deux cellules grâce au `colspan="3"`, et toutes les balises sont fermées dans l'ordre.

À RETENIR L'essentiel sur les tableaux

- Un tableau HTML présente des données en lignes (`<tr>`) et colonnes, construit ligne par ligne dans le conteneur `<table>`.
- `<th>` = en-tête (gras, centré) ; `<td>` = donnée.

- `<thead>`, `<tbody>`, `<tfoot>` structurent le tableau ; `<caption>` le titre.
- `colspan` fusionne horizontalement, `rowspan` verticalement ; les cellules recouvertes ne s'écrivent pas.
- La bordure (et toute la mise en forme) relève du CSS, pas du HTML moderne.

2 INSERTION DES FICHIERS MULTIMÉDIAS

Dans la section précédente, nous avons appris à structurer des données tabulaires avec `<table>`, `<tr>`, `<th>` et `<td>`. Notre page commence à ressembler à un vrai document organisé. Mais avouons-le : une page composée uniquement de texte et de tableaux reste austère. Quand tu visites le site d'un lycée, tu t'attends à voir la photo du bâtiment, à écouter peut-être l'hymne de l'établissement, à regarder une vidéo de la cérémonie de remise des prix. Ces contenus — images, sons, vidéos — sont appelés des **fichiers multimédias**. Cette section t'apprend à les insérer proprement dans une page HTML.

2.1 Notion de fichier multimédia

DÉFINITION Fichier multimédia

Un **fichier multimédia** est un fichier numérique qui transporte un contenu perceptible par les sens : une image (vue), un son (ouïe) ou une vidéo (vue et ouïe combinées). Contrairement au texte, ces contenus ne sont pas écrits directement dans le fichier HTML : ils sont stockés dans des fichiers séparés, que la page référence par un chemin d'accès.

Chaque type de contenu possède des **formats** de fichier normalisés. Le navigateur reconnaît le format grâce à l'**extension** du nom de fichier.

Type	Formats courants	Caractéristiques
Image	jpg (ou jpeg), png, gif	jpg : photos, taille réduite ; png : images avec transparence ; gif : petites animations
Son	mp3, wav	mp3 : compressé, léger ; wav : non compressé, lourd mais fidèle
Vidéo	mp4	mp4 : le format le plus universel, lu par tous les navigateurs modernes

ATTENTION Le poids des fichiers

Une image ou une vidéo est beaucoup plus lourde qu'un texte. Une photo de plusieurs mégaoctets mettra des minutes à s'afficher chez un visiteur dont la connexion est lente — situation fréquente sur les réseaux mobiles. Pense donc à redimensionner et compresser tes images avant de les publier.

2.2 Insérer une image : la balise `<img>`

Une balise orpheline

Pour afficher une image, HTML utilise la balise `<img>` (de l'anglais image). Cette balise a une particularité fondamentale : elle n'a **pas de balise fermante**. On dit que c'est une **balise orpheline** (ou auto-fermante). Pourquoi ? Parce qu'une image n'a pas de « contenu » textuel à encadrer : toute l'information nécessaire est portée par ses **attributs**.

DÉFINITION Balise orpheline

Une **balise orpheline** (ou balise auto-fermante) est une balise qui ne possède pas de balise fermante, car elle n'encadre aucun contenu. Toute l'information est fournie par ses attributs. Exemple : ``. Autres balises orphelines déjà rencontrées : `<br>`, `<hr>`, `<meta>`.

PROPRIÉTÉ Syntaxe et attributs de `<img>`

```

```

- `src` (source) : obligatoire. Indique le chemin d'accès au fichier image.
- `alt` (alternative) : texte de remplacement, fortement recommandé (voir plus bas).
- `width` et `height` : largeur et hauteur d'affichage en pixels. On peut n'en préciser qu'une seule : le navigateur calcule l'autre en conservant les proportions.

MÉTHODE Insérer une image en 3 étapes

1. Placer le fichier image dans le dossier du site, par exemple dans un sous-dossier `images` (nous verrons l'organisation des dossiers en section 4). Exemple : copier `lycee.jpg` dans `monsite/images/`.
2. Écrire la balise `<img>` à l'endroit de la page où l'image doit apparaître.
3. Renseigner les attributs : `src` avec le chemin relatif vers le fichier, `alt` avec une des-

cription fidèle, et éventuellement `width`.

EXEMPLE Exemple chiffré : photo du lycée

```
<!-- Le fichier lycee.jpg se trouve dans le sous-dossier images -->  

```

Le navigateur affiche l'image `lycee.jpg` avec une largeur de 300 pixels ; la hauteur est calculée automatiquement pour ne pas déformer la photo. Si l'image ne peut pas être chargée (fichier absent, connexion coupée), le texte « Bâtiment principal du lycée » s'affiche à sa place.

Le rôle crucial de l'attribut `alt`

L'attribut `alt` n'est pas une option de confort : il joue un double rôle.

- **Texte de remplacement** : si l'image ne se charge pas (chemin erroné, fichier supprimé, connexion lente), le navigateur affiche le contenu de `alt` à la place. Le visiteur comprend alors ce qu'il aurait dû voir.
- **Accessibilité** : les personnes malvoyantes utilisent des lecteurs d'écran, des logiciels qui lisent la page à voix haute. Pour une image, le lecteur vocalise le contenu de `alt`. Sans cet attribut, l'image est tout simplement invisible pour ces utilisateurs.

LESAVAIS-TU ? Le Web pour tous

L'attribut `alt` est lu à voix haute par les lecteurs d'écran utilisés par les personnes malvoyantes ou aveugles. Le W3C, organisme qui définit les standards du Web, impose sa présence dans ses règles d'accessibilité (WCAG). Rédiger un bon `alt` — court et descriptif, comme « élèves en uniforme devant le portail du lycée » plutôt que « image1 » — c'est rendre ton site utilisable par tous. Un bon développeur est un développeur inclusif !

2.3 Insérer un son : la balise `<audio>`

Pour diffuser un fichier sonore (musique, hymne, enregistrement d'un cours), HTML5 fournit la balise `<audio>`. Contrairement à `<img>`, c'est une balise **paire** : elle possède une balise fermante `</audio>`, car elle peut contenir des éléments enfants.

PROPRIÉTÉ Syntaxe de `<audio>`

```
<audio controls>
```

```
<source src="sons/hymne.mp3" type="audio/mpeg">
<source src="sons/hymne.wav" type="audio/wav">
Votre navigateur ne supporte pas la lecture audio.
</audio>
```

- L'attribut `controls` (sans valeur) fait apparaître le lecteur : boutons lecture/pause, barre de progression, volume. Sans lui, le son est invisible et impossible à piloter !
- Chaque balise `<source>` propose un fichier. Le navigateur utilise la première source qu'il sait lire ; les suivantes sont des solutions de repli.
- Le texte placé avant `</audio>` ne s'affiche que si le navigateur est trop ancien pour gérer `<audio>`.

2.4 Insérer une vidéo : la balise `<video>`

La balise `<video>` fonctionne exactement comme `<audio>`, avec en plus les attributs de dimension `width` et `height` pour délimiter la zone d'affichage de la vidéo.

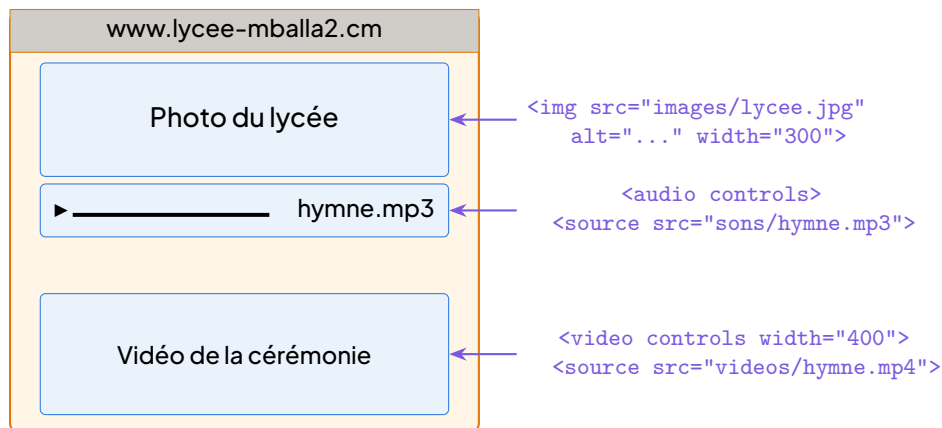
EXEMPLE Exemple chiffré : vidéo de l'hymne du lycée

```
<video controls width="400">
  <source src="videos/hymne.mp4" type="video/mp4">
  Votre navigateur ne supporte pas la lecture video.
</video>
```

Le lecteur vidéo s'affiche sur 400 pixels de large, avec les boutons de lecture. Le fichier lu est `videos/hymne.mp4`.

ATTENTION Oublier `controls`

L'erreur classique du débutant est d'oublier l'attribut `controls`. Résultat : la vidéo ou le son est chargé mais aucun bouton n'apparaît — impossible de lancer la lecture. Pense aussi à toujours fournir le format `mp4` pour les vidéos, c'est le plus compatible.



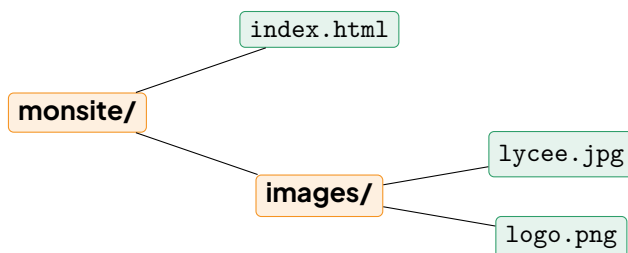
Une page web affichant une image, un lecteur audio et un lecteur vidéo, avec en regard les balises HTML qui produisent chaque élément.

2.5 Chemins d'accès : relatif ou absolu ?

L'attribut `src` contient un **chemin d'accès** qui dit au navigateur où trouver le fichier. Il existe deux façons d'écrire ce chemin.

DÉFINITION Chemin relatif et chemin absolu

- Un **chemin relatif** indique la position du fichier par rapport à la page HTML qui l'appelle. Exemple : `images/logo.png` signifie « dans le sous-dossier `images`, le fichier `logo.png` ». La notation `../` permet de remonter d'un dossier.
- Un **chemin absolu** (URL complète) indique l'adresse entière d'une ressource sur le Web : `https://www.exemple.cm/images/logo.png`.



Depuis `index.html`, le chemin relatif vers le logo est : `images/logo.png`

Arborescence du dossier du site : la page `index.html` atteint `logo.png` par le chemin relatif `images/logo.png`.

Règle d'or : pour les fichiers qui appartiennent à ton site, utilise toujours un **chemin relatif**. Ainsi, quand tu déplaceras tout le dossier `monsite` vers un hébergeur, tous les liens internes continueront de fonctionner. Le chemin absolu (URL) se réserve aux ressources hébergées sur d'autres sites.

ATTENTION Le piège du chemin absolu local

Erreur fréquente : écrire ``. Sur ton ordinateur, l'image s'affiche... mais une fois le site publié sur Internet, le navigateur du visiteur cherchera ce fichier sur son propre disque C :, où il n'existe pas ! L'image sera cassée pour tout le monde sauf toi. Toujours copier le fichier dans le dossier du site et utiliser un chemin relatif.

2.6 Image cliquable : combiner `<a>` et `<img>`

On souhaite souvent qu'un clic sur une image (un logo, une miniature) conduise vers une autre page. Rien de plus simple : il suffit de placer la balise `<img>` à l'intérieur d'un lien `<a>`, à la place du texte du lien.

EXEMPLE Logo cliquable renvoyant vers l'accueil

```
<a href="index.html">
  
</a>
```

L'image devient un lien : le curseur se transforme en main au survol, et un clic ouvre `index.html`. Remarque que l'`alt` précise la destination du lien, ce qui aide les lecteurs d'écran.

2.7 Mise en pratique

EXPÉRIENCE TP : une page « galerie » multimédia

Objectif : créer une page présentant ton établissement avec une image, un son et une vidéo.

Matériel : un ordinateur, un éditeur de texte (Notepad++, VS Code), un navigateur, une image `lycee.jpg`, un son `hymne.mp3` et une vidéo `fete.mp4`.

Manipulation :

1. Crée le dossier `monsite` avec deux sous-dossiers `images` et `medias`. Place l'image dans `images`, le son et la vidéo dans `medias`.
2. Crée le fichier `galerie.html` avec le code suivant :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Galerie du lycée</title>
</head>
<body>
  <h1>Notre établissement en images</h1>

  <h2>Le bâtiment principal</h2>
  

  <h2>Notre hymne</h2>
  <audio controls>
    <source src="medias/hymne.mp3" type="audio/mpeg">
  </audio>

  <h2>La fête de la jeunesse</h2>
  <video controls width="400">
    <source src="medias/fete.mp4" type="video/mp4">
  </video>
</body>
</html>
```

3. Ouvre `galerie.html` dans le navigateur et observe.
4. Renomme `lycee.jpg` en `ecole.jpg` sans modifier le code, puis recharge la page.

Observations : à l'étape 3, l'image, le lecteur audio et le lecteur vidéo s'affichent. À l'étape 4, l'image ne s'affiche plus : le texte de l'attribut `alt` apparaît à la place, car le chemin `src` ne correspond plus à aucun fichier.

Interprétation : le navigateur charge les fichiers multimédias séparément du fichier HTML, en suivant les chemins indiqués. Un chemin erroné « casse » la ressource, et l'`alt` prend le relais.

EXERCICE RÉSOLU 2 Corriger une page défilante

Énoncé. Un élève écrit la ligne suivante dans sa page, mais l'image ne s'affiche pas et aucune description n'apparaît pour les malvoyants :

```

</img>
```

Identifie les trois erreurs et propose un code corrigé.

SOLUTION

Solution.

1. Chemin absolu local : `C:\Users\...` ne fonctionne que sur l'ordinateur de Marie. Il faut copier l'image dans le dossier du site et utiliser un chemin relatif.
2. Attribut `alt` absent : sans lui, rien ne s'affiche si l'image est introuvable, et les lecteurs d'écran restent muets.
3. Balise fermante `</img>` : `<img>` est une balise orpheline, elle ne se ferme jamais.

Code corrigé (après avoir placé `classe.png` dans le sous-dossier `images`) :

```

```

EXERCICE 2 À toi de jouer

Ton site possède l'arborescence suivante : `monsite/index.html`, `monsite/images/sport.jpg`, `monsite/videos/match.mp4`. Écris le code HTML de `index.html` affichant : le titre « Sport au lycée », l'image `sport.jpg` sur 250 pixels de large avec un `alt` pertinent, puis la vidéo `match.mp4` dans un lecteur de 350 pixels de large. Enfin, rends l'image cliquable vers une page `sport.html`.

CORRIGÉ Exercice

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Sport au lycée</title>
</head>
<body>
  <h1>Sport au lycée</h1>
  <a href="sport.html">
    
```

```
</a>
<video controls width="350">
  <source src="videos/match.mp4" type="video/mp4">
  Votre navigateur ne supporte pas la lecture video.
</video>
</body>
</html>
```

Tous les chemins sont relatifs, l'attribut `alt` est renseigné, et l'attribut `controls` n'est pas oublié.

À RETENIR L'essentiel de la section

- Les fichiers multimédias (image, son, vidéo) sont stockés hors du fichier HTML et référencés par un chemin d'accès.
- `<img>` est une balise orpheline : attributs `src` (obligatoire), `alt` (accessibilité et remplacement), `width/height`.
- `<audio>` et `<video>` sont des balises paires, utilisées avec `controls` et une ou plusieurs balises `<source>`.
- Toujours utiliser des chemins relatifs pour les fichiers du site ; jamais de chemin local du type `C:\...`
- Une image devient cliquable en l'insérant dans un lien `<a>`.

Nos pages savent désormais parler aux yeux et aux oreilles. Mais un site vit aussi de l'interaction : permettre au visiteur de saisir son nom, de choisir une option, d'envoyer un message. C'est précisément l'objet de la section suivante : les **formulaires en HTML**.

3 LES FORMULAIRES EN HTML

Après avoir vu comment enrichir nos pages avec des images, du son et de la vidéo, nous abordons maintenant une fonctionnalité fondamentale du Web : l'interaction avec le visiteur. Un site vitrine qui ne fait qu'afficher de l'information est statique. Dès lors que l'on souhaite recueillir des données — une inscription, une commande, un avis, une recherche — il faut un **formulaire**. C'est la porte d'entrée de toute application web dynamique. Dans cette section, nous allons construire, balise par balise, un formulaire complet, rigoureux et accessible.

3.1 Rappel et transition : du contenu statique à l'interaction

Dans la section précédente, nous avons manipulé des balises `<img>`, `<audio>`, `<video>` qui sont des contenus remplacés : le navigateur va chercher une ressource externe et l'affiche. Le formulaire, lui, ne s'affiche pas « tout seul » : il crée des **contrôles interactifs** (champs, boutons, listes) que l'utilisateur manipule. La grande différence ? Les données saisies doivent pouvoir quitter le navigateur pour aller vers un serveur. C'est le rôle de la balise conteneur `<form>`.

DÉFINITION Formulaire HTML et balise `<form>`

Un formulaire est une zone délimitée d'une page web qui regroupe des contrôles interactifs (champs de saisie, listes, boutons) permettant à l'utilisateur d'entrer des données pour les envoyer à un serveur en vue d'un traitement.

La balise `<form>` est le conteneur racine. Elle possède deux attributs indispensables :

- `action="URL"` : l'adresse (relative ou absolue) du script côté serveur qui recevra et traitera les données (ex. `traiter_inscription.php`, `/api/eleve`). Si absent, le formulaire se soumet à l'URL de la page courante.
- `method="get | post"` : la méthode HTTP utilisée pour l'envoi.

PROPRIÉTÉ Choix de la méthode : GET vs POST (admise)

- GET : les données sont encodées dans l'URL après un `?` (ex. `page.php?nom=Kamga&serie=C`). L'URL est visible, partageable, mise en favori, mais limitée en longueur (environ 2048 caractères selon les navigateurs) et non sécurisée pour des données sensibles (mots de passe).
- POST : les données sont envoyées dans le corps de la requête HTTP, invisibles dans la barre d'adresse. Pas de limite pratique de taille. C'est la méthode obligatoire pour toute modification de données serveur (inscription, paiement, mot de passe, upload de fichier).

Note programme : Le traitement réel des données (PHP, Python, Node.js, Java EE) est hors programme de Première. Nous nous concentrons sur la construction correcte du formulaire côté client.

3.2 Les champs de saisie de base : `<input>`

La balise `<input>` est caméléon : son comportement change radicalement selon son attribut type. C'est une balise **auto-fermante** (void element). Trois attributs sont transversaux et cruciaux :

- `name="identifiant"` : **nom de la variable** envoyée au serveur. **Obligatoire** pour que la valeur soit transmise. Sans `name`, le champ est muet.
- `id="identifiant_unique"` : identifiant unique dans la page, indispensable pour lier une `<label>` (accessibilité) et pour le JavaScript (`document.getElementById`).
- `value="valeur_par_defaut"` : valeur pré-remplie.

EXEMPLE Champ texte simple et mot de passe

```
<!-- Champ texte visible -->
<label for="nom">Nom complet :</label>
<input type="text" id="nom" name="nom_complet" placeholder="Ex : Kamga Jean" required>

<!-- Champ masqué (bullets) -->
<label for="pwd">Mot de passe :</label>
<input type="password" id="pwd" name="mot_de_passe" placeholder="8 caractères min" required
  minlength="8">
```

Attributs clés : `placeholder` (indice grisé qui disparaît à la saisie), `required` (validation HTML5 : empêche l'envoi si vide), `minlength`/`maxlength` (longueur).

3.3 Choix unique : boutons radio (`type="radio"`)

Pour une question à **choix unique** parmi un ensemble fini (Sexe, Série, Oui/Non), on utilise des `<input type="radio">`.

PROPRIÉTÉ Règle d'or des boutons radio : le `name` commun

Tous les boutons radio d'un même groupe (une seule question) doivent avoir exactement le même attribut `name`. C'est ce `name` partagé qui dit au navigateur : « un seul de ces boutons peut être coché à la fois ». Chaque bouton a sa propre `value` (la valeur envoyée si ce bouton est le choisi).

ATTENTION Erreur classique : `name` différents = groupes différents

Si vous écrivez :

```
<input type="radio" name="sexe_h" value="H"> Homme
<input type="radio" name="sexe_f" value="F"> Femme
```

Le navigateur voit deux groupes distincts. L'utilisateur peut cocher les deux en même temps. C'est une erreur logique grave. Correct : `name="sexe"` pour les deux.

EXEMPLE Groupe radio « Série »

```
<fieldset>
  <legend>Série :</legend>
  <label><input type="radio" name="serie" value="C" required> C</label>
  <label><input type="radio" name="serie" value="D"> D</label>
  <label><input type="radio" name="serie" value="E"> E</label>
  <label><input type="radio" name="serie" value="autre"> Autre</label>
</fieldset>
```

Astuce : `<fieldset>` + `<legend>` groupe visuellement et sémantiquement les champs liés (accessibilité lecteurs d'écran). Le `required` sur un seul radio du groupe suffit.

3.4 Choix multiples : cases à cocher (`type="checkbox"`)

Pour une question où **plusieurs réponses** sont possibles (options, langues parlées, centres d'intérêt), on utilise `<input type="checkbox">`.

PROPRIÉTÉ Cases à cocher : `name` distincts ou tableau

Chaque case à cocher est indépendante. Deux stratégies pour le `name` :

1. **Noms distincts** : `name="opt_sport"`, `name="opt_musique"`. Le serveur reçoit `opt_sport=on` (si cochée) ou rien (si décochée). Simple pour peu d'options.
2. **Même nom avec crochets** : `name="options[]"`. Le serveur reçoit un tableau de valeurs (ex. `options[]=sport&options[]=musique`). Standard en PHP, très pratique pour traiter une liste dynamique.

L'attribut `checked` (sans valeur) pré-coche la case.

3.5 Listes déroulantes et zones de texte étendues

EXEMPLE Liste déroulante `<select>` et zone multiligne `<textarea>`

```
<!-- Liste déroulante (choix unique par défaut) -->
<label for="classe">Classe :</label>
<select id="classe" name="classe" required>
  <option value="">-- Choisir --</option> <!-- Option vide pour forcer le choix -->
  <option value="1ereC">1ère C</option>
  <option value="1ereD">1ère D</option>
  <option value="1ereE">1ère E</option>
  <option value="termC">Terminale C</option>
<!-- ... -->
</select>

<!-- Zone de texte multiligne (motivation, commentaire) -->
<label for="motivation">Motivation :</label>
<textarea id="motivation" name="motivation" rows="5" cols="40"
  placeholder="Expliquez votre choix..." required maxlength="500"></textarea>
```

Points clés : `<option value="">` avec valeur vide force l'utilisateur à choisir (sinon la 1ère option est envoyée par défaut). `<textarea>` n'a pas d'attribut `value`, son contenu initial se place entre les balises d'ouverture et de fermeture. `rows/cols` définissent la taille d'affichage initiale (CSS préféré pour le design final).

3.6 L'accessibilité : la balise `<label>`

Un formulaire sans `<label>` correctement lié est inutilisable pour les malvoyants (lecteurs d'écran) et peu ergonomique pour tous (clic sur le texte ne focus pas le champ).

MÉTHODE Lier un `<label>` à son champ (deux méthodes équivalentes)

1. **Explicite (recommandée)** : `<label for="id_du_champ">Texte</label> + <input id="id_du_champ" ...>`. Le `for` pointe vers l'id du champ. Permet de séparer label et input dans le HTML (flexibilité CSS).
2. **Implicite (imbriquée)** : `<label>Texte <input ...></label>`. Le champ est à l'intérieur du label. Plus court, mais moins flexible pour la mise en page complexe.

Dans les deux cas, cliquer sur le texte du label donne le focus au champ associé.

3.7 Boutons d'action : `submit` et `reset`

PROPRIÉTÉ Boutons de formulaire

- `<input type="submit" value="Envoyer">` **OU** `<button type="submit">Envoyer</button>` : déclenche l'envoi des données vers l'action avec la `method`. `<button>` permet du contenu HTML riche (icône + texte).
- `<input type="reset" value="Effacer">` **OU** `<button type="reset">Effacer</button>` : remet tous les champs du formulaire à leur valeur initiale (vide ou `value` par défaut). À utiliser avec parcimonie : l'utilisateur peut cliquer par erreur et perdre sa saisie.
- `<button type="button">` : bouton inert côté formulaire (ne soumet pas, ne reset pas). Utilisé uniquement pour du JavaScript (ex. ouvrir une modale, calculer un prix en direct).

3.8 Illustration : Maquette annotée d'un formulaire d'inscription

```
<form action="traitement.php" method="post">
```

Formulaire Inscription Élève

Nom complet _____

```
<input type="text" name="nom" required>
```

Sexe (•) M (•) F

```
<input type="radio" name="sexe" value="M">
```

Série [1ère C]

```
<select name="serie">...
```

Classe [1ère C]

```
<select name="classe">...
```

Motivation [.....] [.....]

```
<textarea name="motiv">
```

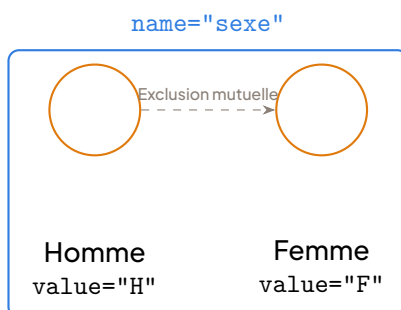
[Envoyer] [Effacer]

```
<button type="submit">
```

Maquette annotée : correspondance visuel ↔ balise HTML pour un formulaire d'inscription élève.

3.9 Illustration : Radio vs Checkbox — Schéma comparatif

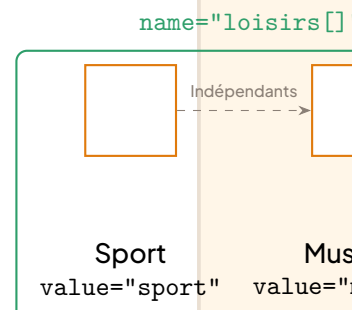
Boutons Radio (Choix unique)



```
<input type="radio" name="sexe" value="H">
<input type="radio" name="sexe" value="F">
```

Même *name* → un seul choix possible.

Cases à cocher (Choix multiples)



```
<input type="checkbox" name="loisirs[]" value="sport">
<input type="checkbox" name="loisirs[]" value="musique">
```

name identique + [] → tableau de valeurs côté serveur

Comparatif structurel : Radio (exclusion mutuelle via *name* commun) vs Checkbox (indépendance, *name[]* pour tableau).

3.10 Exemple guidé complet : Formulaire d'inscription à la réunion parents-professeurs

Construisons pas à pas le formulaire demandé. Contexte : le proviseur du Lycée Bilingue de Yaoundé veut collecter les présences pour la réunion de rentrée.

EXERCICE RÉSOLU 3

Formulaire « Réunion Parents-Professeurs »

Énoncé : Créer le formulaire HTML permettant à un parent d'inscrire sa présence. Champs requis : Nom du parent (texte), Téléphone (tel), Série de l'enfant (liste : C, D, E, Autre), Bouton « Valider mon inscription ». Le formulaire doit être envoyé en POST vers `inscription_reunion.php`.

Solution détaillée :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Inscription Réunion Parents-Professeurs</title>
  <style>
    /* CSS minimal pour lisibilité (voir section 5) */
    body { font-family: Arial, sans-serif; max-width: 500px; margin: 2rem auto; padding: 1rem; }
    .champ { margin-bottom: 1rem; display: flex; flex-direction: column; gap: 0.3rem; }
    label { font-weight: bold; }
```

```

    input, select { padding : 0.5rem; font-size : 1rem; border : 1px solid #ccc; border-radius : 4px; }
    button { padding : 0.7rem 1.5rem; background : #0066cc; color : white; border : none; border-
radius : 4px; cursor : pointer; font-size : 1rem; }
    button :hover { background : #004499; }
    fieldset { border : 1px solid #ddd; border-radius : 4px; padding : 1rem; margin-bottom : 1rem; }
    legend { font-weight : bold; color : #333; }
</style>
</head>
<body>
<h1>Réunion de rentrée 2024-2025</h1>
<p>Merci de confirmer votre présence.</p>

<form action="inscription_reunion.php" method="post" novalidate>
  <!-- novalidate : désactive la validation HTML5 native si on veut gérer les messages soi-même en JS
  . Ici on la garde active. -->

  <div class="champ">
    <label for="nom_parent">Nom du parent / tuteur *</label>
    <input type="text" id="nom_parent" name="nom_parent" placeholder="Ex : M. Kamga Joseph
" required>
  </div>

  <div class="champ">
    <label for="telephone">Téléphone (WhatsApp) *</label>
    <input type="tel" id="telephone" name="telephone" placeholder="+237 6XX XX XX XX"
required pattern="\+?\d{9,15}">
    <small>Format international recommandé.</small>
  </div>

  <fieldset>
    <legend>Série de l'enfant *</legend>
    <div class="champ" style="flex-direction : row; flex-wrap : wrap; gap : 1rem; align-items :
center;">
      <label><input type="radio" name="serie_enfant" value="C" required> C</label>
      <label><input type="radio" name="serie_enfant" value="D"> D</label>
      <label><input type="radio" name="serie_enfant" value="E"> E</label>
      <label><input type="radio" name="serie_enfant" value="autre"> Autre</label>
    </div>
  </fieldset>

  <div class="champ">
    <button type="submit">Valider mon inscription</button>
  </div>
</form>
</body>
</html>

```

Analyse technique :

1. `<form action="inscription_reunion.php" method="post">` : Envoi sécurisé (POST) vers le script de traitement.
2. `type="tel" + pattern` : Clavier numérique sur mobile + validation format basique (regex).
3. `<fieldset>/<legend>` : Groupe sémantique pour les radios. Le `required` sur un radio du groupe oblige le choix.

4. `novalidate` sur `<form>` : Utile si vous écrivez votre propre validation JavaScript (section JS). Sans, le navigateur bloque l'envoi et affiche ses bulles natives (souvent moches).
5. `<small>` : Aide contextuelle (accessibilité `aria-describedby` possible).

3.11 Méthode : Choisir le bon contrôle pour la bonne donnée

MÉTHODE Arbre de décision pour le choix du champ

1. Donnée libre, courte, visible ? → `<input type="text">` (nom, ville, référence).
2. Donnée sensible (mot de passe, code PIN) ? → `<input type="password">`.
3. Donnée numérique avec contraintes ? → `<input type="number" min="0" max="100" step="1">` ou `type="range"` (curseur).
4. Choix unique parmi peu d'options (2 à 5) visibles ? → `<input type="radio">` (groupe `name` commun).
5. Choix unique parmi beaucoup d'options (liste longue) ? → `<select>` (gain de place).
6. Choix multiples possibles ? → `<input type="checkbox">` (chaque case indépendante, `name="tab[]"` pour tableau).
7. Texte long, multiligne (commentaire, motivation) ? → `<textarea rows="5">`.
8. Fichier à uploader (photo, PDF) ? → `<input type="file" accept="image/*,.pdf">` + impératif : `<form method="post" enctype="multipart/form-data">`.
9. Date / Heure / Couleur / Email / URL / Recherche ? → Types HTML5 spécialisés : `date`, `time`, `color`, `email`, `url`, `search` (validation + UI native navigateur).

3.12 Organisation du dossier du site (rappel et application)

Un formulaire référence souvent des ressources : une page de traitement (`action`), une feuille de style, un script de validation, une image de fond. Une arborescence propre évite les liens cassés.

EXPÉRIENCE TP Guidé : Structurer le mini-projet « Inscription »

Objectif : Organiser les fichiers pour le formulaire ci-dessus + CSS + futur JS. Arborescence cible :

```

projet_inscription/≡≡≡
  index.html      # Page d'accueil avec le formulaire≡≡≡
  css/≡
```

```
=== style.css      # Styles communs===  
js/  
=== validation.js  # Validation côté client (plus tard)===  
php/  
=== inscription_reunion.php # Traitement serveur (hors programme, mais dossier prévu)===  
assets/  
  === images/  
  === logo_lycee.png  
  === uploads/      # Dossier d'écriture pour les fichiers uploadés (droits 755/775)
```

Manipulation :

1. Créer les dossiers dans votre éditeur (VS Code) ou terminal : `mkdir -p css js php assets/images assets/uploads`.
2. Placer `index.html` à la racine.
3. Dans `index.html`, lier le CSS : `<link rel="stylesheet" href="css/style.css">`.
4. Dans `<form>`, `action="php/inscription_reunion.php"` (chemin relatif descendant).
5. Tester : ouvrir `index.html` dans le navigateur, vérifier que le style s'applique.

Observation : Les chemins relatifs (`css/style.css`, `php/...`) fonctionnent car `index.html` est à la racine. Si vous déplacez `index.html` dans un sous-dossier `vues/`, il faudrait remonter : `href="../../css/style.css"`.

3.13 Contribution du CSS : du formulaire brut au formulaire pro

Le HTML structure, le CSS met en forme, aligne, espace, colore, répond. Sans CSS, un formulaire est une colonne verticale disgracieuse, les labels ne sont pas alignés, les messages d'erreur invisibles.

À RETENIR Rôles clés du CSS sur un formulaire

1. Mise en page (Layout) : `display: flex` ou `grid` sur le conteneur `.champ` pour aligner label et input sur la même ligne (ex. `label { width: 150px; }` ou `gap: 1rem`).
2. États visuels (Pseudo-classes) :
 - `input:focus` : contour visible (accessibilité clavier), fond clair.
 - `input:valid` / `input:invalid` : bordure verte/rouge après interaction (attention : `:invalid` est vrai au chargement si `required` et vide → utiliser `:not(:placeholder-shown):invalid` ou classe JS `.touched`).
 - `input:disabled` / `:read-only` : grisé.
3. Messages d'erreur : Cacher par défaut (`display: none`), afficher via classe JS `.erreur-visible` ou `input:invalid + .msg-erreur`.
4. Responsive : `@media (max-width: 600px) { .champ { flex-direction: column; }`

label { width: auto; } } : passage en colonne sur mobile.

5. **Apparence native** : `appearance: none` permet de restyler complètement `<select>`, `<input type="checkbox">`, `<input type="radio">` (remplacement par SVG/ICônes).

EXEMPLE Extrait CSS « Pro » pour formulaire (à mettre dans `css/style.css`)

```
/* Variables pour cohérence */
:root {
  --primary: #0066cc;
  --danger: #cc0000;
  --success: #008800;
  --border: #ccc;
  --bg-input: #fafafa;
}

/* Conteneur champ : label + input sur même ligne */
.champ { display: flex; flex-wrap: wrap; gap: 0.5rem; align-items: center; margin-bottom: 1rem; }
.champ > label { flex: 0 0 180px; text-align: right; font-weight: 600; }
.champ > input, .champ > select, .champ > textarea { flex: 1; min-width: 200px; padding: 0.6rem; border: 1px solid var(--border); border-radius: 4px; background: var(--bg-input); transition: border-color 0.2s, box-shadow 0.2s; }

/* Focus accessible */
.champ > input:focus, .champ > select:focus, .champ > textarea:focus {
  outline: none; border-color: var(--primary); box-shadow: 0 0 3px rgba(0,102,204,0.2);
}

/* Validation visuelle (après interaction via classe JS .validé/.erreur) */
.input-ok { border-color: var(--success) !important; }
.input-err { border-color: var(--danger) !important; }
.msg-erreur { color: var(--danger); font-size: 0.85rem; margin-top: 0.2rem; display: none; }
.input-err + .msg-erreur { display: block; } /* Frère adjacent */

/* Boutons */
button[type="submit"] { background: var(--primary); color: white; padding: 0.7rem 2rem; border: none; border-radius: 4px; cursor: pointer; font-weight: bold; transition: background 0.2s; }
button[type="submit"]:hover { background: #004499; }
button[type="reset"] { background: #eee; color: #333; margin-left: 1rem; }

/* Mobile first : label au-dessus */
@media (max-width: 599px) {
  .champ { flex-direction: column; align-items: stretch; }
  .champ > label { text-align: left; flex: none; }
}
```

3.14 Synthèse de la section : Check-list du formulaire parfait

Avant de passer à la section 4 (Organisation du dossier) et 5 (CSS approfondi), vérifiez que votre formulaire respecte ces points :

Critère	Vérification
Balise <code><form></code>	action (URL valide) + method="post" (par défaut sécurisé)
Chaque <code><input>/<select>/<textarea></code>	Attribut name unique et significatif (pas de name="a")
Boutons radio	Même name pour le groupe, value distinctes
Cases à cocher	name="tab[]" si traitement tableau attendu, sinon name uniques
Labels	Tous liés (for="id" ou imbriqués), id unique sur chaque champ
Validation HTML5	required, type="email/number/tel", pattern, min/max/length
Accessibilité	<code><fieldset>/<legend></code> pour groupes, contraste suffisant, focus visible
Envoi fichier	Si type="file" → enctype="multipart/form-data" sur <code><form></code>

LE SAVAIS-TU ?**Le premier formulaire du Web et le Cameroun**

Le tout premier formulaire web standardisé (HTML 2.0, 1995) servait à... rechercher dans un index de documents (balise `<isindex>` obsolète, ancêtre de `<input type="search">`). Aujourd'hui, au Cameroun, les formulaires sont au cœur de la dématérialisation : inscription au concours de l'ENS sur `minefop.gov.cm`, déclaration d'impôts sur `impots.cm`, demande de visa, paiement factures CAMWATER/ENEO via Mobile Money (API USSD/Web). Un formulaire HTML mal codé (champ sans name, method="get" pour un paiement) = service public inaccessible ou transaction échouée. La rigueur que vous apprenez ici a un impact citoyen direct.

EXERCICE 3**Entraînement : Formulaire « Club Informatique »**

Créez le fichier `club.html` pour l'inscription au club informatique du lycée. Champs :

1. Prénom + Nom (deux champs texte, requis).
2. Email (type `email`, requis).
3. Sexe (Radio : F / M / Autre, requis).
4. Langues connus (Checkbox : Python, C, JavaScript, HTML/CSS, Autre — name="langages[]").
5. Niveau (Select : Débutant, Intermédiaire, Avancé).
6. Motivation (Textarea, 4 lignes, max 300 caractères).
7. Bouton « S'inscrire » (submit) et « Recommencer » (reset).

Lien CSS `css/club.css`, action `php/traitement_club.php`, méthode POST. Indice : Utilisez `<fieldset>` pour Sexe et Langues.

CORRIGÉ Exercice : Formulaire « Club Informatique »

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Inscription Club Informatique</title>
  <link rel="stylesheet" href="css/club.css">
</head>
<body>
  <h1>Club Informatique – Lycée Bilingue</h1>
  <form action="php/traitement_club.php" method="post">
    <div class="champ">
      <label for="prenom">Prénom *</label>
      <input type="text" id="prenom" name="prenom" required>
    </div>
    <div class="champ">
      <label for="nom">Nom *</label>
      <input type="text" id="nom" name="nom" required>
    </div>
    <div class="champ">
      <label for="email">Email *</label>
      <input type="email" id="email" name="email" required>
    </div>
    <fieldset>
      <legend>Sexe *</legend>
      <div class="radio-group">
        <label><input type="radio" name="sexe" value="F" required> Féminin</label>
        <label><input type="radio" name="sexe" value="M"> Masculin</label>
        <label><input type="radio" name="sexe" value="autre"> Autre / Préfère ne pas dire</label>
      </div>
    </fieldset>
    <fieldset>
      <legend>Langages connus</legend>
      <div class="check-group">
        <label><input type="checkbox" name="langages[]" value="Python"> Python</label>
        <label><input type="checkbox" name="langages[]" value="C"> C</label>
        <label><input type="checkbox" name="langages[]" value="JS"> JavaScript</label>
        <label><input type="checkbox" name="langages[]" value="HTML"> HTML/CSS</label>
        <label><input type="checkbox" name="langages[]" value="autre"> Autre</label>
      </div>
    </fieldset>
    <div class="champ">
      <label for="niveau">Niveau *</label>
      <select id="niveau" name="niveau" required>
        <option value="">-- Sélectionner --</option>
```

```
<option value="debutant">Débutant</option>
<option value="intermediaire">Intermédiaire</option>
<option value="avance">Avancé</option>
</select>
</div>

<div class="champ">
  <label for="motivation">Motivation (max 300 car.)</label>
  <textarea id="motivation" name="motivation" rows="4" maxlength="300" placeholder="
Pourquoi rejoindre le club ?"></textarea>
</div>

<div class="champ boutons">
  <button type="submit">S'inscrire</button>
  <button type="reset">Recommencer</button>
</div>
</form>
</body>
</html>
```

4

ORGANISATION DU DOSSIER DU SITE

Après avoir appris à structurer le contenu avec les tableaux, à enrichir nos pages avec des médias et à interagir avec l'utilisateur grâce aux formulaires, nous voici face à une réalité incontournable : un site web réel ne se résume pas à un unique fichier `index.html` posé sur le Bureau. Il comporte des dizaines, voire des centaines de fichiers : pages `.html`, feuilles de style `.css`, scripts `.js`, images, vidéos, sons, polices, documents `.pdf`, etc. Sans organisation rigoureuse, le projet devient ingérable, les liens se brisent et le déploiement sur un serveur (chez CAMTEL, un hébergeur local ou GitHub Pages) tourne au cauchemar. Cette section te donne la méthode professionnelle pour structurer ton dossier de travail, comprendre les chemins relatifs et adopter des conventions de nommage qui t'épargneront des heures de débogage.

4.1 Pourquoi organiser : le désordre est l'ennemi du développeur

Imagine un cybercafé à Yaoundé où les factures, les fiches clients, les mots de passe Wi-Fi et les manuels de réparation sont tous mélangés dans un seul tiroir. Trouver l'information prendrait une éternité. C'est exactement ce qui arrive si tu places tous tes fichiers (`accueil.html`, `contact.html`, `logo.png`, `style.css`, `video_promo.mp4`, `son_alerte.mp3`) à la racine d'un unique dossier.

À RETENIR**Avantages d'une arborescence propre**

- **Maintenabilité** : Tu localises instantanément un fichier pour le modifier.
- **Collaboration** : Tes camarades (ou toi dans six mois) comprennent l'architecture du

projet en un coup d'œil.

- Déploiement sans surprise : Le passage du poste de développement (ton ordinateur) au serveur de production conserve les chemins d'accès intacts.
- Séparation des responsabilités : Le contenu (HTML), la présentation (CSS), le comportement (JS) et les ressources (médias) sont isolés logiquement.

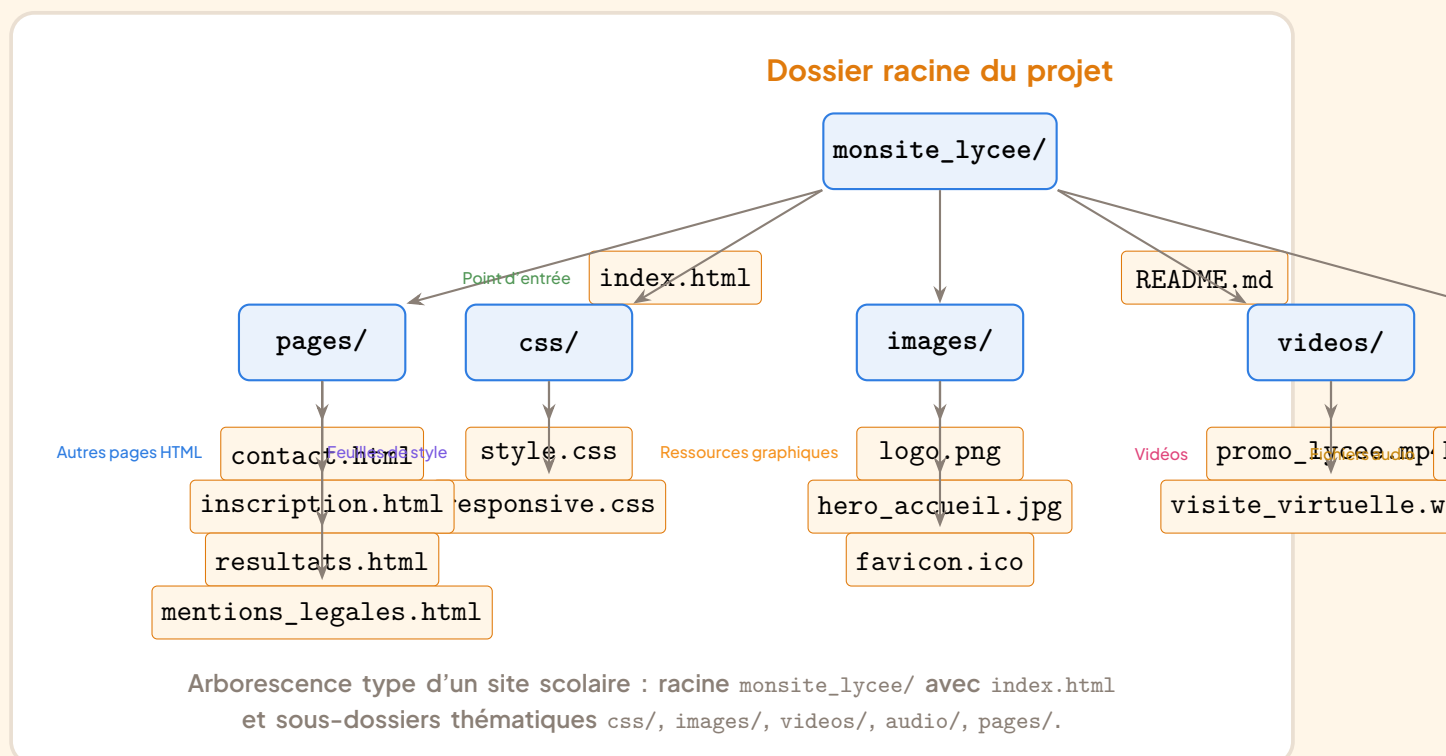
4.2 L'arborescence type : racine, page d'accueil et sous-dossiers

La convention universelle veut que le dossier principal du projet — appelé **dossier racine** — contienne obligatoirement un fichier nommé `index.html`. C'est la **page d'accueil** : quand un visiteur tape l'URL de base (ex. `http://www.lycee-bilingue-yaounde.cm/`), le serveur web renvoie automatiquement ce fichier sans que l'utilisateur ait à le préciser.

DÉFINITION Dossier racine et page d'accueil

Le dossier racine (ou root folder) est le répertoire qui contient l'ensemble des fichiers et sous-dossiers constituant le site web. La page d'accueil est le fichier `index.html` placé directement à la racine. C'est le point d'entrée par défaut du site.

Autour de ce noyau, on crée des sous-dossiers thématiques. Voici l'architecture recommandée pour un site scolaire (lycée, bibliothèque, association d'anciens élèves) :



PROPRIÉTÉ Rôles des sous-dossiers standards

Sous-dossier	Contenu	Exemple
css/	Feuilles de style (design, responsive)	style.css, print.css
images/	Photos, icônes, logos, favicon, schémas	logo.png, hero.jpg, favicon.ico
videos/	Fichiers vidéo (MP4, WebM, OGG)	promo.mp4, tuto.webm
audio/	Fichiers son (MP3, WAV, OGG)	hymne.mp3, notification.ogg
pages/	Pages HTML secondaires (sauf index.html)	contact.html, inscription.html
js/ (optionnel)	Scripts JavaScript	main.js, validation.js
fonts/ (optionnel)	Polices locales (WOFF2)	roboto.woff2
assets/ (alternative)	Regroupe images, videos, audio, fonts	Structure plate pour petits sites

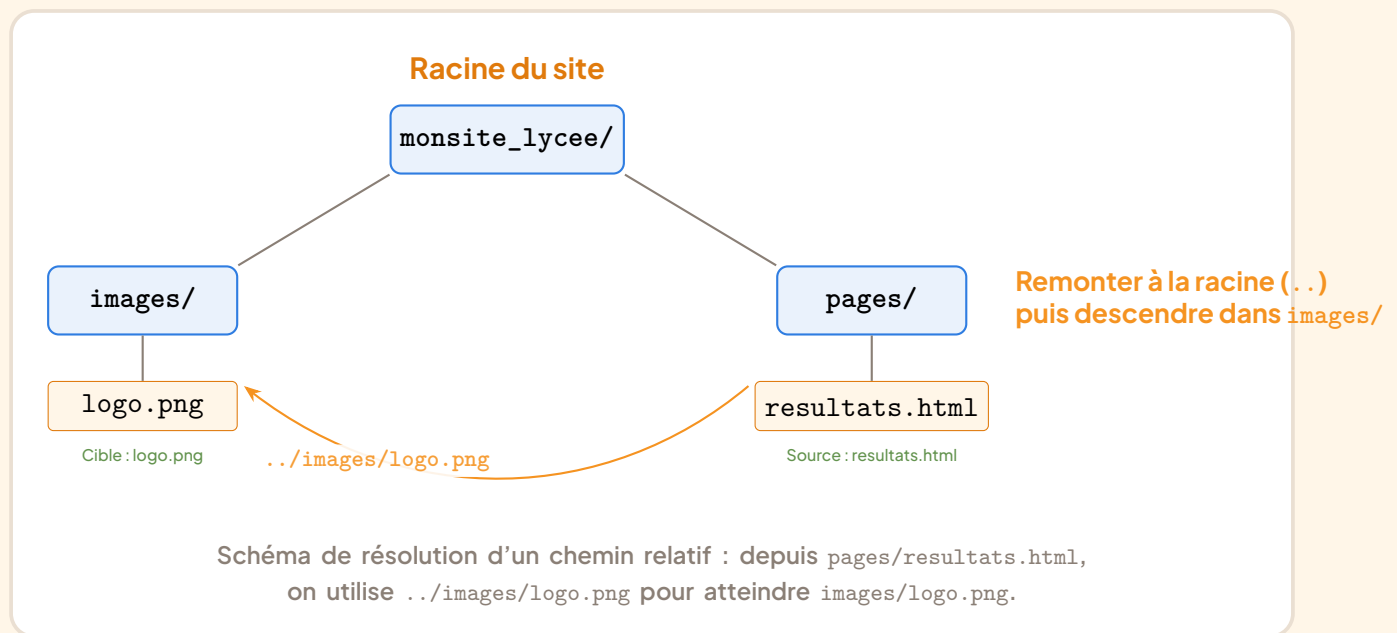
4.3 Les chemins relatifs : la clé des liens qui tiennent la route

Un **chemin relatif** indique la position d'un fichier cible par rapport au fichier qui contient le lien (le fichier source). C'est la seule méthode robuste pour un site destiné à être déplacé (copié sur clé USB, envoyé par WeTransfer, déployé sur un serveur). Les chemins absolus (commençant par / ou file:///C:/Users/...) cassent dès que le site change de machine.

DÉFINITION Chemin relatif vs chemin absolu

- **Chemin relatif** : Ne commence pas par / ni par un protocole (http://, file://). Il se résout à partir du dossier du fichier courant. Exemple : images/logo.png ou ../index.html.
- **Chemin absolu (sur serveur)** : Commence par / (racine du serveur web). Exemple : /images/logo.png. Utile seulement en production sur un domaine connu.
- **Chemin absolu (système de fichiers local)** : Chemin complet du disque dur (C:/Users/Prof/monsite/images/logo.png). Interdit dans le code HTML destiné au web.

La notation .. signifie « **remonter d'un niveau** » (aller dans le dossier parent). On peut l'enchaîner : ../ ../ remonte de deux niveaux.

**EXEMPLE** Cas pratiques de chemins relatifs

Supposons l'arborescence de la figure ci-dessus. Voici comment écrire les attributs `src` (pour `<img>`, `<video>`, `<audio>`, `<script>`) ou `href` (pour `<a>`, `<link>`) selon la position du fichier source :

Fichier source	Cible	Chemin relatif à écrire
<code>index.html</code> (racine)	<code>images/logo.png</code>	<code>images/logo.png</code>
<code>index.html</code> (racine)	<code>css/style.css</code>	<code>css/style.css</code>
<code>pages/contact.html</code>	<code>index.html</code> (racine)	<code>../index.html</code>
<code>pages/contact.html</code>	<code>images/logo.png</code>	<code>../images/logo.png</code>
<code>pages/resultats.html</code>	<code>css/style.css</code>	<code>../css/style.css</code>
<code>css/style.css</code> (dans <code>CSS : background-image</code>)	<code>../images/bg.png</code>	<code>../images/bg.png</code>

Règle mnémotechnique : `../` = « sors de mon dossier actuel ». Répète autant de fois qu'il y a de niveaux à remonter.

4.4 Règles de nommage : la discipline qui évite les bugs

Sur ton ordinateur (Windows), `Ma Photo.jpg`, `ma photo.jpg` et `ma_photo.jpg` désignent le même fichier. Sur un serveur Linux (la très grande majorité des hébergements web, y compris ceux du Cameroun), ce sont **trois fichiers différents** car le système est **sensible à la casse** (majuscules/minuscules) et ne tolère pas les espaces dans les URLs sans encodage (`%20`).

PROPRIÉTÉ Convention de nommage universelle (Best Practice)

1. Tout en minuscules : `page_contact.html`, `logo_ecole.png`, `style_principal.css`.
2. Pas d'espaces : Remplace l'espace par un tiret bas `_` (recommandé) ou un tiret `-`. Jamais `ma page.html`.
3. Pas d'accents, pas de caractères spéciaux : `inscription.html` (pas `inscription_élèves.html`), `mentions_legales.html` (pas `mentions_légales.html`).
4. Noms descriptifs et courts : `resultats_probatoire_2025.html` vaut mieux que `page1.html` ou `r.html`.
5. Extension en minuscules : `.html`, `.css`, `.js`, `.png`, `.jpg`, `.mp4`, `.mp3`.

ATTENTION Erreur fatale : espaces et accents dans les noms

```
<!-- MAUVAIS : espaces et accents -->

<a href="pages/inscription élèves.html">Inscription</a>
<link rel="stylesheet" href="css/ma Feuille de Style.css">

<!-- BON : minuscules, tirets bas, sans accents -->

<a href="pages/inscription_eleves.html">Inscription</a>
<link rel="stylesheet" href="css/ma_feuille_de_style.css">
```

Si tu laisses des espaces, le navigateur encode l'URL en `photo%20de%20classe.jpg`. Si le fichier physique sur le disque s'appelle vraiment `photo de classe.jpg`, le serveur Linux ne le trouvera pas car il cherche littéralement `photo%20de%20classe.jpg`. Résultat : image cassée (icône déchirée), lien mort (erreur 404), style non appliqué.

4.5 Méthode : organiser son site en 4 étapes

Applique cette procédure systématiquement à chaque nouveau projet (TP, site personnel, site pour l'association des parents d'élèves).

MÉTHODE Créer une structure de site web propre et portable

1. Crée le dossier racine sur ton disque (ex. `D:/TP_HTML/lycee_bilingue/`). Donne-lui un nom court, sans espaces, en minuscules (ex. `monsite_lycee`).
2. Crée les sous-dossiers standards à l'intérieur : `css/`, `images/`, `videos/`, `audio/`, `pages/` (et `js/` si besoin).

3. Crée le fichier `index.html` directement dans la racine. C'est ta page d'accueil. Ne le mets pas dans `pages/`.
4. Place tes ressources dans les bons sous-dossiers (images dans `images/`, etc.).
5. Écris ton code HTML en utilisant uniquement des chemins relatifs depuis le fichier qui les appelle. Teste chaque lien et chaque média en ouvrant `index.html` dans ton navigateur (double-clic).

EXPÉRIENCE**TP guidé : Structurer le site « Bibliothèque CDI »**

Objectif : Mettre en place l'arborescence complète d'un site pour le Centre de Documentation et d'Information (CDI) d'un lycée, y compris pages, styles, images et médias.

Matériel / Logiciels : Éditeur de code (VS Code, Notepad++, Sublime Text), navigateur web (Firefox, Chrome), explorateur de fichiers Windows.

Manipulation :

1. Dans ton dossier `Documents`, crée un dossier nommé `site_cdi_lycee`.
2. À l'intérieur, crée les 5 sous-dossiers : `css`, `images`, `videos`, `audio`, `pages`.
3. Télécharge (ou crée) une image de logo (ex. `logo_cdi.png`) et une image d'affiche (ex. `affiche_cdi.jpg`) et place-les dans `images/`. Place une vidéo de présentation (ex. `presentation_cdi.mp4`) dans `videos/`.
4. Crée un fichier `style.css` vide dans `css/`.
5. Crée le fichier `index.html` à la racine avec le contenu minimal suivant :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>CDI – Lycée Bilingue de Yaoundé</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <header>
    
    <h1>Centre de Documentation et d'Information</h1>
  </header>
  <nav>
    <a href="pages/catalogue.html">Catalogue</a> |
    <a href="pages/nouveautes.html">Nouveautés</a> |
    <a href="pages/horaires.html">Horaires & Accès</a>
  </nav>
  <main>
    <section>
      <h2>Bienvenue au CDI</h2>
      <p>Consultez nos ouvrages, revues et ressources numériques.</p>
      <video src="videos/presentation_cdi.mp4" controls width="400" poster="images/affiche_cdi.jpg"></video>
```

```
        </section>
    </main>
    <footer>
        <p>&copy; ; 2025 Lycée Bilingue de Yaoundé – <a href="pages/mentions_legales.html">
        Mentions légales</a></p>
    </footer>
</body>
</html>
```

6. Crée le fichier `pages/catalogue.html` avec un lien de retour vers l'accueil :

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Catalogue – CDI</title>
    <link rel="stylesheet" href="../css/style.css">
</head>
<body>
    <header>
        
        <h1>Catalogue en ligne</h1>
    </header>
    <nav><a href="../index.html">← Retour à l'accueil</a></nav>
    <main>
        <!-- Contenu du catalogue ici -->
        <p>Recherche par titre, auteur, thème...</p>
    </main>
</body>
</html>
```

7. Ouvre `index.html` dans ton navigateur. Vérifie : le logo s'affiche-t-il ? Le lien vers `catalogue.html` fonctionne-t-il ? Le lien retour depuis `catalogue.html` fonctionne-t-il ? La vidéo (si fichier présent) se lit-elle ?

Observations attendues :

- Depuis `index.html` (racine) : `images/logo_cdi.png`, `css/style.css`, `pages/catalogue.html` fonctionnent sans `../`.
- Depuis `pages/catalogue.html` : il faut `../` pour sortir de `pages/` avant d'aller dans `images/`, `css/` ou la racine.
- Si tu renommes `logo_cdi.png` en `Logo CDI.png` (majuscules, espace), l'image disparaît dans le navigateur (erreur 404 dans l'inspecteur réseau) alors qu'elle s'affichait dans l'explorateur Windows.

Interprétation : Ce TP valide que les chemins relatifs dépendent uniquement de la position du fichier HTML source. L'organisation en sous-dossiers et le respect strict des conventions de nommage garantissent la portabilité du site : tu peux copier le dossier `site_cdi_lycee` sur une clé USB, l'envoyer à un camarade ou l'uploader sur un hébergeur (Netlify, GitHub Pages, serveur du lycée) sans changer une seule ligne de code.

4.6 Exemple résolu : corriger les chemins cassés d'un projet mal organisé

EXERCICE RÉSOLU 4 Réparation d'arborescence et de chemins

Énoncé : Un élève a créé un site pour la fête de l'excellence. Il a tout mis dans un dossier MonSiteFete/ mais sans sous-dossiers. Les fichiers sont : Accueil.html, Programme.html, Photo Groupe.jpg, Style.CSS, Video Fete.mp4. Dans Accueil.html, il a écrit :

```

<link rel="stylesheet" href="Style.CSS">
<video src="Video Fete.mp4" controls></video>
<a href="Programme.html">Voir le programme</a>
```

Il copie le dossier sur la clé USB de son professeur. Sur le poste du prof (Linux), rien ne s'affiche (pas de style, pas d'image, vidéo noire, lien mort). Explique pourquoi et propose la structure corrigée avec les bons noms et les bons chemins.

Solution détaillée :

1. Analyse des causes (3 erreurs majeures) :

- **Espaces dans les noms** : Photo Groupe.jpg, Video Fete.mp4. Sur Linux, le navigateur cherche Photo%20Groupe.jpg mais le fichier s'appelle Photo Groupe.jpg (avec un vrai espace). Incohérence → 404.
- **Majuscules incohérentes** : Accueil.html (maj A), Style.CSS (maj S, maj CSS). Linux distingue style.css de Style.CSS. Le lien href="Style.CSS" ne trouve pas style.css si le fichier réel est en minuscules, ou vice-versa.
- **Page d'accueil mal nommée** : Accueil.html au lieu de index.html. Le serveur web ne l'affichera pas automatiquement à l'URL de base.

2. Restructuration (application de la méthode) :

- **Dossier racine** : site_fete_excellence/
- **Sous-dossiers** : images/, css/, videos/, audio/, pages/
- **Renommage** (minuscules, tirets bas, pas d'accents) :

```
Accueil.html → index.html   (à la racine)
Programme.html → pages/programme.html
Photo Groupe.jpg → images/photo_groupe.jpg
Style.CSS → css/style.css
Video Fete.mp4 → videos/video_fete.mp4
(nouveau) → images/affiche_fete.jpg (pour l'attribut poster)
(nouveau) → audio/hymne_ecole.mp3 (pour la page programme)
```

3. Code corrigé pour index.html (racine) :

```
<!DOCTYPE html>
<html lang="fr">
<head>
```

```

<meta charset="UTF-8">
<title>Fête de l'Excellence 2025</title>
<link rel="stylesheet" href="css/style.css">
</head>
<body>
  <header>
    
    <h1>Fête de l'Excellence – Lycée de Ngaoundéré</h1>
  </header>
  <nav>
    <a href="pages/programme.html">Voir le programme détaillé</a>
  </nav>
  <main>
    <video src="videos/video_fete.mp4" controls width="640" poster="images/affiche_fete.jpg">
      Votre navigateur ne supporte pas la vidéo HTML5.
    </video>
  </main>
</body>
</html>

```

4. Code corrigé pour pages/programme.html :

```

<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Programme – Fête de l'Excellence</title>
  <link rel="stylesheet" href=" ../css/style.css">
</head>
<body>
  <header>
    
    <h1>Programme de la cérémonie</h1>
  </header>
  <nav>
    <a href=" ../index.html">< Retour à l'accueil</a>
  </nav>
  <main>
    <ul>
      <li>08h00 : Accueil des invités</li>
      <li>09h00 : Discours du Proviseur</li>
      <li>10h00 : Remise des prix</li>
      <li>12h00 : Cocktail & Animation musicale</li>
    </ul>
    <audio src=" ../audio/hymne_ecole.mp3" controls></audio>
  </main>
</body>
</html>

```

5. Vérification : Ouvre index.html → tout s'affiche. Clique sur « Voir le programme » → programme.html s'affiche avec style, image et lien retour fonctionnels. Le dossier site_fete_excellence/ est maintenant portable et professionnel.

4.7 Conséquence pratique : déplacer un fichier casse les liens

C'est la leçon la plus importante de cette section. **Les chemins relatifs sont calculés au moment où tu écris le code, en fonction de l'emplacement actuel du fichier source.** Si tu déplaces `contact.html` de `pages/` vers la racine sans modifier le code, les chemins `../css/style.css` et `../images/logo.png` deviennent invalides (ils cherchent `css/` et `images/` au-dessus de la racine, ce qui n'existe pas).

ATTENTION Piège du « glisser-déposer » dans l'explorateur de fichiers

Si tu réorganises ton projet après avoir écrit le code (ex. tu crées le dossier `pages/` et tu y glisses `contact.html` avec la souris), tu casses tous les chemins relatifs présents dans `contact.html`.

- **Solution 1 (préventive) :** Crée l'arborescence avant d'écrire la première ligne de code (Méthode étapes 1 et 2).
- **Solution 2 (corrective) :** Si tu dois déplacer un fichier, ouvre-le immédiatement dans ton éditeur de code et mets à jour tous les `src` et `href` (remplace `../css/` par `css/`, etc.).
- **Astuce VS Code :** Si tu utilises VS Code, le déplacement d'un fichier via l'explorateur latéral (panel gauche) met à jour automatiquement les chemins dans les autres fichiers qui y font référence (fonction « Rename Refactoring »). C'est un gain de temps énorme.

4.8 Le savais-tu ? : Le pouvoir secret de `index.html`

LESAVAIS-TU ? Pourquoi `index.html` est roi

Quand tu tapes `https://www.camtelecom.cm/` dans ton navigateur, tu ne précises pas de fichier. Le serveur web (Apache, Nginx, IIS, Caddy) reçoit la requête pour le répertoire racine `/`. Sa configuration par défaut (DirectoryIndex chez Apache, `index` chez Nginx) lui dit : « S'il y a un fichier nommé `index.html` (ou `index.php`, `index.htm`) dans ce dossier, envoie-le. Sinon, affiche la liste des fichiers (si autorisé) ou une erreur 403 Forbidden. »

C'est pour cela que chaque dossier de ton site qui doit avoir une page d'entrée visible directement via son URL (ex. `/inscription/`, `/resultats/`) doit contenir un `index.html`.

- URL `monsite.cm/` → sert `/index.html` (racine).
- URL `monsite.cm/inscription/` → sert `/inscription/index.html` (si tu as créé un dossier `inscription/` avec son propre `index.html`).

Cela permet des URLs propres (`.../inscription/` au lieu de `.../inscription.html`) et cache l'implémentation technique. C'est la base du routage moderne (utilisé par WordPress, Django, Laravel, Next.js, etc.).

4.9 Synthèse : les 5 commandements de l'organisation

À RETENIR Check-list avant de valider ton projet HTML

1. **Arborescence** : Racine / contient `index.html` + dossiers `css/`, `images/`, `videos/`, `audio/`, `pages/` (+`js/`).
2. **Nommage** : Tout en minuscules, tirets bas `_` pour les espaces, pas d'accents, extensions en minuscules.
3. **Chemins** : Relatifs uniquement. `../` pour remonter, dossier/fichier pour descendre. Jamais de `C:/Users/...` ni de `file:///`.
4. **Cohérence** : Le chemin écrit dans `src` / `href` correspond exactement à la position réelle du fichier cible par rapport au fichier source.
5. **Test** : Ouvre `index.html` en double-clic. Vérifie tous les liens, toutes les images, tous les médias. Ouvre la console développeur (F12 → onglet Réseau / Network) pour voir les erreurs 404 (fichier non trouvé) en rouge.

4.10 Exercices d'entraînement

EXERCICE 4 Arborescence et chemins

On donne l'arborescence suivante pour un site de gestion de notes :

```
notes_lycee/  
  index.html  
  css/  
    style.css  
  images/  
    logo.png  
    fond.jpg  
  pages/  
    bulletin.html  
    connexion.html  
  js/  
    main.js
```

Écrire les chemins relatifs exacts à utiliser dans les fichiers suivants :

1. Dans `index.html` : charger `style.css`, afficher `logo.png`, lien vers `pages/connexion.html`.
2. Dans `pages/bulletin.html` : charger `style.css`, afficher `logo.png`, lien vers `index.html`, charger `main.js`.

3. Dans `css/style.css` (pour une image de fond `background-image`) : charger `../images/fond.jpg`.

EXERCICE 5 Correction d'erreurs

Identifie et corrige les 4 erreurs dans ce fragment de code (suppose l'arborescence standard avec `index.html` à la racine, `images/`, `css/`, `pages/`) :

```
<!-- Fichier : pages/inscription.html -->
<link rel="stylesheet" href="css/style.css">

<a href="index.html">Accueil</a>
<script src="js/validation.js"></script>
```

EXERCICE 6 Nommage

Propose un nom de fichier correct (convention) pour chacun de ces cas :

1. Photo de la remise des prix du 14 juillet 2025.
2. Feuille de style pour l'impression des bulletins.
3. Page HTML « Liste des admis au Probatoire ».
4. Vidéo de présentation du laboratoire de chimie.
5. Fichier son pour la sonnerie de fin de cours.

CORRIGÉ Exercice 1

1. `css/style.css`, `images/logo.png`, `pages/connexion.html`
2. `../css/style.css`, `../images/logo.png`, `../index.html`, `../js/main.js`
3. `../images/fond.jpg` (depuis `css/`, on remonte à la racine puis on descend dans `images/`)

CORRIGÉ Exercice 2

Le fichier source est `pages/inscription.html` (dans un sous-dossier). Tous les chemins doivent remonter d'un niveau (`../`).

```
<!-- CORRIGÉ -->
<link rel="stylesheet" href="../css/style.css">

<a href="../index.html">Accueil</a>
<script src="../js/validation.js"></script>
```

CORRIGÉ Exercice 3

1. remise_prix_14_juillet_2025.jpg
2. print_bulletin.css (ou style_print.css)
3. liste_admis_probatoire.html
4. presentation_lab0_chimie.mp4
5. sonnerie_fin_cours.mp3

5 CONTRIBUTION DU CSS

Dans la section précédente, nous avons appris à organiser le dossier de notre site : un dossier `images/` pour les fichiers multimédias, un dossier `css/` pour les feuilles de style, et les pages `.html` à la racine. Tu t'es peut-être demandé : à quoi servira exactement ce dossier `css/` ? C'est précisément l'objet de cette section. Jusqu'ici, nos pages HTML fonctionnent, mais elles sont austères : texte noir sur fond blanc, liens bleus soulignés, tableaux sans bordures visibles, aucune couleur, aucune personnalité. Le CSS va transformer ces pages ternes en pages attrayantes, sans toucher à leur contenu.

5.1 Constat : les limites du HTML seul

Ouvre dans ton navigateur la page contenant le tableau des notes de la classe que nous avons construit dans la première section de cette leçon. Que observes-tu ?

- Le texte est noir, dans la police par défaut du navigateur ;
- les liens sont bleus (ou violets une fois visités) et soulignés ;
- le tableau n'a **aucune bordure visible** : les lignes et colonnes se devinent à peine ;
- tout est collé au bord gauche de la fenêtre, sans espacement harmonieux.

Ce constat n'est pas un défaut : c'est un **choix de conception**. Le langage HTML a été conçu pour décrire la structure et le sens du contenu (ceci est un titre, ceci est un tableau, ceci est un formulaire), et non son apparence. Pour l'apparence, le Web dispose d'un langage spécialisé : le CSS.

5.2 Définition et rôle du CSS

DÉFINITION CSS — Cascading Style Sheets

Le **CSS** (Cascading Style Sheets, en français « feuilles de style en cascade ») est un langage de mise en forme qui décrit la présentation d'un document HTML : couleurs, polices de caractères, tailles, bordures, arrière-plans, espacements, alignements. Le terme « cascade » signifie que plusieurs règles de style peuvent s'appliquer à un même élément, et qu'un ordre de priorité (la cascade) détermine laquelle l'emporte.

La philosophie du Web moderne repose sur une **séparation stricte des rôles** :

Langage	Rôle	Exemples
HTML	Structure et contenu de la page	titres, paragraphes, tableaux, formulaires, images
CSS	Présentation et habillage	couleurs, polices, bordures, marges, alignements

PROPRIÉTÉ Séparation contenu / présentation (admise)

La séparation du contenu (HTML) et de la présentation (CSS) facilite la maintenance d'un site et garantit son homogénéité : toutes les pages liées à la même feuille de style partagent automatiquement la même apparence, et une seule modification dans le fichier CSS met à jour l'ensemble du site. Nous admettons cette propriété et nous l'illustrerons concrètement à la fin de la section.

HTML : le squelette

```
<h1>Lycée de Ngaoundéré</h1>
<table>
<tr><td>Amina</td>
<td>15/20</td></tr>
</table>
```

CSS : l'habillage

```
h1 { color: navy; }
table { border: 1px solid; }
td { padding: 6px; }
```

Page web affichée : structurée ET élégante

Deux rôles complémentaires : HTML fournit le squelette (structure et contenu), CSS fournit l'habillage (présentation). Le navigateur combine les deux pour afficher la page finale.

5.3 Syntaxe d'une règle CSS

Une feuille de style est constituée de **règles**. Chaque règle suit toujours la même structure :

```
selecteur {  
    propriete1 : valeur1 ;  
    propriete2 : valeur2 ;  
}
```

- le **sélecteur** désigne les éléments HTML auxquels la règle s'applique (`h1` pour tous les titres de niveau 1, `table` pour tous les tableaux, etc.);
- chaque **déclaration** est un couple `propriété : valeur ;` — n'oublie jamais le point-virgule final;
- les accolades `{ }` encadrent l'ensemble des déclarations de la règle.

EXEMPLE Premières règles CSS

```
/* Tous les titres h1 en bleu marine, centres */  
h1 {  
    color : navy ;  
    text-align : center ;  
}  
  
/* Tous les paragraphes en police Arial, taille 14 pixels */  
p {  
    font-family : Arial, sans-serif ;  
    font-size : 14px ;  
}
```

ATTENTION Pièges de syntaxe CSS

Trois erreurs reviennent sans cesse chez les débutants :

1. oublier le point-virgule à la fin d'une déclaration : la déclaration suivante est alors ignorée;
2. utiliser le signe `=` au lieu des deux-points : on écrit `color : red ;` et non `color = red ;`;
3. mal orthographier le nom d'une propriété : le navigateur ignore silencieusement la déclaration fautive, sans message d'erreur — d'où l'importance de vérifier attentivement.

5.4 Les trois façons d'appliquer du CSS

Il existe exactement trois méthodes pour appliquer du CSS à une page HTML. Retiens bien leurs

noms, leurs avantages et leurs limites.

1. Le style en ligne (attribut `style`). On écrit le CSS directement dans la balise ouvrante d'un élément, grâce à l'attribut `style` :

```
<h1 style="color : navy ; text-align : center ;">
  Bienvenue au Lycée de Ngaoundéré
</h1>
```

Cette méthode est rapide pour un essai, mais elle mélange contenu et présentation et doit être répétée sur chaque élément : à réserver aux tests ponctuels.

2. La feuille de style interne (balise `<style>`). On regroupe les règles dans une balise `<style>` placée dans le `<head>` de la page :

```
<head>
  <meta charset="UTF-8">
  <title>Notes de la classe</title>
  <style>
    h1 { color : navy ; text-align : center ; }
    table { border-collapse : collapse ; }
    td, th { border : 1px solid black ; padding : 6px ; }
  </style>
</head>
```

Le style s'applique à toute la page, mais pas aux autres pages du site.

3. La feuille de style externe (fichier `.css`). C'est la **méthode recommandée** : on écrit les règles dans un fichier séparé (par exemple `css/style.css`, dans le dossier prévu à la section précédente) et on le lie à chaque page.

MÉTHODE**Lier une feuille de style externe à une page HTML**

1. Crée dans le dossier `css/` de ton site un fichier nommé `style.css` et écris-y tes règles (sans balise `<style>`, uniquement du CSS).
2. Dans le `<head>` de chaque page HTML du site, insère la ligne :

```
<link rel="stylesheet" href="css/style.css">
```
3. Enregistre les deux fichiers, puis recharge la page dans le navigateur (touche F5) : le style s'applique immédiatement.

L'attribut `rel="stylesheet"` indique la nature du lien, et `href` donne le chemin relatif vers le fichier CSS — d'où l'importance de l'organisation du dossier vue précédemment.

Méthode	Avantage	Limite
En ligne (<code>style</code>)	test rapide sur un élément	répétée partout, mélange contenu et style
Interne (<code><style></code>)	centralisée pour une page	ne s'applique qu'à une seule page
Externe (<code><link></code>)	un seul fichier pour tout le site	nécessite un fichier supplémentaire

À RETENIR L'avantage décisif du fichier externe

Avec une feuille de style externe, une seule modification met à jour tout le site. Si le professeur décide que la couleur officielle du site du lycée passe du bleu au vert, il suffit de changer `color : navy ;` en `color : green ;` dans `style.css` : les dix, cent ou mille pages liées à cette feuille changent instantanément d'apparence. C'est l'illustration concrète de la propriété de séparation contenu/présentation.

5.5 Exemple complet : embellir le tableau des notes

Reprenons le tableau des notes de la classe de Première C construit plus tôt. Voici d'abord la page HTML, liée à une feuille externe :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Notes d'informatique – Première C</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <h1>Notes d'informatique – Sequence 2</h1>
  <table>
    <tr>
      <th>Nom</th><th>Note / 20</th>
    </tr>
    <tr><td>Amina</td><td>15</td></tr>
    <tr><td>Brice</td><td>12</td></tr>
    <tr><td>Carine</td><td>17</td></tr>
    <tr><td>Dieudonné</td><td>9</td></tr>
  </table>
</body>
</html>
```

Et voici le fichier `css/style.css` commenté :

```
/* ===== Fichier css/style.css ===== */

/* Titre principal : bleu marine et centre */
h1 {
  color : navy ;
  text-align : center ;
}
```

```

/* Tableau : largeur 60 %, centre, bordures fusionnees */
table {
    width : 60% ;
    margin : auto ;           /* centre le tableau */
    border-collapse : collapse ; /* une seule bordure entre cellules */
}

/* Cellules : bordure fine noire et espacement interieur */
th, td {
    border : 1px solid black ;
    padding : 6px ;
    text-align : center ;
}

/* Ligne d'en-tete : fond bleu, texte blanc */
th {
    background-color : navy ;
    color : white ;
}

/* Alternance des couleurs de lignes (lisibilite) */
tr:nth-child(even){
    background-color : #DCE6F1 ; /* bleu tres clair */
}

```

Deux propriétés méritent une explication :

- `border-collapse : collapse ;` **fusionne** les bordures des cellules voisines en une seule ligne (sans elle, chaque cellule a sa propre bordure et le tableau paraît « doublement quadrillé ») ;
- `tr:nth-child(even)` est un sélecteur qui cible les **lignes paires** du tableau : c'est l'astuce classique pour alterner les couleurs de fond et améliorer la lisibilité, comme sur les relevés de notes officiels.

Sans CSS

Notes d'informatique
 Nom Note / 20
 Amina 15
 Brice 12
 Carine 17
 Dieudonné 9

texte noir, aucune bordure

+ CSS →

Avec CSS

Notes d'informatique

Nom	Note
Amina	15
Brice	12
Carine	17

bordures, en-tête colorée, lignes alternées

Avant / après : le même tableau de notes, sans CSS (à gauche) puis habillé par la feuille `style.css` (à droite). Le code HTML n'a pas changé d'une ligne.

EXERCICE RÉSOLU 5

Colorer un titre et encadrer un tableau

Énoncé. La page `index.html` du site du club informatique contient un titre `<h1>` et un tableau des membres. On te demande : (a) d'afficher le titre en vert foncé et centré ; (b) d'encadrer le tableau avec des bordures fusionnées ; (c) de colorer la ligne d'en-tête en vert avec du texte blanc. Tu utiliseras une feuille de style externe.

SOLUTION

Solution détaillée. Étape 1 : créer le fichier `css/style.css` avec les trois règles demandées :

```
h1 {
    color : darkgreen ;    /* (a) titre vert foncé */
    text-align : center ;
}
table {
    border-collapse : collapse ; /* (b) bordures fusionnées */
}
th, td {
    border : 1px solid black ;    /* (b) encadre chaque cellule */
    padding : 5px ;
}
th {
    background-color : green ; /* (c) en-tête verte */
    color : white ;           /* (c) texte blanc */
}
```

Étape 2 : lier la feuille dans le `<head>` de la page :

```
<link rel="stylesheet" href="css/style.css">
```

Étape 3 : enregistrer et recharger la page avec F5. Vérification : le titre est vert et centré, le tableau est quadrillé, son en-tête est vert à texte blanc. Si rien ne change, vérifier le chemin `href` et le nom du fichier.

ATTENTION Ne confonds pas les rôles !

Une erreur fréquente consiste à utiliser d'anciennes balises HTML de présentation, comme `<font color="red">` pour colorer un texte ou `<center>` pour centrer un titre. Ces balises sont obsolètes : elles ne font plus partie du HTML moderne, car elles mélangent contenu et présentation — exactement ce que le CSS cherche à éviter. La règle d'or : toute question d'apparence (couleur, centrage, taille, bordure) se résout en CSS, jamais en HTML.

EXPÉRIENCE TP guidé : habiller la page du lycée

Objectif : constater concrètement la puissance de la séparation contenu/présentation.

Matériel : un ordinateur avec un éditeur de texte (Notepad++, VS Code) et un navigateur.

1. Crée le dossier `site_lycee/` avec un sous-dossier `css/`, puis la page `index.html` contenant un titre `<h1>`, un paragraphe et un tableau de trois élèves. Ouvre-la dans le navigateur : observe l'aspect austère.
2. Crée `css/style.css` avec les règles de l'exemple de la section (titre coloré, tableau encadré, en-tête colorée, lignes alternées) et lie-le à la page. Recharge : observe la transformation.
3. Sans toucher au fichier HTML, remplace dans `style.css` `navy` par `darkred` et recharge : toute la page change de thème en une seule modification.
4. Duplique la page en `membres.html`, ajoute-lui le même `<link>` : elle hérite instantanément du même style. C'est l'homogénéité du site.

Interprétation : le contenu (HTML) est resté identique du début à la fin ; seule la présentation (CSS) a varié. Tu viens de vérifier expérimentalement la propriété de séparation contenu/présentation.

LE SAVAIS-TU ? CSS Zen Garden : un seul HTML, mille visages

En 2003, le concepteur canadien Dave Shea a lancé le site CSS Zen Garden : une unique page HTML, que personne n'a le droit de modifier, habillée par des centaines de feuilles CSS différentes envoyées par des designers du monde entier. Résultat spectaculaire : le même contenu apparaît tantôt comme un journal ancien, tantôt comme une affiche futuriste, tantôt comme un jardin japonais — seul le fichier CSS change. Ce site de démonstration a convaincu le monde entier de la puissance de la séparation contenu/présentation, et il reste aujourd'hui encore une référence enseignée dans les écoles d'informatique.

EXERCICE 7 À toi de jouer

Le site du cybercafé « Espace Numérique » de Bafoussam possède trois pages : `accueil.html`, `tarifs.html` et `contact.html`.

1. Explique pourquoi il est préférable d'utiliser une feuille de style externe plutôt qu'une balise `<style>` dans chaque page.
2. Écris la ligne HTML à insérer dans chaque page pour lier la feuille `css/cyber.css`.
3. Écris les règles CSS pour : titres `h1` en orange et centrés ; tableaux avec bordures fusionnées ; lignes paires des tableaux sur fond gris clair (`#EEEEEE`).
4. Le gérant veut passer la couleur principale de l'orange au bleu. Combien de fichiers dois-tu modifier ? Justifie.

CORRIGÉ Exercice — À toi de jouer

1. Avec une feuille externe, les trois pages partagent le même fichier : le style est homogène sur tout le site et une seule modification le met à jour partout. Avec une balise `<style>` par page, il faudrait recopier puis modifier le style trois fois, avec un risque d'incohérence.
2. Dans le `<head>` de chaque page : `<link rel="stylesheet" href="css/cyber.css">`.
3. Règles CSS :

```
h1 { color : orange ; text-align : center ; }
table { border-collapse : collapse ; }
th, td { border : 1px solid black ; padding : 5px ; }
tr:nth-child(even) { background-color : #EEEEEE ; }
```
4. Un seul fichier : `css/cyber.css`. Il suffit de remplacer orange par blue (ou navy) : les trois pages liées changent automatiquement. C'est l'avantage décisif de la feuille externe.

Nous disposons désormais de tous les ingrédients : structure (titres, tableaux, formulaires), multimédia (images, sons, vidéos), organisation des dossiers et habillage CSS. La prochaine section, la synthèse, te guidera pour assembler tout cela et construire une page web complète et professionnelle.

6**SYNTHÈSE : CONSTRUIRE UNE PAGE COMPLÈTE**

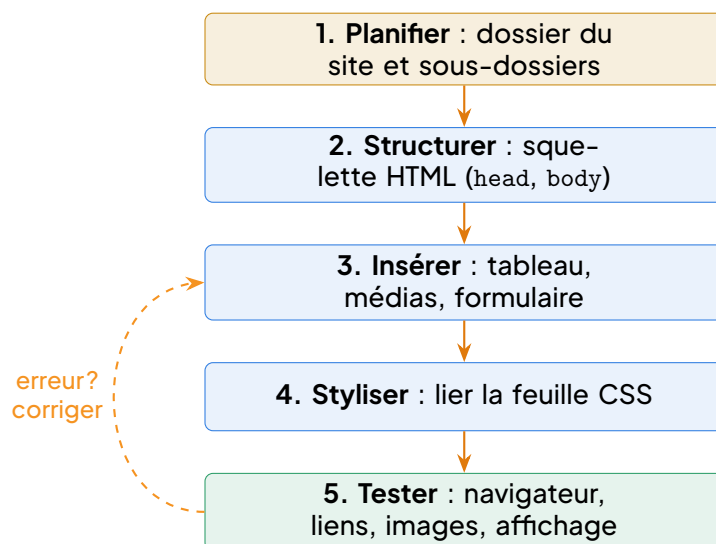
Dans la section précédente, tu as découvert la contribution du CSS : séparer le contenu (HTML) de la présentation (couleurs, polices, marges) grâce à une feuille de style liée par la balise `<link>`. Tu disposes maintenant de toutes les pièces du puzzle : la structure d'une page, les tableaux, les fichiers multimédias, les formulaires et l'organisation du dossier du site. Il reste à assembler ces pièces dans le bon ordre. C'est exactement la compétence attendue à l'épreuve pratique : on te donne une situation réelle (par exemple « le proviseur veut publier les résultats du lycée sur le site de l'établissement ») et tu dois produire une page complète, correcte et testée. Cette section te donne la démarche globale, une étude de cas fil rouge, les méthodes de vérification et l'entraînement à l'exercice classique du code à trous.

6.1 Démarche globale de création d'une page web

Un débutant ouvre son éditeur et tape du code au hasard ; un bon élève de Première suit une démarche. Pourquoi ? Parce qu'une page web n'est pas un texte qu'on écrit d'un trait : c'est un petit projet qui mélange des fichiers (HTML, CSS, images, sons), des chemins relatifs et des balises qui doivent toutes être fermées. Sans plan, on obtient une page qui « marche à moitié » : image absente, tableau déformé, formulaire muet.

MÉTHODE Les 5 étapes de création d'une page web

1. **Planifier l'arborescence** : créer le dossier du site et ses sous-dossiers (images/, css/, sons/) avant d'écrire la moindre ligne de code.
2. **Écrire la structure HTML** : le squelette (`<!DOCTYPE html>`, `<html>`, `<head>`, `<body>`), le titre de la page et les titres de sections.
3. **Insérer le contenu** : le tableau (`<table>`, `<tr>`, `<th>`, `<td>`), les médias (`<img>`, `<audio>`, `<video>`) et le formulaire (`<form>`, `<input>`, `<label>`).
4. **Lier le CSS** : créer la feuille de style et la rattacher avec `<link rel="stylesheet" href="css/style.css">` dans le `<head>`.
5. **Vérifier et tester** : ouvrir la page dans le navigateur, contrôler chaque image, chaque lien, l'affichage du tableau, puis corriger.



Organigramme de la démarche de création d'une page web en 5 étapes. La flèche en pointillés symbolise le retour en arrière quand le test révèle une erreur.

ATTENTION L'ordre des étapes n'est pas un détail

L'erreur classique consiste à écrire le code d'abord et à créer les dossiers ensuite : tous les chemins relatifs (images/photo.png) deviennent faux et les images n'apparaissent plus. Crée toujours l'arborescence avant de coder, et place chaque fichier à sa place définitive dès le départ.

6.2 Étude de cas fil rouge : la page « Résultats du lycée »

Le proviseur du Lycée Bilingue de Bafoussam souhaite publier sur le site de l'établissement une page présentant les taux de réussite au Probatoire, avec une photo de la cérémonie de remise des prix et un formulaire permettant aux parents de poser une question. Voici comment mener ce projet de bout en bout.

Étape 1 : l'arborescence. On crée le dossier `site_lycee/` contenant :

```
site_lycee/
|-- index.html      (la page "Resultats du lycee")
|-- css/
|   '-- style.css   (la feuille de style)
|-- images/
|   '-- remise_prix.jpg
'-- sons/
    '-- hymne.mp3
```

Étapes 2 à 4 : le code. Voici le squelette complet et commenté de la page.

EXEMPLE Squelette complet de la page « Résultats du lycée »

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Resultats du Lycee Bilingue de Bafoussam</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <h1>Resultats du Probatoire 2024</h1>
  
  <h2>Taux de reussite par serie</h2>
  <table border="1">
    <tr><th>Serie</th><th>Presents</th><th>Admis</th><th>Taux</th></tr>
    <tr><td>C</td><td>48</td><td>41</td><td>85,4 %</td></tr>
    <tr><td>D</td><td>52</td><td>39</td><td>75 %</td></tr>
    <tr><td>E</td><td>30</td><td>24</td><td>80 %</td></tr>
  </table>
  <h2>Contactez l'administration</h2>
  <form action="traitement.html" method="post">
    <label for="nom">Votre nom :</label>
    <input type="text" id="nom" name="nom" required>
    <label for="msg">Votre message :</label>
    <textarea id="msg" name="message" rows="4" cols="30"></textarea>
    <input type="submit" value="Envoyer">
  </form>
</body>
</html>
```

Observe la logique d'ensemble : le `<head>` contient le titre de l'onglet et le lien vers le CSS ; le `<body>` enchaîne titre, image, tableau puis formulaire. Chaque champ de saisie possède un attribut `name` : sans lui, la donnée saisie ne sera jamais transmise. L'attribut `alt` de l'image garantit

qu'un texte s'affiche si le fichier est absent. Remarque enfin que le tableau compte quatre colonnes (<th> en-têtes, puis quatre <td> par ligne) : chaque ligne doit avoir exactement le même nombre de cellules.

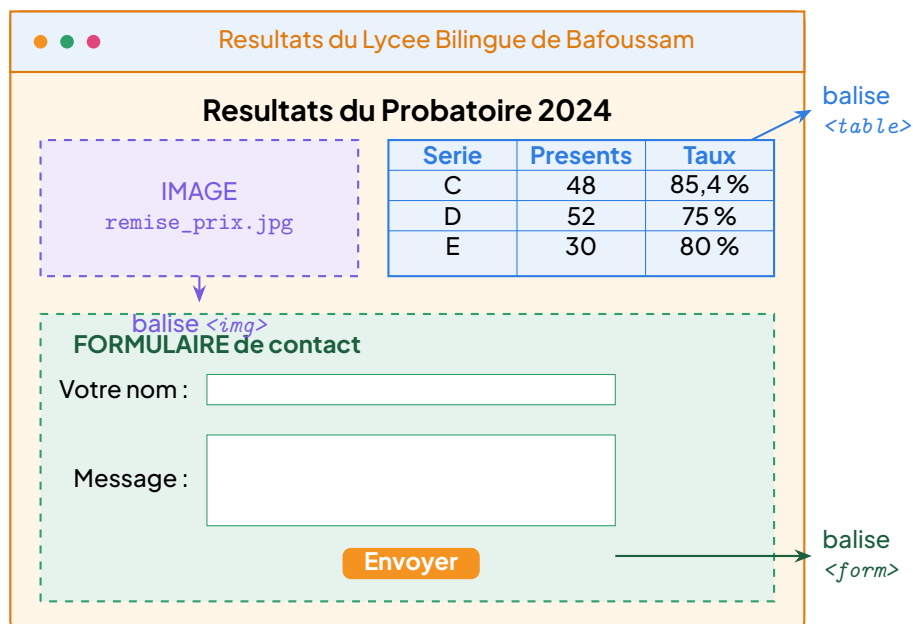


Schéma annoté de la page finale « Résultats du lycée » : la barre d'onglet affiche le <title>, puis on retrouve les trois zones de contenu (image, tableau, formulaire) construites dans le <body>.

6.3 Vérification et test de la page

Une page n'est jamais terminée tant qu'elle n'a pas été testée dans le navigateur. Le test n'est pas un coup d'œil rapide : c'est un contrôle méthodique, point par point.

MÉTHODE Checklist de validation d'une page web

- Structure** : la page s'ouvre sans message d'erreur ; le titre de l'onglet correspond au contenu de <title> ; les accents s'affichent correctement (sinon, vérifier <meta charset="UTF-8">).
- Tableau** : toutes les lignes ont le même nombre de cellules ; la ligne d'en-tête utilise <th> ; la bordure ou le style CSS s'applique.
- Médias** : chaque image s'affiche (sinon, contrôler le chemin relatif et l'extension du fichier) ; le texte alt apparaît si l'image est retirée ; les balises <audio> et <video> affichent leurs contrôles.
- Formulaire** : chaque champ possède un name ; les <label> sont reliés par for/id ; le bouton submit réagit au clic.
- Chemins** : aucun chemin absolu du type C:\Users\... ; tous les chemins sont relatifs

au dossier du site.

6. **CSS** : la feuille est bien liée (modifier une couleur dans `style.css` et recharger la page pour vérifier que l'effet se voit).

ATTENTION Tester dans un seul navigateur ne suffit pas

Ta page peut s'afficher parfaitement dans Chrome et être décalée dans Firefox ou Edge, car chaque navigateur applique des styles par défaut légèrement différents. Prends l'habitude d'ouvrir ta page dans au moins deux navigateurs avant de la déclarer terminée. C'est aussi une remarque appréciée du correcteur en épreuve pratique.

LESAVAIS-TU ? La touche F12 : regarder « sous le capot » de n'importe quel site

Dans tous les navigateurs modernes, la touche **F12** ouvre les outils de développement : tu peux y inspecter le code HTML de la page affichée, voir le CSS appliqué à chaque élément, repérer les images qui ne se chargent pas (onglet « Console » ou « Réseau ») et même modifier le code temporairement pour tester une idée. C'est l'outil préféré des développeurs professionnels — et un excellent moyen d'apprendre en observant comment les sites camerounais (portails universitaires, sites d'actualités) sont construits.

6.4 Erreurs classiques à traquer

L'expérience montre que trois familles d'erreurs reviennent dans presque toutes les copies d'épreuve pratique :

- **Balises non fermées** : un `<td>` oublié décale tout le tableau ; un `</form>` manquant « avale » la fin de la page dans le formulaire. Solution : indenter le code et vérifier que chaque balise ouvrante a sa balise fermante.
- **Chemins incorrects** : écrire `src="remise_prix.jpg"` alors que l'image est dans `images/`, ou confondre `.jpg` et `.jpeg`. Le navigateur n'affiche alors qu'un cadre vide avec le texte `alt`.
- **Attributs name manquants** : un formulaire visuellement parfait mais dont les champs n'ont pas de `name` ne transmet aucune donnée. C'est une erreur invisible à l'écran, d'où l'importance de relire le code et pas seulement le rendu.

6.5 Complément pour le Bac : lire et compléter un code à trous

En épreuve pratique comme en épreuve écrite, on te propose souvent un code HTML à trous : certaines balises ou certains attributs ont été remplacés par des pointillés, et tu dois les reconstituer. La stratégie est toujours la même : repérer le rôle de la ligne (ouverture de tableau ? cellule ? champ de formulaire ?), puis écrire la balise exacte avec sa syntaxe complète.

EXERCICE RÉSOLU 6 Compléter le code de la page « Résultats du lycée »

Énoncé. On a effacé quatre éléments du code ci-dessous, marqués (1), (2), (3) et (4). Reconstitue-les pour que la page affiche un tableau à deux colonnes et un formulaire fonctionnel.

```
<body>
<h1>Resultats du lycee</h1>
(1)
<tr><th>Serie</th><th>Taux</th></tr>
<tr><td>C</td>(2)</tr>
</table>
<form action="traitement.html" method="post">
  <input type="text" (3)="nom">
  <input type="submit" (4)="Envoyer">
</form>
</body>
```

SOLUTION

Solution détaillée.

- (1) : la ligne suivante contient `<tr>`, donc il faut ouvrir le tableau : `<table border="1">`.
- (2) : la ligne d'en-tête annonce deux colonnes (« Série », « Taux »); la cellule `<td>C</td>` donne la série, il manque donc la cellule du taux : `<td>85,4 %</td>`.
- (3) : un champ de saisie doit porter un nom de variable pour transmettre la donnée : c'est l'attribut `name`, d'où `name="nom"`.
- (4) : le texte affiché sur un bouton d'envoi est donné par l'attribut `value`, d'où `value="Envoyer"`.

Le raisonnement est toujours le même : on lit la structure autour du trou (balise parente, colonnes voisines, type de champ) pour déduire l'élément manquant, puis on écrit la syntaxe exacte, guillemets compris.

EXPÉRIENCE TP : construire et valider la page complète

Objectif : réaliser la page « Résultats du lycée » de bout en bout et la valider avec la checklist.

Matériel et logiciels : un ordinateur, un éditeur de texte (Notepad++, Sublime Text ou le Bloc-notes), deux navigateurs (par exemple Chrome et Firefox), une image quelconque renommée `remise_prix.jpg`.

Manipulation :

1. Crée l'arborescence `site_lycee/` avec ses sous-dossiers `css/` et `images/`, et place l'image dans `images/`.
2. Recopie le code de l'exemple fil rouge dans `index.html`; crée un fichier `css/style.css`

contenant au moins une règle, par exemple `h1 { color: blue; }`.

3. Ouvre `index.html` dans le premier navigateur et applique les six points de la checklist de validation.
4. Supprime volontairement le `name` du champ « nom », recharge et observe : le formulaire semble identique, mais la donnée ne serait pas transmise. Rétablis l'attribut.
5. Renomme l'image en `photo.jpg`, recharge : le cadre vide et le texte `alt` apparaissent. Corrige le chemin.
6. Ouvre la page dans le second navigateur et compare l'affichage du tableau et du formulaire.

Observations et interprétation : la page finale affiche le titre dans l'onglet, l'image, le tableau à quatre lignes et le formulaire ; les erreurs provoquées aux étapes 4 et 5 montrent que certaines fautes (chemins, `name`) ne se voient pas au premier regard : seul un test méthodique les révèle. Tu as reproduit exactement la démarche professionnelle : planifier, coder, styliser, tester, corriger.

À RETENIR L'essentiel de la synthèse

- Créer une page web suit une démarche en 5 étapes : planifier l'arborescence, structurer le HTML, insérer tableau/médias/formulaire, lier le CSS, tester.
- Le test est méthodique : structure, tableau, médias, formulaire, chemins, CSS — et dans au moins deux navigateurs.
- Les trois erreurs à traquer : balises non fermées, chemins incorrects, attributs `name` manquants.
- Pour les codes à trous du Bac : repérer le rôle de la ligne, déduire la balise, écrire la syntaxe exacte.

EXERCICE 8 À toi de jouer

Le Lycée de Ngaoundéré veut une page « Vie scolaire » contenant : un tableau des effectifs par classe (Tle C, Tle D, Tle E), une photo de la rentrée placée dans `images/rentree.png`, et un formulaire avec les champs « Nom de l'élève » et « Classe ».

1. Dessine l'arborescence complète du dossier du site.
2. Écris le code HTML complet de la page (structure, tableau à 3 lignes, image, formulaire avec `name` et `label`).
3. Donne la ligne exacte qui lie la feuille `css/vie.css`.

4. Cite les 6 points de la checklist que tu appliqueras avant de rendre ton travail.

CORRIGÉ Exercice

1. Arborescence : site_lycee/ contenant vie_scolaire.html, css/vie.css, images/rentree.png.

2. Code attendu (extrait du corps) :

```
<body>
  <h1>Vie scolaire</h1>
  
  <table border="1">
    <tr><th>Classe</th><th>Effectif</th></tr>
    <tr><td>Tle C</td><td>45</td></tr>
    <tr><td>Tle D</td><td>50</td></tr>
    <tr><td>Tle E</td><td>28</td></tr>
  </table>
  <form action="traitement.html" method="post">
    <label for="eleve">Nom de l'eleve :</label>
    <input type="text" id="eleve" name="eleve">
    <label for="classe">Classe :</label>
    <input type="text" id="classe" name="classe">
    <input type="submit" value="Envoyer">
  </form>
</body>
```

3. Dans le <head> : <link rel="stylesheet" href="css/vie.css">.
4. Structure et titre de l'onglet ; tableau régulier avec <th> ; image affichée et alt présent ; name sur chaque champ ; chemins tous relatifs ; CSS bien lié (test en modifiant une couleur) ; affichage contrôlé dans deux navigateurs.

7 ACTIVITÉ D'INTÉGRATION

8 SYNTHÈSE : CONSTRUIRE UNE PAGE WEB COMPLÈTE

8.1 Objectifs de l'activité

Cette activité d'intégration vise à mobiliser l'ensemble des savoir-faire acquis au cours de la leçon « Programmation en HTML » pour produire une page web complète, structurée, accessible et esthétiquement soignée. À l'issue de ce travail, vous devez être capable de :

- Concevoir une arborescence de site web logique et normalisée (dossiers images, css, pages).
- Construire un tableau de données complexe (<table>, <thead>, <tbody>, <tfoot>, <caption>, fusion de cellules avec `colspan`).
- Intégrer des contenus multimédias (image avec attribut `alt` obligatoire, vidéo ou audio avec contrôles natifs).
- Créer un formulaire fonctionnel côté client (champs texte, téléphone, boutons radio, bouton de soumission) avec une accessibilité basique (<label> lié aux <input>).
- Lier une feuille de style externe (CSS) pour séparer la structure de la présentation (coloration de l'en-tête de tableau, espacements).
- Valider le rendu dans un navigateur et présenter le travail collectivement.

8.2 Situation

Le Lycée Bilingue de Nkolbisson organise le conseil de classe du premier trimestre pour les classes de Première C, D et E. L'administration souhaite publier les résultats synthétiques sur l'intranet de l'établissement afin que les délégués de parents d'élèves puissent les consulter avant la réunion de restitution prévue le samedi prochain. En tant que membres du club informatique, votre groupe (3 à 4 élèves) est chargé de réaliser la page web officielle de ce conseil de classe.

Voici les données officielles fournies par le secrétariat pour la série **Première D** (effectif : 42 élèves) :

Matière	Coefficient	Moyenne de la classe	Meilleure note
Mathématiques	6	12,4 / 20	19 / 20
Physique-Chimie	5	11,8 / 20	18,5 / 20
Sciences de la Vie et de la Terre	4	13,2 / 20	19,5 / 20
Français	4	10,5 / 20	17 / 20
Anglais	3	11,0 / 20	18 / 20
Philosophie	2	12,0 / 20	16 / 20

La moyenne générale pondérée de la classe s'établit à **11,86 / 20**. Le taux de réussite (moyenne ≥ 10) est de **68 %**.

La page doit également permettre aux parents de s'inscrire à la réunion de restitution via un formulaire. Le lycée dispose d'une photo officielle du bâtiment principal (`lycee_nkolbisson.jpg`) et d'une courte vidéo de présentation de l'établissement (`presentation_lycee.mp4`, durée 45 secondes) stockées sur le serveur de l'établissement. Une feuille de style minimaliste (`style.css`) doit donner une identité visuelle sobre : fond clair, police lisible, et surtout un en-tête de tableau coloré aux couleurs de l'établissement (bleu foncé #1B3A5C et texte blanc).

8.3 Consignes

Travaillez en groupe de 3 ou 4. Répartissez les rôles (intégrateur HTML, styliste CSS, gestionnaire d'arborescence/multimédia, testeur/présentateur). Suivez les étapes ci-dessous dans l'ordre.

1. **Arborescence** : Créez un dossier racine nommé `conseil_classe_1D`. À l'intérieur, créez trois sous-dossiers : `images`, `css`, `pages`. Placez les fichiers multimédias fournis (`lycee_nkolbisson.jpg`, `presentation_lycee.mp4`) dans `images`. Créez le fichier `style.css` dans `css`. Créez le fichier principal `index.html` dans `pages`.
2. **Structure HTML de base** : Dans `index.html`, écrivez la structure complète (`<!DOCTYPE html>`, `<html lang="fr">`, `<head>` avec métadonnées `charset`, `viewport`, `title`, lien vers `../css/style.css`, `<body>`).
3. **En-tête de page** : Insérez un `<header>` contenant le logo (image du lycée, largeur 150 px, alt pertinent) et le titre principal `<h1>` « Conseil de Classe — 1^{er} Trimestre — Première D ».
4. **Tableau des résultats** : Construisez un `<table>` sémantique :
 - Légende (`<caption>`) : « Résultats synthétiques du 1^{er} trimestre — Série D (42 élèves) ».
 - En-tête (`<thead>`) : une ligne avec les quatre colonnes « Matière », « Coefficient », « Moyenne de la classe », « Meilleure note ».
 - Corps (`<tbody>`) : les six lignes de données du tableau ci-dessus.
 - Pied (`<tfoot>`) : une unique ligne fusionnée sur 4 colonnes (`colspan="4"`) affichant : « Moyenne générale pondérée : 11,86 / 20 — Taux de réussite : 68 % ».
5. **Multimédia** : Après le tableau, insérez la vidéo `presentation_lycee.mp4` avec les attributs `controls`, `width="480"` et une source de secours (texte « Votre navigateur ne supporte pas la vidéo HTML5. »).
6. **Formulaire d'inscription** : Créez un `<form>` (méthode GET, action # pour simulation) contenant :
 - Un champ « Nom complet du parent » (`<input type="text">`, obligatoire, `id="nom"`).
 - Un champ « Numéro de téléphone (WhatsApp) » (`<input type="tel">`, motif `[0-9]{9}`, placeholder « 6XX XX XX XX », `id="tel"`).
 - Un groupe « Série de l'enfant » avec trois boutons radio (`<input type="radio" name="serie" value="C">`, D, E), « D » précoché.
 - Un bouton de soumission « Valider mon inscription ».

- Chaque champ doit être associé à un `<label>` via `for/id`.

7. Feuille de style (CSS) : Dans `style.css`, définissez au minimum :

- `body { font-family: Arial, sans-serif; line-height: 1.6; margin: 20px; }`
- `table { border-collapse: collapse; width: 100%; margin-bottom: 20px; }`
- `th, td { border: 1px solid #ddd; padding: 10px; text-align: center; }`
- `thead { background-color: #1B3A5C; color: white; }`
- `caption { caption-side: top; font-weight: bold; margin-bottom: 10px; font-size: 1.1em; }`
- `tfoot td { background-color: #f2f2f2; font-weight: bold; }`
- `form { border: 1px solid #ccc; padding: 20px; max-width: 400px; border-radius: 5px; }`
- `label { display: block; margin-top: 10px; font-weight: bold; }`
- `input[type="text"], input[type="tel"] { width: 100%; padding: 8px; margin-top: 4px; box-sizing: border-box; }`
- `input[type="submit"] { margin-top: 15px; padding: 10px 20px; background-color: #1B3A5C; color: white; border: none; border-radius: 3px; cursor: pointer; }`

8. Test et validation : Ouvrez `pages/index.html` dans un navigateur (Chrome, Firefox, Edge). Vérifiez la checklist :

- L'image s'affiche et le texte `alt` apparaît si on survole ou si l'image est cassée.
- La vidéo se lit et les contrôles fonctionnent.
- Le tableau est lisible, l'en-tête est bleu blanc, la ligne « Moyenne générale » fusionne bien les 4 colonnes.
- Le formulaire est utilisable : les labels cliquent sur les champs, les radios sont exclusifs, le bouton réagit.
- Aucun lien brisé (chemins relatifs corrects : `../images/...`, `../css/...`).

9. Présentation : Désignez un porte-parole. Présentez la page en 3 minutes : arborescence, choix sémantiques (pourquoi `<thead>/<tfoot>` ?), accessibilité (labels, alt), rôle du CSS.

8.4 Ressources à mobiliser

- **Syntaxe du tableau HTML5** : `<table>`, `<caption>`, `<thead>`, `<tbody>`, `<tfoot>`, `<tr>`, `<th>` (`scope="col"`), `<td>`, attributs `colspan`, `rowspan`.
- **Multimédia** : ``, `<video controls width="..." src="..." type="video/mp4">Texte de secours</video>`, `<audio controls><source src="..." type="audio/mpeg"></audio>`.
- **Formulaire** : `<form action="..." method="GET|POST">`, `<label for="id">`, `<input type="text|tel|radio" id="..." name="..." value="..." required placeholder="...">`, attribut `checked` pour radio par défaut, `pattern` pour validation téléphone.

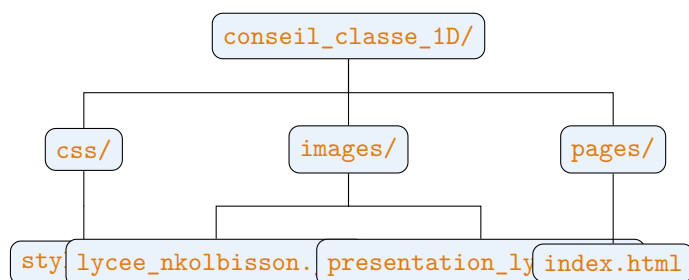
- **Arborescence & Chemins relatifs** : `../` (dossier courant), `../../` (remonter d'un niveau), `dossier/fichier` (descendre).
- **Liaison CSS** : `<link rel="stylesheet" href="../../css/style.css">` dans le `<head>`.
- **Sélecteurs CSS de base** : sélecteur de type (`table`), de classe (`.classe`), d'ID (`#id`), descendant (`thead th`), pseudo-classes (`:hover` optionnel). Propriétés : `color`, `background-color`, `border`, `padding`, `margin`, `width`, `text-align`, `font-weight`, `border-collapse`.

8.5 Démarche et corrigé commenté

EXERCICE RÉSOLU 7

Réalisation complète de la page « Conseil de Classe 1D » Étape 1 — Arborescence du projet On crée la structure de dossiers suivante sur le bureau ou dans le répertoire de travail de l'élève :

```
conseil_classe_1D/
├── css/
│   └── style.css
├── images/
│   ├── lycee_nkolbisson.jpg
│   └── presentation_lycee.mp4
└── pages/
    └── index.html
```



Arborescence du projet conseil_classe_1D (chemins relatifs depuis pages/)

Justification : Cette organisation sépare les responsabilités (contenu, style, médias) et facilite la maintenance. Les chemins relatifs depuis `pages/index.html` vers les ressources seront `../images/...` et `../css/...`.

Étape 2 — Fichier HTML principal (`pages/index.html`) Voici le code complet, commenté et validé W3C (HTML5 sémantique).

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```

<title>Conseil de Classe 1D – Lycée Bilingue de Nkolbisson</title>
<!-- Lien vers la feuille de style externe (chemin relatif depuis pages/) -->
<link rel="stylesheet" href="../css/style.css">
</head>
<body>
  <header>
    
    <h1>Conseil de Classe – 1<sup>er</sup> Trimestre – Première D</h1>
    <p><em>Lycée Bilingue de Nkolbisson – Année scolaire 2024–2025</em></p>
  </header>

  <main>
    <!-- SECTION 1 : TABLEAU DES RÉSULTATS -->
    <section aria-labelledby="titre-resultats">
      <h2 id="titre-resultats">Résultats Synthétiques</h2>

      <table>
        <caption>Résultats synthétiques du 1<sup>er</sup> trimestre – Série D (42 élèves)</caption>

        <thead>
          <tr>
            <th scope="col">Matière</th>
            <th scope="col">Coefficient</th>
            <th scope="col">Moyenne de la classe</th>
            <th scope="col">Meilleure note</th>
          </tr>
        </thead>

        <tbody>
          <tr>
            <td>Mathématiques</td>
            <td>6</td>
            <td>12,4 / 20</td>
            <td>19 / 20</td>
          </tr>
          <tr>
            <td>Physique–Chimie</td>
            <td>5</td>
            <td>11,8 / 20</td>
            <td>18,5 / 20</td>
          </tr>
          <tr>
            <td>Sciences de la Vie et de la Terre</td>
            <td>4</td>
            <td>13,2 / 20</td>
            <td>19,5 / 20</td>
          </tr>
          <tr>
            <td>Français</td>
            <td>4</td>
            <td>10,5 / 20</td>
            <td>17 / 20</td>
          </tr>
          <tr>

```

```

        <td>Anglais</td>
        <td>3</td>
        <td>11,0 / 20</td>
        <td>18 / 20</td>
    </tr>
    <tr>
        <td>Philosophie</td>
        <td>2</td>
        <td>12,0 / 20</td>
        <td>16 / 20</td>
    </tr>
</tbody>

<tfoot>
    <tr>
        <!-- Fusion des 4 colonnes pour la ligne de synthèse -->
        <td colspan="4">
            <strong>Moyenne générale pondérée : 11,86 / 20</strong> —
            <strong>Taux de réussite : 68 \%</strong>
        </td>
    </tr>
</tfoot>
</table>
</section>

<!-- SECTION 2 : MULTIMÉDIA -->
<section aria-labelledby="titre-media" style="margin-top : 40px; text-align : center;">
    <h2 id="titre-media">Présentation de l'établissement</h2>
    <video controls width="480" poster=" ../images/lycee_nkolbisson.jpg">
        <source src=" ../images/presentation_lycee.mp4" type="video/mp4">
        <p>Votre navigateur ne supporte pas la vidéo HTML5.
            <a href=" ../images/presentation_lycee.mp4">Télécharger la vidéo</a>.</p>
    </video>
    <p><small>Vidéo institutionnelle du Lycée Bilingue de Nkolbisson (45 s)</small></p>
</section>

<!-- SECTION 3 : FORMULAIRE D'INSCRIPTION -->
<section aria-labelledby="titre-form" style="margin-top : 40px; max-width : 500px; margin-left : auto; margin-right : auto;">
    <h2 id="titre-form">Inscription à la réunion de restitution</h2>
    <p>La réunion se tiendra le samedi 18 janvier 2025 à 09h00 dans la salle des actes. Merci de confirmer votre présence.</p>

    <form action="#" method="GET" novalidate>
        <!-- Champ Nom complet -->
        <div>
            <label for="nom">Nom complet du parent / tuteur *</label>
            <input type="text"
                id="nom"
                name="nom_parent"
                required
                placeholder="Ex : M. MBALLA Jean-Pierre"
                autocomplete="name">
        </div>

        <!-- Champ Téléphone -->
        <div>

```

```

        <label for="tel">Numéro de téléphone (WhatsApp) *</label>
        <input type="tel"
            id="tel"
            name="telephone"
            required
            pattern="[0-9]{9}"
            placeholder="6XX XX XX XX (9 chiffres)"
            title="9 chiffres sans espace ni indicatif pays, ex : 677123456"
            autocomplete="tel">
        <small style="color : #666;">Format camerounais : 9 chiffres commençant par 6.</
small>
    </div>

    <!-- Groupe Radio : Série -->
    <fieldset>
        <legend>Série de l'enfant *</legend>
        <div style="display : flex; gap : 20px; justify-content : center; margin-top : 10px;">
            <label style="font-weight : normal; display : flex; align-items : center; gap : 5px;
cursor : pointer;">
                <input type="radio" name="serie" value="C" id="serieC"> Première C
            </label>
            <label style="font-weight : normal; display : flex; align-items : center; gap : 5px;
cursor : pointer;">
                <input type="radio" name="serie" value="D" id="serieD" checked> Première D
            </label>
            <label style="font-weight : normal; display : flex; align-items : center; gap : 5px;
cursor : pointer;">
                <input type="radio" name="serie" value="E" id="serieE"> Première E
            </label>
        </div>
    </fieldset>

    <!-- Bouton de soumission -->
    <div style="text-align : center; margin-top : 20px;">
        <button type="submit">Valider mon inscription</button>
    </div>
</form>
</section>
</main>

<!-- Pied de page -->
<footer style="text-align : center; margin-top : 50px; padding-top : 20px; border-top : 1px solid #
eee; color : #666; font-size : 0.9em;">
    <p>&copy; 2025 Lycée Bilingue de Nkolbisson — Club Informatique — Page réalisée par les élèves
de 1ère D</p>
</footer>
</body>
</html>

```

Points techniques commentés (HTML) :

1. `lang="fr"` : Indique la langue principale pour les lecteurs d'écran et le référencement.
2. `scope="col"` sur les `<th>` : Améliore l'accessibilité en liant explicitement l'en-tête à sa colonne.
3. `<caption>` placé juste après `<table>` : Seule position valide ; donne un titre au tableau

pour tous les utilisateurs.

4. `<thead>`, `<tbody>`, `<tfoot>` : Permettent au navigateur de répéter l'en-tête si le tableau est imprimé sur plusieurs pages, et aux CSS de cibler spécifiquement l'en-tête (`thead` `th`) sans affecter le corps.
5. `colspan="4"` sur le `<td>` du `<tfoot>` : Fusionne les 4 cellules pour centrer la synthèse sur toute la largeur.
6. `<video controls>` : L'attribut `controls` impose l'affichage des boutons natifs (play, pause, volume, plein écran). L'attribut `poster` affiche l'image du lycée avant le chargement de la vidéo.
7. `<label for="id">` + `<input id="id">` : Association explicite. Cliquer sur le label focus le champ (accessibilité cruciale).
8. `type="tel" + pattern="[0-9]{9}"` : Ouvre le clavier numérique sur mobile et valide le format 9 chiffres (standard camerounais hors indicatif +237).
9. `name="serie"` commun aux 3 radios : Garantit l'exclusivité (un seul choix possible).
10. `novalidate` sur `<form>` : Désactive la validation HTML5 native pour cet exercice (on simule un envoi GET vers #), mais les attributs `required` et `pattern` restent utiles pour l'UX (bulles d'aide navigateur).

Étape 3 — Feuille de style CSS (`css/style.css`)

```
/* =====
FEUILLE DE STYLE : Conseil de Classe 1D
Lycée Bilingue de Nkolbisson
===== */

/* --- Réinitialisation de base & Typographie --- */
* {
  box-sizing: border-box; /* Inclut padding/border dans width/height */
  margin: 0;
  padding: 0;
}

body {
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  line-height: 1.6;
  color: #333;
  background-color: #fafafa;
  margin: 0;
  padding: 20px;
}

/* --- Mise en page principale --- */
main {
  max-width: 900px;
  margin: 0 auto;
  background: white;
  padding: 30px;
  border-radius: 8px;
```

```
        box-shadow : 0 2px 10px rgba(0,0,0,0.05);
    }

    header {
        text-align : center;
        margin-bottom : 30px;
        padding-bottom : 20px;
        border-bottom : 2px solid #1B3A5C;
    }

    header img {
        border-radius : 50%; /* Logo arrondi */
        border : 3px solid #1B3A5C;
        margin-bottom : 15px;
        box-shadow : 0 4px 8px rgba(0,0,0,0.1);
    }

    header h1 {
        color : #1B3A5C;
        font-size : 1.8rem;
        margin-bottom : 5px;
    }

    header p {
        color : #555;
        font-style : italic;
    }

    /* --- Titres de sections --- */
    section h2 {
        color : #1B3A5C;
        border-bottom : 2px solid #e0e0e0;
        padding-bottom : 10px;
        margin-bottom : 20px;
        font-size : 1.4rem;
    }

    /* --- TABLEAU : Styles principaux --- */
    table {
        border-collapse : collapse; /* Fusionne les bordures adjacentes */
        width : 100%;
        margin-bottom : 30px;
        font-size : 0.95rem;
    }

    caption {
        caption-side : top;
        font-weight : 700;
        font-size : 1.1rem;
        color : #1B3A5C;
        margin-bottom : 12px;
        text-align : left;
    }

    th, td {
        border : 1px solid #dee2e6;
        padding : 12px 10px;
```

```
text-align : center;
vertical-align : middle;
}

/* En-tête du tableau : Couleurs du lycée */
thead {
    background-color : #1B3A5C; /* Bleu foncé officiel */
    color : white;
}

thead th {
    border-color : #2c5a8a; /* Bordure légèrement plus claire pour l'intérieur */
    font-weight : 600;
    text-transform : uppercase;
    font-size : 0.85rem;
    letter-spacing : 0.5px;
}

/* Alternance de lignes (zébrage) pour lisibilité */
tbody tr:nth-child(even) {
    background-color : #f8f9fa;
}

tbody tr:hover {
    background-color : #eef5fb; /* Survol léger bleu */
    transition : background-color 0.2s;
}

/* Pied de tableau : Ligne de synthèse */
tfoot td {
    background-color : #e9ecef;
    font-weight : 700;
    color : #1B3A5C;
    font-size : 1rem;
    border-top : 2px solid #1B3A5C; /* Séparation forte */
}

/* Alignement spécifique : Matière à gauche */
tbody td :first-child {
    text-align : left;
    font-weight : 500;
}

/* --- MULTIMÉDIA --- */
video {
    border-radius : 6px;
    box-shadow : 0 4px 12px rgba(0,0,0,0.1);
    background-color : #000; /* Fond noir pendant chargement */
    max-width : 100%; /* Responsive */
    height : auto;
}

/* --- FORMULAIRE --- */
form {
    border : 1px solid #dee2e6;
    padding : 25px;
    border-radius : 8px;
}
```

```
background-color : #fdfdfd;
box-shadow : 0 1px 3px rgba(0,0,0,0.03);
}

fieldset {
border : 1px solid #dee2e6;
border-radius : 6px;
padding : 15px;
margin : 20px 0;
}

legend {
font-weight : 600;
color : #1B3A5C;
padding : 0 10px;
font-size : 0.95rem;
}

label {
display : block;
margin-bottom : 6px;
font-weight : 600;
font-size : 0.9rem;
color : #444;
}

input[type="text"],
input[type="tel"],
input[type="email"] {
width : 100%;
padding : 10px 12px;
margin-bottom : 18px;
border : 1px solid #ced4da;
border-radius : 4px;
font-size : 1rem;
transition : border-color 0.2s, box-shadow 0.2s;
}

input[type="text"]:focus,
input[type="tel"]:focus {
outline : none;
border-color : #1B3A5C;
box-shadow : 0 0 3px rgba(27, 58, 92, 0.15);
}

small {
display : block;
margin-top : -14px;
margin-bottom : 18px;
font-size : 0.8rem;
color : #6c757d;
}

/* Style des radios (amélioration visuelle) */
input[type="radio"] {
accent-color : #1B3A5C; /* Couleur du bouton radio coché (CSS moderne) */
margin-right : 8px;
```

```
        transform : scale(1.1);
        vertical-align : middle;
    }

    fieldset > div label {
        font-weight : normal;
        margin-bottom : 0;
        cursor : pointer;
    }

    /* Bouton submit */
    button[type="submit"]{
        width : 100%;
        padding : 12px;
        background-color : #1B3A5C;
        color : white;
        border : none;
        border-radius : 4px;
        font-size : 1rem;
        font-weight : 600;
        cursor : pointer;
        transition : background-color 0.2s, transform 0.1s;
    }

    button[type="submit"]:hover{
        background-color : #152d47;
    }

    button[type="submit"]:active{
        transform : scale(0.98);
    }

    /* --- FOOTER --- */
    footer{
        max-width : 900px;
        margin : 30px auto 0;
        padding-top : 20px;
        border-top : 1px solid #eee;
        color : #6c757d;
        font-size : 0.85rem;
        text-align : center;
    }
```

Points techniques commentés (CSS) :

1. `box-sizing: border-box (sur *)` : Évite les surprises de largeur quand on ajoute du padding ou des border sur les input (largeur 100% reste 100%).
2. `border-collapse: collapse` sur table : Indispensable pour éviter les doubles bordures entre cellules.
3. `thead { background-color: #1B3A5C; color: white; }` : Répond exactement à la consigne « colore l'en-tête du tableau aux couleurs de l'établissement ».
4. `tbody tr:nth-child(even)` : Zébrage automatique (lignes paires grisées) pour la lisibilité de grandes tables.

5. `tfoot td { border-top: 2px solid #1B3A5C; }` : Sépare visuellement la synthèse du corps du tableau.
6. `accent-color: #1B3A5C` sur `input[type="radio"]` : Propriété CSS moderne (supportée partout) pour colorer les contrôles natifs sans hacks complexes.
7. `max-width: 100%; height: auto` sur `video` : Rend la vidéo responsive (ne déborde pas sur mobile).
8. `:focus` sur les champs : Indication visuelle forte (ombre bleue) pour l'accessibilité clavier (navigation Tab).

Étape 4 — Test et vérification (Checklist)

Point de contrôle	Résultat attendu / Action corrective
Arborescence respectée	Dossiers <code>css</code> , <code>images</code> , <code>pages</code> présents à la racine.
Chemins relatifs (HTML)	<code>../images/lycee_nkolbisson.jpg</code> (image), <code>../css/style.css</code> (CSS). Pas de <code>C:\Users....</code>
Image (logo)	Affichée 150px, arrondie, <code>alt</code> visible au survol / si erreur.
Tableau : Structure	<code>caption</code> , <code>thead</code> , <code>tbody</code> , <code>tfoot</code> présents dans l'inspecteur.
Tableau : Fusion	Ligne pied de tableau : une seule cellule large (<code>colspan="4"</code>).
Tableau : Style CSS	En-tête bleu #1B3A5C, texte blanc. Zébrage actif.
Vidéo	Lecteur natif affiché, boutons fonctionnels, <code>poster</code> visible au chargement.
Formulaire : Labels	Clic sur « Nom complet » → focus sur champ <code>id="nom"</code> . Idem téléphone.
Formulaire : Radios	Un seul choix possible (C, D ou E). « D » coché au chargement.
Formulaire : Validation	Champ téléphone refuse les lettres / impose 9 chiffres (bulle navigateur).
Responsive (mobile)	Tableau scrollable horizontalement si nécessaire (ajout <code>overflow-x:auto</code> sur conteneur si absent), formulaire largeur 100%.

Étape 5 — Présentation orale (Modèle de discours)

« Bonjour. Notre groupe a réalisé la page du conseil de classe de Première D. L'arborescence sépare les ressources : `css` pour la présentation, `images` pour les médias, `pages` pour le HTML. Nous avons utilisé un tableau sémantique avec `thead`, `tbody`, `tfoot` pour permettre aux lecteurs d'écran de naviguer dans les données et au CSS de cibler l'en-tête spécifiquement — d'où le bleu officiel du lycée sur les titres de colonnes. La ligne de synthèse utilise `colspan="4"` pour fusionner les colonnes. La vidéo utilise l'élément `<video>`

natif avec `controls` pour ne pas dépendre de lecteurs externes. Le formulaire associe chaque `<label>` à son `<input>` via `for/id`; le téléphone utilise `type="tel"` et un `pattern` pour le format camerounais à 9 chiffres. Le CSS utilise `border-collapse`, des variables de couleur implicites (`#1B3A5C`) et `accent-color` pour les radios. Merci. »

SOLUTION

Erreurs fréquentes observées et corrections apportées :

- Chemins absolus (`C : \...`) : Ne fonctionnent que sur la machine de l'élève. Correction : Chemins relatifs `../images/...`
- Oubli de `alt` sur l'image : Non-conforme accessibilité (RGAA/WCAG). Correction : `alt="Façade principale du Lycée..."`.
- `<caption>` hors du `<table>` ou après `<thead>` : Invalide HTML5. Correction : `<caption>` premier enfant de `<table>`.
- Fusion `colspan` mal placée : Mise sur un `<th>` du `<thead>` au lieu du `<td>` du `<tfoot>`. Correction : Un seul `<td colspan="4">` dans `<tfoot>`.
- Radios sans `name` commun : Tous cochables en même temps. Correction : `name="serie"` identique pour les trois.
- CSS non chargé : `href="css/style.css"` au lieu de `href="../css/style.css"` depuis `pages/`. Correction : Remonter d'un niveau avec `../`.
- Tableau illisible sur mobile : Pas de `overflow-x:auto` sur un conteneur `<div>` englobant le `<table>`. Astuce : Ajouter `<div style="overflow-x:auto">...</div>` autour du tableau si le contenu est large.

8.6 Bilan de l'activité

Cette activité d'intégration a permis de valider concrètement l'ensemble des compétences du module « Création des sites Web » pour le langage HTML/CSS côté client.

- **Structuration de l'information** : Le passage d'un tableau « brut » (données du secrétariat) à un tableau HTML sémantique (`thead`, `tbody`, `tfoot`, `caption`, `scope`, `colspan`) a montré que le balisage n'est pas seulement visuel : il porte du sens pour les machines (moteurs de recherche, technologies d'assistance) et facilite le stylage CSS (sélecteur `thead th` sans classe ajoutée).
- **Gestion des ressources et arborescence** : La contrainte des chemins relatifs (`../`) depuis le sous-dossier `pages/` est une source d'erreurs classique. L'activité ancre la rigueur nécessaire : un site web est une arborescence de fichiers, pas un document unique.
- **Multimédia natif** : L'utilisation de `<video controls>` sans JavaScript ni plugin Flash (obsolète) illustre la puissance du HTML5. L'attribut `poster` améliore l'expérience utilisateur (UX) pendant le chargement.

- **Formulaire et accessibilité** : L'association `label/input` (via `for/id`), l'usage de `type="tel"` + `pattern` pour le contexte camerounais, et le regroupement radios par `name` sont des fondamentaux de l'ergonomie et de l'accessibilité (RGAA critère 11.1, 11.9, 12.1).
- **Séparation Structure / Présentation** : Le fichier `style.css` externe, lié par `<link>`, démontre le principe fondateur du Web moderne : le HTML décrit quoi (contenu, sémantique), le CSS décrit comment (couleurs, espacements, responsive). Le changement de couleur de l'en-tête de tableau ne nécessite aucune modification du HTML.
- **Travail collaboratif et communication** : La répartition des rôles et la présentation orale finale développent les compétences transverses (soft skills) essentielles : explication technique claire, justification des choix (pourquoi `tfoot`? pourquoi `scope="col"`?), écoute des retours de la classe via la checklist.

À RETENIR**Compétences validées par cette activité**

- Créer une arborescence de site web normée (css, images, pages).
- Écrire un document HTML5 valide, sémantique et accessible (tableau complexe, multimédia, formulaire).
- Maîtriser les chemins relatifs pour lier ressources (CSS, images, vidéos) depuis une page dans un sous-dossier.
- Appliquer une feuille de style CSS externe pour styliser un tableau (en-tête coloré, zébrage, fusion), une vidéo et un formulaire.
- Tester systématiquement dans un navigateur et corriger les erreurs de chemin, de syntaxe ou d'accessibilité.
- Présenter et justifier ses choix techniques devant un public.

ATTENTION**Pièges à éviter pour le Probatoire**

- Confondre `colspan` (fusion horizontale) et `rowspan` (fusion verticale).
- Placer `<caption>` après `<thead>` ou en dehors de `<table>`.
- Oublier `scope="col"` ou `scope="row"` sur les `<th>` (accessibilité).
- Utiliser des chemins absolus locaux (C:\Users...) au lieu de chemins relatifs (../images/...).
- Oublier l'attribut `alt` sur `<img>` (zéro point accessibilité).
- Ne pas lier `<label>` et `<input>` (attributs `for` et `id` identiques).
- Mettre le CSS dans le HTML (`<style>` ou `style=""`) au lieu d'un fichier `.css` externe lié par `<link>`.

9 MÉTHODES ET EXERCICES RÉSOLUS

10 PROGRAMMATION EN HTML

10.1 Les tableaux en HTML

Méthode 1 : Insérer un tableau dans une page web

MÉTHODE Construire un tableau HTML pas à pas

Pour créer un tableau correct en HTML, suis toujours la même démarche :

1. Analyser les données : compte le nombre de lignes et de colonnes nécessaires, et repère la ligne d'en-tête (titres des colonnes).
2. Ouvrir le tableau avec la balise `<table>` et lui donner une bordure visible avec l'attribut `border="1"` (à des fins pédagogiques ; la mise en forme finale se fera en CSS).
3. Ajouter un titre au tableau avec `<caption>` (placé juste après `<table>`).
4. Créer chaque ligne avec `<tr>` (table row).
5. Remplir chaque ligne : cellules d'en-tête avec `<th>` (table header, texte gras et centré par défaut), cellules de données avec `<td>` (table data).
6. Fusionner si besoin : `colspan="n"` pour fusionner n colonnes horizontalement, `rowspan="n"` pour fusionner n lignes verticalement. Attention : une cellule fusionnée remplace les cellules absorbées, il ne faut donc pas les écrire.
7. Fermer toutes les balises dans l'ordre inverse de leur ouverture : `</td>`, `</tr>`, `</table>`.

Réflexe de vérification : chaque ligne `<tr>` doit contenir le même nombre total de colonnes, en comptant les `colspan`.

EXERCICE RÉSOLU 8 Tableau des moyennes d'une classe de Première D

Le proviseur du lycée de Bafoussam veut publier sur le site de l'établissement le tableau suivant :

Élève	Moyenne /20	Observation
Ngo Bell	15,5	Admis
Fotso Ulrich	9,75	Ajourné

Écrire le code HTML complet (structure de la page incluse) qui produit ce tableau avec bordures et titre « Résultats du 1er trimestre ».

SOLUTION

Étape 1 : analyse. Le tableau comporte 3 colonnes (Élève, Moyenne, Observation) et 3 lignes : 1 ligne d'en-tête + 2 lignes de données.

Étape 2 : structure. La ligne d'en-tête utilisera `<th>`, les deux autres `<td>`. Le titre du tableau est placé dans `<caption>`.

Étape 3 : rédaction du code.

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Resultats du lycee</title>
</head>
<body>
  <table border="1">
    <caption>Resultats du 1er trimestre</caption>
    <tr>
      <th>Eleve</th>
      <th>Moyenne /20</th>
      <th>Observation</th>
    </tr>
    <tr>
      <td>Ngo Bell</td>
      <td>15,5</td>
      <td>Admis</td>
    </tr>
    <tr>
      <td>Fotso Ulrich</td>
      <td>9,75</td>
      <td>Ajourne</td>
    </tr>
  </table>
</body>
</html>
```

Vérification. Chaque `<tr>` contient exactement 3 cellules : le tableau est régulier. L'attribut `border="1"` rend les bordures visibles, et `<caption>` affiche le titre au-dessus du tableau.

Conclusion. Le code ci-dessus, enregistré dans un fichier `resultats.html` puis ouvert dans un navigateur, affiche exactement le tableau demandé avec bordures, en-têtes en gras et titre.

EXERCICE RÉSOLU 9 Tableau avec cellules fusionnées

On veut afficher l'emploi du temps simplifié d'une classe : une première ligne d'en-tête « Jour | Horaire | Matière », puis une ligne où la cellule « Lundi » s'étend sur deux lignes

(`rowspan="2"`), avec les cours « 7h–10h : Mathématiques » puis « 10h–12h : Informatique ». Écrire le code HTML du tableau.

SOLUTION

Raisonnement. Le tableau a 3 colonnes. La cellule « Lundi » occupe la première colonne sur deux lignes : on l'écrit une seule fois avec `rowspan="2"` dans la première ligne de données, et on ne la répète pas dans la ligne suivante. La deuxième ligne de données ne contient donc que 2 cellules `<td>`, car la première colonne est déjà occupée par la cellule fusionnée.

```
<table border="1">
  <caption>Emploi du temps du lundi</caption>
  <tr>
    <th>Jour</th>
    <th>Horaire</th>
    <th>Matiere</th>
  </tr>
  <tr>
    <td rowspan="2">Lundi</td>
    <td>7h – 10h</td>
    <td>Mathematiques</td>
  </tr>
  <tr>
    <td>10h – 12h</td>
    <td>Informatique</td>
  </tr>
</table>
```

Vérification. Ligne 1 : 3 cellules `<th>`. Ligne 2 : 1 cellule `<td rowspan="2">` + 2 `<td>`, soit $1+2 = 3$ colonnes. Ligne 3 : 2 `<td>` + la colonne déjà occupée par le `rowspan`, soit 3 colonnes. Le tableau est régulier.

Conclusion. La cellule « Lundi » s'affiche sur deux lignes et le tableau reste correctement aligné : la règle d'or est de ne jamais réécrire une cellule absorbée par une fusion.

10.2 Insertion des fichiers multimédias

Méthode 2 : Insérer une image, un son ou une vidéo

MÉTHODE Insérer des fichiers multimédias

1. Placer le fichier multimédia dans le dossier du site (idéalement dans un sous-dossier `images/`, `sons/` ou `videos/`).
2. Choisir la bonne balise selon la nature du fichier :
 - image : `` (balise orpheline, sans fermeture);

- son : `<audio controls>` avec une ou plusieurs sources `<source src="..." type="audio/mpeg">` ;
 - vidéo : `<video controls width="...">` avec `<source src="..." type="video/mp4">`.
3. Renseigner le chemin correctement : chemin relatif par rapport à la page HTML (ex. `images/logo.png`), jamais un chemin absolu de type `C:\Mes documents\...`
 4. Toujours écrire l'attribut `alt` pour une image : il décrit l'image si elle ne s'affiche pas et sert à l'accessibilité.
 5. Ajouter `controls` pour l'audio et la vidéo, sinon l'utilisateur ne peut pas lancer la lecture.
 6. Tester la page dans le navigateur et vérifier que le fichier se charge (respecter la casse du nom de fichier : `Logo.png` ≠ `logo.png`).

EXERCICE RÉSOLU 10 Page multimédia d'une école

Le site du collège bilingue de Ngaoundéré doit présenter une page contenant : le logo de l'école (fichier `logo.png` dans le sous-dossier `images`), l'hymne de l'école (fichier `hymne.mp3` dans `sons`) et une vidéo de présentation (`presentation.mp4` dans `videos`, largeur 480 pixels). Écrire le code HTML du corps de la page.

SOLUTION

Raisonnement. On applique la méthode : chaque fichier est référencé par un chemin relatif depuis la page HTML, l'image reçoit un `alt` descriptif, et les balises `<audio>` et `<video>` reçoivent l'attribut `controls` pour permettre la lecture.

```
<body>
  <h1>Bienvenue au College Bilingue de Ngaoundere</h1>

  <h2>Notre logo</h2>
  

  <h2>Notre hymne</h2>
  <audio controls>
    <source src="sons/hymne.mp3" type="audio/mpeg">
    Votre navigateur ne supporte pas la balise audio.
  </audio>

  <h2>Video de presentation</h2>
  <video controls width="480">
    <source src="videos/presentation.mp4" type="video/mp4">
    Votre navigateur ne supporte pas la balise video.
  </video>
</body>
```

Justifications. (1) `<img>` est une balise orpheline : elle n'a pas de balise fermante. (2) Les textes placés entre `<audio>` et `</audio>` (resp. `<video>`) ne s'affichent que si le navigateur est trop ancien : c'est un message de repli. (3) Les chemins `images/logo.png`, `sons/hymne.mp3` et `videos/presentation.mp4` sont relatifs : la page fonctionnera même si le dossier du site est déplacé ou hébergé en ligne.

Conclusion. Cette page affiche le logo redimensionné à 200 pixels de large, un lecteur audio pour l'hymne et un lecteur vidéo de 480 pixels de large, tous les fichiers étant correctement organisés dans les sous-dossiers du site.

10.3 Les formulaires en HTML

Méthode 3 : Créer un formulaire en HTML

MÉTHODE Construire un formulaire

1. Identifier les informations à saisir (nom, âge, classe, choix multiples, message...) et choisir pour chacune le type de champ adapté.
2. Encadrer le tout par la balise `<form>` avec ses attributs : **action** (page ou adresse qui recevra les données) et **method** (get ou post).
3. Associer chaque champ à une étiquette `<label>` pour que l'utilisateur sache quoi saisir (attribut `for` égal à l'id du champ).
4. Choisir la balise de saisie :
 - `<input type="text">` : **texte court**; `type="password"` : **mot de passe masqué**; `type="number"` : **nombre**; `type="date"` : **date**; `type="email"` : **adresse e-mail**;
 - `type="radio"` : **choix unique** parmi plusieurs (même attribut **name** pour tout le groupe);
 - `type="checkbox"` : **choix multiples**;
 - `<select>` avec `<option>` : **liste déroulante**;
 - `<textarea>` : **texte long** sur plusieurs lignes.
5. Nommer chaque champ avec l'attribut **name** : sans lui, la donnée saisie n'est pas transmise.
6. Terminer par les boutons : `<input type="submit" value="Envoyer">` pour valider, `type="reset"` pour effacer.

Réflexe : pour un groupe de boutons radio, vérifier que tous partagent le même **name** mais des **value** différentes.

EXERCICE RÉSOLU 11 Formulaire d'inscription au club informatique

Le club informatique du lycée de Maroua veut un formulaire en ligne demandant : le nom de l'élève (texte), sa date de naissance, sa classe (liste déroulante : 2nde, 1ère, Tle), son sexe (boutons radio : Masculin/Féminin), les activités souhaitées (cases à cocher : Programmation, Réseaux, Robotique) et un bouton « S'inscrire ». Écrire le code HTML du formulaire (méthode `post`, action `inscription.html`).

SOLUTION

Raisonnement. On applique la méthode champ par champ : texte court → `type="text"`; date → `type="date"`; choix unique dans une liste → **<select>**; choix unique visible → `type="radio"` avec le même **name**; choix multiples → `type="checkbox"`.

```
<form action="inscription.html" method="post">
  <p>
    <label for="nom">Nom de l'eleve :</label>
    <input type="text" id="nom" name="nom" required>
  </p>
  <p>
    <label for="naissance">Date de naissance :</label>
    <input type="date" id="naissance" name="naissance">
  </p>
  <p>
    <label for="classe">Classe :</label>
    <select id="classe" name="classe">
      <option value="2nde">Seconde</option>
      <option value="1ere">Premiere</option>
      <option value="tle">Terminale</option>
    </select>
  </p>
  <p>
    Sexe :
    <input type="radio" id="m" name="sexe" value="M">
    <label for="m">Masculin</label>
    <input type="radio" id="f" name="sexe" value="F">
    <label for="f">Feminin</label>
  </p>
  <p>
    Activites souhaitees :<br>
    <input type="checkbox" name="prog" value="prog"> Programmation
    <input type="checkbox" name="res" value="res"> Reseaux
    <input type="checkbox" name="rob" value="rob"> Robotique
  </p>
  <p>
    <input type="submit" value="S'inscrire">
    <input type="reset" value="Effacer">
  </p>
</form>
```

Justifications. (1) Les deux boutons radio portent le même `name="sexe"` : le navigateur garantit qu'un seul choix est possible ; leurs `value` différentes permettent d'identifier le choix. (2) L'attribut `required` sur le nom rend la saisie obligatoire. (3) L'attribut `for` du **<label>** correspond à l'`id` du champ : un clic sur l'étiquette active le champ. (4) Chaque

champ possède un **name**, condition indispensable pour que la donnée soit transmise. Conclusion. Ce formulaire collecte toutes les informations demandées avec les types de champs adaptés et un bouton d'envoi : il est prêt à être intégré au site du club.

10.4 Organisation du dossier du site

Méthode 4 : Organiser l'arborescence d'un site web

MÉTHODE Structurer l'arborescence d'un site web

1. Créer un dossier racine portant le nom du site, en minuscules et sans espaces ni accents (ex. `site_lycee`).
2. Placer la page d'accueil à la racine sous le nom **index.html** : c'est la page ouverte par défaut par les navigateurs et les serveurs.
3. Créer des sous-dossiers thématiques : `images/`, `css/`, `js/`, `pages/`, `videos/`, `sons/`.
4. Nommer tous les fichiers en minuscules, sans espaces (utiliser `_` ou `-`), sans accents : `emploi_du_temps.html` et non `Emploi du temps.html`.
5. Utiliser des chemins relatifs dans les liens et les insertions : `pages/contact.html` pour descendre dans un sous-dossier, `../index.html` pour remonter d'un niveau.
6. Sauvegarder régulièrement tout le dossier (clé USB, cloud) : un site se déplace et se sauvegarde en bloc.

EXERCICE RÉSOLU 12 Arborescence du site d'une bibliothèque scolaire

La bibliothèque du lycée de Douala veut un site composé de : une page d'accueil, une page « catalogue », une page « contact », un logo, trois photos de la salle de lecture et une feuille de style. 1) Proposer une arborescence complète du dossier du site. 2) Écrire le code HTML à placer dans la page `catalogue.html` pour afficher le logo et créer un lien retour vers l'accueil.

SOLUTION

1) Arborescence. On applique la méthode : page d'accueil **index.html** à la racine, pages secondaires dans `pages/`, images dans `images/`, style dans `css/`.

```
site_bibliotheque/  
|-- index.html  
|-- pages/  
|   |-- catalogue.html  
|   |-- contact.html
```

```
|-- images/
|   |-- logo.png
|   |-- salle1.jpg
|   |-- salle2.jpg
|   |-- salle3.jpg
|-- css/
|   |-- style.css
```

2) Code dans `pages/catalogue.html`. Le fichier se trouve dans le sous-dossier `pages/` : pour atteindre `images/logo.png`, il faut d'abord remonter d'un niveau avec `../`. Même raisonnement pour le lien vers **index.html**.

```
<img src = "../images/logo.png" alt = "Logo de la bibliotheque" >
<p><a href = "../index.html">Retour a l'accueil</a></p>
```

Justification. Le chemin `../images/logo.png` signifie : « remonte au dossier parent (`site_bibliotheque`), puis descends dans `images` ». Un chemin `images/logo.png` (sans `../`) chercherait un dossier `images` à l'intérieur de `pages/`, qui n'existe pas : l'image ne s'afficherait pas.

Conclusion. Avec cette arborescence claire et ces chemins relatifs corrects, le site est portable : il suffit de copier le dossier `site_bibliotheque` sur n'importe quelle machine pour qu'il fonctionne sans modification.

10.5 Contribution du CSS

Méthode 5 : Décrire la contribution du CSS

MÉTHODE Comprendre et appliquer le rôle du CSS

1. Retenir la séparation des rôles : HTML décrit le contenu et la structure de la page (titres, paragraphes, tableaux, formulaires) ; CSS décrit la présentation (couleurs, polices, tailles, marges, alignements, bordures).
2. Citer les contributions du CSS : il rend le site plus attrayant, harmonise la présentation de toutes les pages depuis un seul fichier, facilite la maintenance (une seule modification pour tout le site) et allège le code HTML.
3. Connaître les trois modes d'application :
 - en ligne : attribut `style` dans une balise (à éviter, peu maintenable) ;
 - interne : balise **<style>** dans le **<head>** ;
 - externe : fichier `.css` lié par **<link rel = "stylesheet" href = "css/style.css">** (méthode recommandée).
4. Maîtriser la syntaxe d'une règle CSS : `selecteur { propriete: valeur; }`. Le sélecteur désigne les éléments visés (nom de balise, `.classe`, `#identifiant`).

EXERCICE RÉSOLU 13 Mise en forme du site du lycée

1) Citer quatre contributions du CSS à un site web. 2) Écrire un fichier `style.css` qui : met le fond de la page en gris clair, colore les titres `<h1>` en bleu et les centre, et donne aux tableaux des bordures noires fusionnées. 3) Montrer comment lier cette feuille à la page `index.html`.

SOLUTION

1) Contributions du CSS.

- Il embellit la présentation des pages (couleurs, polices, espacements) et rend le site attractif ;
- il sépare le contenu de la présentation, ce qui rend le code HTML plus lisible ;
- il harmonise l'apparence de toutes les pages du site grâce à une feuille de style unique ;
- il facilite la maintenance : modifier une seule ligne du fichier CSS met à jour tout le site.

2) Fichier `style.css`. On applique la syntaxe `selecteur { propriete: valeur; }`. Pour obtenir des bordures sur les cellules et non seulement autour du tableau, on cible `<th>` et `<td>`.

```
body {
    background-color : #f0f0f0;
}

h1 {
    color : blue;
    text-align : center;
}

th, td {
    border : 1px solid black;
}

table {
    border-collapse : collapse;
}
```

3) Liaison dans `index.html`. La balise `<link>` se place dans le `<head>` :

```
<head>
  <meta charset="UTF-8">
  <title>Accueil du lycee</title>
  <link rel="stylesheet" href="css/style.css">
</head>
```

Justifications. `border-collapse: collapse` fusionne les bordures doubles des cellules en une seule ligne, ce qui donne un tableau plus net. Le chemin `css/style.css` est relatif à la page `index.html` située à la racine : il descend dans le sous-dossier `css`.

Conclusion. Grâce à cette feuille de style externe unique, toutes les pages du site du ly-

cée qui la lient auront la même apparence, et toute retouche graphique se fera en un seul endroit.

10.6 Synthèse : construire une page complète

Méthode 6 : Démarche globale de création d'une page web

MÉTHODE

Démarche globale de création d'une page web

1. Analyser la demande : lister les éléments attendus (textes, tableau, images, formulaire, style).
2. Organiser le dossier du site avant de coder (arborescence, noms de fichiers).
3. Écrire le squelette HTML : `<!DOCTYPEhtml>`, `<html>`, `<head>` (encodage UTF-8, titre, lien CSS), `<body>`.
4. Structurer le contenu avec les titres hiérarchisés `<h1>` à `<h6>` et les paragraphes `<p>`.
5. Insérer les éléments riches : tableau, images, liens, formulaire, en vérifiant chaque chemin relatif.
6. Appliquer la mise en forme via la feuille CSS externe.
7. Tester dans le navigateur : vérifier l'affichage de chaque image, le fonctionnement de chaque lien et l'envoi du formulaire.

EXERCICE RÉSOLU 14

Page complète du site d'un cybercafé

Le gérant du cybercafé « Digital Buea » veut une page d'accueil contenant : un titre principal « Bienvenue chez Digital Buea », une photo de la salle (`images/salle.jpg`), un tableau des tarifs (2 lignes de données : 1 h à 300 FCFA ; 5 h à 1 200 FCFA), et un petit formulaire de réservation (nom + nombre d'heures + bouton « Réserver »). La page utilise la feuille `css/style.css`. Écrire le code HTML complet de la page.

SOLUTION

Raisonnement. On suit la démarche globale : squelette HTML5 avec encodage UTF-8 et lien CSS dans le `<head>`, puis dans le `<body>` : titre, image avec `alt`, tableau à 2 colonnes (Durée, Prix) avec en-têtes `<th>`, et formulaire en méthode `post` avec champs nommés.

```
<!DOCTYPEhtml>
<html lang="fr">
<head>
  <meta charset="UTF-8">
```

```
<title>Digital Buea – Accueil</title>
<link rel="stylesheet" href="css/style.css">
</head>
<body>
  <h1>Bienvenue chez Digital Buea</h1>

  <h2>Nos tarifs</h2>
  <table border="1">
    <tr>
      <th>Duree</th>
      <th>Prix (FCFA)</th>
    </tr>
    <tr>
      <td>1 heure</td>
      <td>300</td>
    </tr>
    <tr>
      <td>5 heures</td>
      <td>1200</td>
    </tr>
  </table>

  <h2>Reservation</h2>
  <form action="reservation.html" method="post">
    <p>
      <label for="nom">Votre nom :</label>
      <input type="text" id="nom" name="nom" required>
    </p>
    <p>
      <label for="heures">Nombre d'heures :</label>
      <input type="number" id="heures" name="heures" min="1">
    </p>
    <p>
      <input type="submit" value="Reserver">
    </p>
  </form>
</body>
</html>
```

Vérifications. (1) Le tableau est régulier : 2 cellules par ligne. (2) Les chemins `images/salle.jpg` et `css/style.css` sont relatifs à la page située à la racine. (3) Chaque champ du formulaire possède un **name** et une étiquette **<label>**. (4) L'attribut `min="1"` empêche la saisie d'un nombre d'heures négatif ou nul.

Conclusion. Cette page complète, enregistrée sous **index.html** à la racine du dossier du site, présente le cybercafé, ses tarifs et permet les réservations : elle illustre l'intégration de tous les savoir-faire de la leçon.

ATTENTION Les 5 erreurs qui coûtent des points au Bac

1. Oublier de fermer les balises (ou les fermer dans le mauvais ordre) : chaque **<table>**,

<tr>, **<td>**, **<form>** ouvert doit être fermé dans l'ordre inverse de son ouverture. Un tableau sans **</table>** peut casser toute la mise en page.

2. Confondre **<th>** et **<td>**, ou mal gérer les fusions : avec `rowspan` ou `colspan`, la cellule fusionnée remplace les cellules absorbées ; si tu les réécrits quand même, le tableau se décale et devient irrégulier.
3. Écrire des chemins absolus du type `C:\Users\...\photo.jpg` : l'image s'affiche sur ta machine mais disparaît dès que le site est copié ou hébergé. Toujours utiliser des chemins relatifs (`images/photo.jpg`, `../index.html`) et respecter la casse des noms de fichiers.
4. Oublier l'attribut **name** dans les champs de formulaire : sans **name**, la donnée saisie n'est jamais transmise. Autre piège classique : donner des **name** différents aux boutons radio d'un même groupe, ce qui permet alors de cocher plusieurs réponses.
5. Mélanger contenu et présentation, ou mal placer le lien CSS : la mise en forme se fait en CSS, pas avec des balises obsolètes ; et la balise **<link rel="stylesheet" href="...">** se place dans le **<head>**, jamais dans le **<body>**. Précise aussi toujours l'encodage `<meta charset="UTF-8">` pour éviter l'affichage de caractères bizarres à la place des accents.

11 EXERCICES

11.1 Je vérifie mes connaissances

EXERCICE 9 QCM : les bonnes balises

Pour chaque question, choisis la (ou les) bonne(s) réponse(s) et justifie brièvement.

1. Pour définir une ligne dans un tableau HTML, on utilise :
(a) `<ligne>` b) `<tr>` c) `<td>` d) `<row>`
2. Pour insérer une image dans une page web, on utilise :
(a) `<image src="photo.png">` b) `` c) `<img>photo.png</img>` d) `<a href="photo.png">`
3. Pour fusionner deux cellules horizontalement, on utilise l'attribut :
(a) `rowspan` b) `merge` c) `colspan` d) `border`
4. Pour créer une liste déroulante dans un formulaire, on utilise :
(a) `<input type="liste">` b) `<select>` avec des `<option>` c) `<textarea>` d) `<radio>`

EXERCICE 10 Vrai ou faux, avec justification

Réponds par vrai ou faux, puis justifie ta réponse en une ou deux phrases.

1. La balise `<img>` est une balise orpheline : elle ne possède pas de balise fermante.
2. L'attribut `alt` d'une image sert uniquement à décorer la page.
3. Plusieurs boutons radio portant le même attribut `name` peuvent être cochés simultanément.
4. Le CSS est un langage qui permet de définir la présentation (couleurs, polices, mise en page) d'une page HTML.
5. Un chemin absolu du type `C:\Mes documents\photo.jpg` est recommandé pour insérer une image dans un site web destiné à être publié.

EXERCICE 11 Définitions

Défins clairement, en deux ou trois phrases chacun, les notions suivantes :

1. un **tableau** HTML et ses trois balises fondamentales ;

2. un **formulaire** HTML et la balise `<form>`;
3. une **feuille de style** CSS externe;
4. un **chemin relatif** dans l'organisation d'un site web.

EXERCICE 12 Lecture de code

On donne le code HTML suivant :

```
<table border="1">
  <tr>
    <th>Matiere</th>
    <th>Note</th>
  </tr>
  <tr>
    <td>Informatique</td>
    <td>17</td>
  </tr>
</table>
```

1. Combien de lignes et combien de colonnes ce tableau contient-il ?
2. Quel est le rôle de la balise `<th>` par rapport à `<td>` ?
3. Que signifie l'attribut `border="1"` ?
4. Décris précisément l'affichage obtenu dans le navigateur.

11.2 Je m'entraîne

EXERCICE 13 Compléter un tableau de notes

Le proviseur du lycée de Bafoussam veut publier le tableau des moyennes de la classe de Première D. Voici le code incomplet :

```
<table border="1">
  <tr>
    <th>Nom</th>
    <th>Maths</th>
    <th>Info</th>
  </tr>
  <tr>
    <td>Kamga</td>
    <td>15</td>
    -----
  </tr>
  -----
  <td>Ngo Bassa</td>
```

```
<td>12</td>
<td>16</td>
</tr>
</table>
```

1. Recopie et complète le code en remplaçant les pointillés par les balises manquantes (la note d'informatique de Kamga est 14).
2. Ajoute une dernière ligne contenant une seule cellule fusionnée sur les trois colonnes affichant le texte « Moyenne de classe : 14,25 » (utilise l'attribut `colspan`).
3. Prédis l'affichage final obtenu dans le navigateur.

EXERCICE 14 Le bon chemin pour une image

Le site du lycée bilingue de Ngaoundéré est organisé ainsi :

```
site_lycee/
|-- index.html
|-- pages/
|   |-- presentation.html
|-- images/
|   |-- logo.png
|-- css/
|   |-- style.css
```

1. Écris la balise `<img>` complète (avec un attribut `alt` pertinent) à insérer dans `index.html` pour afficher `logo.png`.
2. Écris la balise `<img>` à insérer dans `pages/presentation.html` pour afficher ce même logo. Attention au dossier de départ !
3. Explique le rôle de la notation `../` dans un chemin relatif.
4. Pourquoi est-il déconseillé d'écrire `src="C:\Users\eleve\Desktop\logo.png"` ?

EXERCICE 15 Formulaire d'inscription au club informatique

Le club informatique du collège de Sangmélina souhaite un formulaire d'inscription en ligne. Écris le code HTML complet d'un formulaire contenant :

- un champ texte pour le nom et prénom ;
- un champ texte pour la classe ;
- deux boutons radio (même groupe) pour le sexe : Masculin / Féminin ;
- une liste déroulante pour l'atelier choisi : « Initiation », « Programmation », « Mainte-

nance » ;

- deux boutons : « Envoyer » et « Effacer ».

Chaque champ devra porter un attribut `name` cohérent.

EXERCICE 16 Chasse aux erreurs

Le code suivant, écrit par un élève pressé, contient cinq erreurs. Identifie-les, explique chacune, puis propose le code corrigé.

```
<form>
<table border="1">
  <tr>
    <td>Nom : <input type="text" name="nom">
  </tr>
</table>

<p>Sexe :
  <input type="radio" name="m"> Masculin
  <input type="radio" name="f"> Feminin
</p>
<input type="submit" value="Envoyer">
```

11.3 Je me prépare au Probatoire

EXERCICE 17 Type Probatoire : le bulletin de notes en ligne

Le lycée classique de Bafoussam souhaite publier sur son site web un modèle de bulletin de notes. Le webmestre a commencé le code ci-dessous, mais il est incomplet.

```
<table border="1">
  <tr>
    <th>Matiere</th>
    <th>Note / 20</th>
    <th>Appreciation</th>
  </tr>
  <tr>
    <td>Mathematiques</td>
    <td>14</td>
    -----
  </tr>
  <tr>
    <td>Informatique</td>
    <td>16</td>
    <td>Tres bien</td>
  </tr>
  <tr>
    ----- Moyenne generale : 15 / 20 -----
```

```
</tr>  
</table>
```

1. Question de cours. Définis : tableau HTML ; balise `<tr>` ; balise `<td>`. Quelle différence fais-tu entre `<th>` et `<td>` ?
2. Recopie et complète le code en remplaçant les pointillés : l'appréciation en Mathématiques est « Bien », et la dernière ligne doit contenir une unique cellule fusionnée sur les trois colonnes.
3. Dessine à main levée le tableau tel qu'il s'affichera dans le navigateur (bordures comprises).
4. Le proviseur veut que les deux premières lignes de données partagent une même cellule « Classe : P^{le} C » sur une quatrième colonne. Quel attribut faut-il utiliser ? Réécris le début du tableau en conséquence.
5. Le fichier s'appelle `bulletin.html` et se trouve dans le dossier `pages/` ; le logo `logo.png` est dans le dossier `images/` situé à la racine du site. Écris la balise `<img>` permettant d'afficher le logo en haut du bulletin.

EXERCICE 18 Type Probatoire : la commande de tenues scolaires

La coopérative scolaire du lycée de Maroua veut créer une page web permettant aux parents de commander les tenues de leurs enfants. Le site est organisé ainsi : `index.html` à la racine, un dossier `images/` contenant `tenue.jpg`, un dossier `css/` contenant `style.css`.

1. Question de cours. Définis le CSS, cite ses trois modes d'application dans une page HTML, puis explique pourquoi la feuille de style externe est la méthode la plus adaptée à un site de plusieurs pages.
2. Écris le code HTML complet du formulaire de commande contenant :
 - un champ texte « Nom de l'élève » ;
 - une liste déroulante « Taille » avec les options S, M, L et XL ;
 - trois cases à cocher indépendantes : « Chemise », « Pantalon », « Jupe » ;
 - un champ numérique « Quantité » ;
 - les boutons « Envoyer » et « Effacer ».
3. Écris la balise permettant d'afficher l'image `tenue.jpg` depuis `index.html`, avec un texte alternatif « Tenue scolaire du lycée ».
4. Écris la ligne de code à placer dans l'en-tête `<head>` de la page pour lier la feuille de style externe `style.css`.
5. Un parent affirme : « Les cases à cocher et les boutons radio, c'est pareil. » Corrige

cette affirmation en expliquant la différence de comportement et le rôle de l'attribut `name` pour les boutons radio.

EXERCICE 19 Type Probatoire : le site de la bibliothèque scolaire

Dans le cadre de la semaine du livre, la bibliothèque du lycée de Nkongsamba crée un mini-site. On donne le tableau ci-dessous, tel qu'il doit apparaître à l'écran : la cellule « Romans » occupe deux lignes dans la première colonne, et la dernière ligne contient une cellule unique étalée sur les trois colonnes avec le texte « Total : 450 ouvrages ».

Categorie	Titre	Exemplaires
Romans	Things Fall Apart	120
	L'Avaleur de sables	80
Total : 450 ouvrages		

1. Question de cours. Explique la différence entre les attributs `rowspan` et `colspan` de la balise `<td>`.
2. Écris le code HTML complet du tableau ci-dessus (avec bordures visibles), en utilisant les attributs de fusion adéquats.
3. La page `catalogue.html` se trouve dans le dossier `pages/`. On veut y insérer :
 - la photo `rayonnage.jpg` située dans le dossier `images/` à la racine ;
 - un lien vers la page `index.html` située à la racine.

Écris les deux balises correspondantes avec les chemins relatifs corrects.

4. La bibliothécaire veut un petit formulaire de recherche : un champ texte « Titre recherche », deux boutons radio « Auteur » et « Titre » pour choisir le critère, et un bouton « Rechercher ». Écris ce code.
5. Propose une organisation complète du dossier du site (arborescence avec dossiers `pages/`, `images/`, `css/`) et justifie en trois arguments l'intérêt d'une telle organisation.

12 CORRIGÉS DES EXERCICES

12.1 Je vérifie mes connaissances

CORRIGÉ Exercice 1 — QCM : les bonnes balises

1. Réponse b) `<tr>`. Justification : `<tr>` signifie Table Row (ligne de tableau). C'est le conteneur direct des cellules (`<td>` ou `<th>`). `<td>` définit une cellule, `<ligne>` et `<row>` n'existent pas en HTML standard.
2. Réponse b) ``. Justification : `<img>` est une balise vide (auto-fermante) : elle ne contient pas de contenu et n'a pas de balise fermante. L'attribut `src` (source) est obligatoire pour indiquer l'URL de l'image. L'attribut `alt` (texte alternatif) est obligatoire pour l'accessibilité (lecteurs d'écran) et le référencement. La proposition a) utilise une syntaxe de balise paire incorrecte, c) tente de mettre le contenu entre balises (interdit), d) crée un lien hypertexte, pas une image affichée.
3. Réponse c) `colspan`. Justification : `colspan` (column span) fusionne une cellule avec ses voisines horizontalement (vers la droite). `rowspan` fusionne verticalement (vers le bas). `merge` et `border` ne sont pas des attributs de fusion.
4. Réponse b) `<select>` avec des `<option>`. Justification : `<select>` crée le conteneur de la liste déroulante, chaque `<option>` définit un choix. `<input type="liste">` n'existe pas, `<textarea>` est une zone de saisie multiligne, `<radio>` n'existe pas (c'est `<input type="radio">`).

CORRIGÉ Exercice 2 — Vrai ou faux, avec justification

1. Vrai. La balise `<img>` est une balise vide (ou auto-fermante) : elle ne contient pas de contenu et n'a pas de balise fermante `</img>`. En HTML5, on l'écrit simplement `<img ...>` (ou `<img ... />` en XHTML).
2. Faux. L'attribut `alt` (texte alternatif) est obligatoire et sert principalement à l'accessibilité : il est lu par les synthétiseurs vocaux pour les malvoyants, affiché si l'image ne charge pas, et utilisé par les moteurs de recherche (SEO). Ce n'est pas un attribut décoratif (pour cela on utiliserait CSS `background-image`).
3. Faux. Les boutons radio (`<input type="radio">`) partageant le même attribut `name` forment un groupe exclusif : un seul peut être coché à la fois. Cocher l'un décoche automatiquement les autres du même groupe.
4. Vrai. CSS (Cascading Style Sheets) est le langage standard de feuilles de style. Il dé-

crit la présentation (couleurs, polices, espacements, mise en page flexbox/grid, animations, responsive) en séparant le contenu (HTML) de l'apparence.

5. Faux. Un chemin absolu local (C:\Users\...) ne fonctionne que sur la machine de l'auteur. Sur un serveur web (Linux/Unix), les chemins utilisent des slashes /, et l'arborescence C:\ n'existe pas. Il faut utiliser des chemins relatifs (portables) ou des URLs absolues (https://...).

CORRIGÉ Exercice 3 — Définitions

1. Tableau HTML : Structure de données tabulaire organisée en lignes et colonnes, délimitée par la balise `<table>`. Ses trois balises fondamentales sont : `<tr>` (Table Row) pour chaque ligne, `<th>` (Table Header) pour les cellules d'en-tête (gras, centrées, sémantique), et `<td>` (Table Data) pour les cellules de données standard.
2. Formulaire HTML : Zone interactive (`<form>`) permettant à l'utilisateur de saisir des données envoyées à un serveur (attributs `action` = URL de traitement, `method` = GET ou POST). La balise `<form>` est le conteneur racine qui regroupe tous les champs de saisie (`<input>`, `<select>`, `<textarea>`, etc.) et les boutons d'envoi/réinitialisation.
3. Feuille de style CSS externe : Fichier distinct (extension `.css`) contenant uniquement des règles de style (sélecteurs + déclarations). Il est lié au document HTML via `<link rel="stylesheet" href="chemin/style.css">` dans le `<head>`. Cela assure la séparation structure/présentation, la réutilisation sur toutes les pages du site (mise en cache navigateur) et une maintenance centralisée.
4. Chemin relatif : Chemin d'accès à une ressource (image, page, CSS) exprimé par rapport à la position du fichier courant qui l'appelle. Il utilise `./` (dossier courant, implicite), `../` (remonter d'un niveau vers le dossier parent) et `dossier/` (descendre dans un sous-dossier). Indispensable pour la portabilité du site (fonctionne en local comme sur serveur).

CORRIGÉ Exercice 4 — Lecture de code

1. 2 lignes (deux balises `<tr>`) et 2 colonnes (deux cellules par ligne : `<th>/<th>` puis `<td>/<td>`).
2. `<th>` (Table Header) définit une cellule d'en-tête : le navigateur l'affiche par défaut en gras et centré, et les technologies d'assistance (lecteurs d'écran) l'associent sémantiquement aux cellules de données de sa colonne/ligne (`scope="col"` implicite). `<td>` (Table Data) définit une cellule de données standard (alignement gauche par défaut, pas de gras).

3. L'attribut `border="1"` (obsolète en HTML5, à remplacer par CSS `border: 1px solid black;`) demande au navigateur d'afficher une bordure de 1 pixel autour du tableau et autour de chaque cellule.
4. Affichage : Un tableau à bordures visibles (grille 2×2). Première ligne (en-tête) : « Matière » | « Note » (gras, centré). Deuxième ligne (données) : « Informatique » (gauche) | « 17 » (gauche).

12.2 Je m'entraîne

CORRIGÉ Exercice 5 — Compléter un tableau de notes

1. Code complété :

```
<table border="1">
  <tr>
    <th>Nom</th>
    <th>Maths</th>
    <th>Info</th>
  </tr>
  <tr>
    <td>Kamga</td>
    <td>15</td>
    <td>14</td>
  </tr>
  <tr>
    <td>Ngo Bassa</td>
    <td>12</td>
    <td>16</td>
  </tr>
</table>
```

2. Ajout de la ligne de moyenne (fusion 3 colonnes) :

```
<tr>
  <td colspan="3">Moyenne de classe : 14,25</td>
</tr>
```

3. Affichage final prévu : Tableau à 4 lignes × 3 colonnes avec bordures.

- Ligne 1 (en-tête) : Nom | Maths | Info
- Ligne 2 : Kamga | 15 | 14
- Ligne 3 : Ngo Bassa | 12 | 16
- Ligne 4 : une seule grande cellule centrée horizontalement (par défaut pour `<td>` fusionné) sur toute la largeur : « Moyenne de classe : 14,25 »

ATTENTION Attribut `border` **obsolète**

L'attribut `border` sur la balise `<table>` est obsolète en HTML5. Il est utilisé ici uniquement pour visualiser rapidement la structure du tableau lors des exercices. En production, on doit utiliser CSS :

```
table, th, td { border : 1px solid black ; border-collapse : collapse ; }
```

CORRIGÉ Exercice 6 — Le bon chemin pour une image

1. Dans `index.html` (à la racine `site_lycee/`) :

```

```

2. Dans `pages/presentation.html` (un niveau plus bas) :

```

```

3. `../` signifie « remonter d'un niveau dans l'arborescence » (aller dans le dossier parent). Depuis `pages/`, `../` ramène à la racine `site_lycee/`, d'où l'on peut descendre dans `images/`.
4. Un chemin absolu local `C:\Users\...` :
 - Ne fonctionne que sur le poste de l'élève (chemin physique Windows).
 - Utilise des antislashes `\` invalides dans les URLs (le web utilise des slashes `/`).
 - Casse la portabilité : le site ne s'affichera pas une fois mis en ligne sur un serveur Linux/Unix.
 - Divulgue la structure locale du disque (faille mineure de sécurité).

CORRIGÉ Exercice 7 — Formulaire d'inscription au club informatique

```
<form action="traitement_inscription.php" method="post">
  <!-- Traitement côté serveur (non vu en Première) -->
  <fieldset>
    <legend>Inscription au Club Informatique</legend>

    <p>
      <label for="nom_prenom">Nom et prénom :</label>
      <input type="text" id="nom_prenom" name="nom_prenom" required>
    </p>
  </fieldset>
</form>
```

```

</p>

<p>
  <label for="classe">Classe :</label>
  <input type="text" id="classe" name="classe" required>
</p>

<fieldset>
  <legend>Sexe</legend>
  <label><input type="radio" name="sexe" value="M"> Masculin</label>
  <label><input type="radio" name="sexe" value="F"> Féminin</label>
</fieldset>

<p>
  <label for="atelier">Atelier choisi :</label>
  <select id="atelier" name="atelier">
    <option value="initiation">Initiation</option>
    <option value="programmation">Programmation</option>
    <option value="maintenance">Maintenance</option>
  </select>
</p>

<p>
  <input type="submit" value="Envoyer">
  <input type="reset" value="Effacer">
</p>
</fieldset>
</form>

```

Points clés : name cohérents (nom_prenom, classe, sexe, atelier). Boutons radio : même name="sexe" pour l'exclusivité, value distincts. Liste déroulante : <select> + <option value="...">. Boutons : type="submit" et type="reset". <label> liés par for/id (accessibilité).

CORRIGÉ Exercice 8 — Chasse aux erreurs

Les 5 erreurs identifiées et corrigées :

1. Balise <td> non fermée. La cellule « Nom : <input ...> » n'a pas de </td>. Correction : ajouter </td> avant </tr>.
2. Balise <tr> non fermée. La ligne du tableau n'a pas de </tr>. Correction : ajouter </tr> avant </table>.
3. Chemin d'image absolu Windows avec antislashes. src="C:\photos\ecole.jpg" invalide sur le web. Correction : chemin relatif src="images/ecole.jpg" (en supposant l'image dans un dossier images/ à côté du fichier HTML) et slashes /.
4. Boutons radio avec name différents. name="m" et name="f" créent deux groupes distincts : les deux peuvent être cochés en même temps. Correction : même name="sexe" pour les grouper, value="M" et value="F" pour différencier la valeur envoyée.

5. Balise `<form>` non fermée. Pas de `</form>` à la fin. Correction : ajouter `</form>` après le bouton Envoyer.

Code corrigé complet :

```
<form action="traitement.php" method="post">
  <!-- Traitement côté serveur (non vu en Première) -->
  <table border="1">
    <tr>
      <td>
        <label>Nom : <input type="text" name="nom"></label>
      </td>
    </tr>
  </table>
  
  <p>
    Sexe :
    <label><input type="radio" name="sexe" value="M"> Masculin</label>
    <label><input type="radio" name="sexe" value="F"> Féminin</label>
  </p>
  <input type="submit" value="Envoyer">
</form>
```

12.3 Je me prépare au Probatoire

CORRIGÉ

Exercice 9 — Type Probatoire : le bulletin de notes en ligne

Barème indicatif : 10 pts (Q1 : 2pts, Q2 : 3pts, Q3 : 2pts, Q4 : 2pts, Q5 : 1pt)

1. Question de cours (2 pts).

- Tableau HTML : Élément `<table>` structurant des données tabulaires en lignes (`<tr>`) et colonnes (`<td>/<th>`).
- Balise `<tr>` : Table Row, conteneur d'une ligne de cellules.
- Balise `<td>` : Table Data, cellule de données standard (alignée à gauche par défaut).
- Différence `<th>` vs `<td>` : `<th>` = cellule d'en-tête (gras, centré, sémantique `scope="col/row"`), `<td>` = cellule de donnée courante.

2. Code complété (3 pts).

```
<table border="1">
  <tr>
    <th>Matiere</th>
    <th>Note / 20</th>
    <th>Appreciation</th>
  </tr>
  <tr>
    <td>Mathematiques</td>
```

```

        <td>14</td>
        <td>Bien</td>
    </tr>
    <tr>
        <td>Informatique</td>
        <td>16</td>
        <td>Tres bien</td>
    </tr>
    <tr>
        <td colspan="3">Moyenne generale : 15 / 20</td>
    </tr>
</table>

```

3. Dessin du rendu (2 pts).

Matiere	Note / 20	Appreciation
Mathematiques	14	Bien
Informatique	16	Tres bien
Moyenne generale : 15 / 20		

(Bordures extérieures et intérieures visibles, dernière cellule fusionnée sur 3 colonnes, contenu centré).

4. Nouvelle colonne « Classe » avec fusion verticale (2 pts). Attribut : `rowspan="2"` sur la cellule « Classe : 1^{ère} C » placée dans la première ligne de données (Mathématiques).

```

<table border="1">
    <tr>
        <th>Matiere</th>
        <th>Note / 20</th>
        <th>Appreciation</th>
        <th>Classe</th>
    </tr>
    <tr>
        <td>Mathematiques</td>
        <td>14</td>
        <td>Bien</td>
        <td rowspan="2">Classe : P$^{\text{le}} C</td>
    </tr>
    <tr>
        <td>Informatique</td>
        <td>16</td>
        <td>Tres bien</td>
        <!-- pas de <td> ici, la cellule du dessus s'étend sur 2 lignes -->
    </tr>
    <tr>
        <td colspan="4">Moyenne generale : 15 / 20</td>
    </tr>
</table>

```

Note : la ligne moyenne fusionne maintenant 4 colonnes (`colspan="4"`).

5. Chemin relatif pour le logo (1 pt). Fichier `pages/bulletin.html` → image `images/logo.png` (racine).

```

```

Points de vigilance Probatoire :

- Citer `rowspan` pour fusion verticale, `colspan` pour horizontale.
- Ne pas oublier `</tr>`, `</td>`, `</table>`.
- Chemin relatif : `../` pour sortir de pages/, puis `images/`.
- Différence `<th>/<td>` : sémantique + style par défaut.

CORRIGÉ Exercice 10 — Type Probatoire : la commande de tenues scolaires

Barème indicatif : 12 pts (Q1 : 3pts, Q2 : 4pts, Q3 : 1pt, Q4 : 1pt, Q5 : 3pts)

1. Question de cours (3 pts).

- CSS (Cascading Style Sheets) : Langage de description de la présentation des documents HTML (couleurs, polices, boîte, positionnement, animations, responsive).
- Trois modes d'application :
 - (a) Externe (recommandé) : Fichier `.css` lié par `<link rel="stylesheet" href="...">` dans `<head>`.
 - (b) Interne (embarquée) : Balise `<style>` dans le `<head>` de la page.
 - (c) En ligne (inline) : Attribut `style="..."` directement sur une balise HTML.
- Pourquoi externe pour un site multi-pages ?
 - (a) Séparation des responsabilités : HTML = structure, CSS = présentation → code plus lisible, maintenable.
 - (b) Réutilisation & Cache : Une seule feuille téléchargée et mise en cache pour toutes les pages → chargement plus rapide.
 - (c) Cohérence graphique : Modifier une règle (ex. couleur principale) met à jour tout le site instantanément.

2. Formulaire de commande complet (4 pts).

```
<form action="commande.php" method="post">
  <!-- Traitement côté serveur (non vu en Première) -->
  <fieldset>
    <legend>Commande de tenues scolaires</legend>

    <p>
      <label for="nom_eleve">Nom de l'élève :</label>
      <input type="text" id="nom_eleve" name="nom_eleve" required>
    </p>
```

```

<p>
  <label for="taille">Taille :</label>
  <select id="taille" name="taille">
    <option value="S">S</option>
    <option value="M">M</option>
    <option value="L">L</option>
    <option value="XL">XL</option>
  </select>
</p>

<fieldset>
  <legend>Articles (choix multiples)</legend>
  <label><input type="checkbox" name="articles[]" value="chemise"> Chemise</label>
  <label><input type="checkbox" name="articles[]" value="pantalon"> Pantalon</label>
  <label><input type="checkbox" name="articles[]" value="jupe"> Jupe</label>
</fieldset>

<p>
  <label for="quantite">Quantité :</label>
  <input type="number" id="quantite" name="quantite" min="1" value="1">
</p>

<p>
  <input type="submit" value="Envoyer">
  <input type="reset" value="Effacer">
</p>
</fieldset>
</form>

```

3. Image depuis index.html (1 pt).

```

```

4. Lien CSS dans <head> (1 pt).

```
<link rel="stylesheet" href="css/style.css">
```

5. Correction de l'affirmation (3 pts). Non, ce n'est pas pareil.

- Boutons radio (type="radio") : Choix unique exclusif (un seul dans le groupe). Indispensable : même attribut name pour tous les boutons du groupe. Le name crée le groupe ; cocher l'un décoche les autres.
- Cases à cocher (type="checkbox") : Choix multiples indépendants (0, 1 ou plusieurs). Chaque case a généralement son propre name (ou name="articles[]" pour tableau côté serveur). Cocher l'une n'affecte pas les autres.

Exemple : Sexe = radio (un seul choix), Articles = checkboxes (chemise ET pantalon possibles).

Points de vigilance Probatoire :

- Citer les 3 modes CSS et justifier l'externe par cache navigateur, maintenance unique, séparation HTML/CSS.

- `name` identique pour radios, distincts (ou tableau `name[]`) pour checkboxes.
- `<label for="id"> + id` sur `<input>` (accessibilité).
- Chemins relatifs : `images/` depuis racine, `css/style.css` depuis racine.

CORRIGÉ Exercice 11 — Type Probatoire : le site de la bibliothèque scolaire

Barème indicatif : 12 pts (Q1 : 2pts, Q2 : 4pts, Q3 : 2pts, Q4 : 2pts, Q5 : 2pts)

1. Différence `rowspan` vs `colspan` (2 pts).

- `rowspan="n"` : fusion verticale. La cellule s'étend sur `n` lignes (elle « mange » les cellules situées en dessous dans les lignes suivantes). La cellule ne doit pas être répétée dans les lignes suivantes.
- `colspan="n"` : fusion horizontale. La cellule s'étend sur `n` colonnes (elle « mange » les cellules situées à droite dans la même ligne). La cellule ne doit pas être répétée dans la même ligne.

2. Code HTML du tableau (4 pts).

```
<table border="1">
  <tr>
    <th>Categorie</th>
    <th>Titre</th>
    <th>Exemplaires</th>
  </tr>
  <tr>
    <td rowspan="2">Romans</td>
    <td>Things Fall Apart</td>
    <td>120</td>
  </tr>
  <tr>
    <!-- Pas de <td> pour Catégorie ici, rowspan=2 la couvre -->
    <td>L'Avaleur de sables</td>
    <td>80</td>
  </tr>
  <tr>
    <td colspan="3">Total : 450 ouvrages</td>
  </tr>
</table>
```

3. Chemins relatifs depuis `pages/catalogue.html` (2 pts).

- Image : ``
- Lien : `<a href="../../index.html">Accueil du site</a>`

Justification : `../` sort du dossier `pages/` vers la racine, puis `images/` ou `index.html`.

4. Formulaire de recherche (2 pts).

```
<form action="recherche.php" method="get">
  <!-- Traitement côté serveur (non vu en Première) -->
  <p>
    <label for="titre_recherche">Titre recherche :</label>
    <input type="text" id="titre_recherche" name="titre_recherche">
  </p>
  <fieldset>
    <legend>Critère de recherche</legend>
    <label><input type="radio" name="critere" value="auteur"> Auteur</label>
    <label><input type="radio" name="critere" value="titre"> Titre</label>
  </fieldset>
  <p>
    <input type="submit" value="Rechercher">
  </p>
</form>
```

5. Organisation du dossier et justifications (2 pts). Arborescence proposée :

```
site_bibliotheque/
|-- index.html
|-- pages/
|   |-- catalogue.html
|   |-- recherche.html
|-- images/
|   |-- rayonnage.jpg
|   |-- logo.png
|-- css/
|   |-- style.css
```

Trois arguments pour cette organisation :

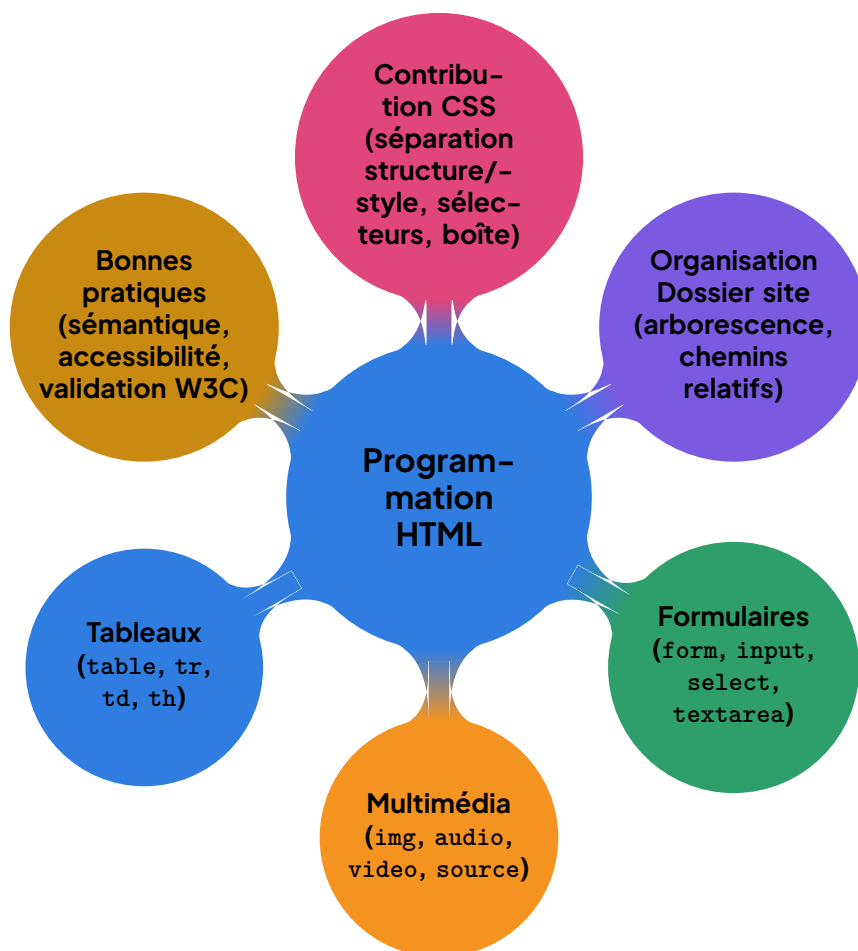
- (a) Séparation des responsabilités (Modèle MVC simplifié) : HTML (contenu/structure) dans / et pages/, CSS (présentation) dans css/, médias (images) dans images/. Chaque équipe (intégrateur, graphiste, rédacteur) travaille dans son dossier sans conflits.
- (b) Portabilité et déploiement facilités : Chemins relatifs stables (../css/style.css, ../images/...) fonctionnent identiquement en local (Windows/Mac/Linux) et sur le serveur web distant (Linux/Apache/Nginx).
- (c) Performance et maintenance : Fichier CSS unique (style.css) mis en cache navigateur après la première page visitée → chargement instantané des pages suivantes. Modification du design (couleurs, police) en un seul fichier pour tout le site.

Points de vigilance Probatoire :

- rowspan sur la première cellule de la fusion (ligne « Things Fall Apart »), puis omettre le <td> correspondant dans la ligne suivante (« L'Avaleur... »).
- colspan="3" sur la dernière ligne (3 colonnes dans le tableau).
- Chemins relatifs : toujours partir du fichier qui contient le code (pages/catalogue.html).
- Radio : même name="critere" pour l'exclusivité.

- Justifications organisation : citer séparation, portabilité/chemins relatifs, cache/-maintenance.

13 FICHE BILAN



Carte mentale de la leçon N08 : Programmation HTML — 6 piliers pour le Probatoire

FICHE BILAN L'essentiel à connaître pour l'examen

13.1 Définitions clés à maîtriser

- **Tableau HTML** : Structure de données tabulaire (`<table>`) organisée en lignes (`<tr>`), cellules d'en-tête (`<th>`) et cellules de données (`<td>`). Attributs clés : `colspan`, `rowspan`, `scope`.
- **Média natif** : Insertion sans plugin via `<img>` (image), `<audio>` / `<video>` (média temporel) avec `<source>` pour les formats de secours (MP4, WebM, Ogg).
- **Formulaire** : Zone interactive (`<form action="..." method="GET|POST">`) contenant des contrôles (`<input type="...">`, `<select>`, `<textarea>`, `<button>`) associés à des

étiquettes (`<label for="id">`).

- **Arborescence de site** : Dossier racine contenant `index.html`, sous-dossiers `/css`, `/js`, `/images`, `/medias`, `/pages`. Chemins relatifs obligatoires pour la portabilité.
- **CSS (Cascading Style Sheets)** : Langage de présentation. Sépare le contenu (HTML) de l'apparence. Règle : sélecteur { propriété: valeur; }. Modèle de boîte : `content`, `padding`, `border`, `margin`.

13.2 Aide-mémoire syntaxique (à connaître par cœur)

Structure	Syntaxe minimale / Attributs indispensables
Tableau	<code><table><tr><th scope="col">Titre</th></tr><tr><td>Donnée</td></code>
Image	<code></code>
Vidéo	<code><video controls width="640"><source src="film.mp4" type="video/mp4">Votre navigateur ne supporte pas la vidéo.</video></code>
Audio	<code><audio controls><source src="son.mp3" type="audio/mpeg"></audio></code>
Formulaire	<code><form action="traitement.php" method="POST"><label for="nom">Nom :</label><input type="text" id="nom" name="nom" required><button type="submit">Valider</button></form></code>
Input texte/email/mot de passe	<code><input type="text email password" name="..." id="..." placeholder="..." required></code>
Input choix (radio/checkbox)	<code><input type="radio checkbox" name="groupe" value="val" id="id"><label for="id">Libellé</label></code>
Liste déroulante	<code><select name="pays" id="pays"><option value="cm">Cameroun</option><option value="fr">France</option></select></code>
Zone de texte	<code><textarea name="msg" rows="5" cols="30" placeholder="Votre message"></textarea></code>
Lien CSS externe	<code><link rel="stylesheet" href="css/style.css"></code> (dans <head>)
Sélecteur CSS	balise, .classe, #id, balise.classe, parent > enfant, ancêtre descendant
Modèle de boîte	<code>box-sizing: border-box; width: 300px; padding: 10px; border: 2px solid popdark; margin: 15px auto;</code>

13.3 Méthodes express (étapes numérotées)

MÉTHODE Insérer un tableau accessible

1. Ouvrir `<table>`, ajouter `<caption>` (titre visible).
2. Créer `<thead>` avec une ligne `<tr>` de `<th scope="col">`.
3. Créer `<tbody>` avec lignes `<tr>` et cellules `<td>`.
4. (Optionnel) `<tfoot>` pour totaux.
5. Fusionner cellules : `colspan="n"` (horizontal) ou `rowspan="n"` (vertical).

MÉTHODE Insérer un média robuste

1. Placer le fichier dans `/images`, `/medias` ou `/videos`.
2. Utiliser chemin relatif : `src="medias/demo.mp4"`.
3. Fournir au moins 2 formats via `<source>` (ex : MP4 + WebM).
4. Ajouter attributs `controls`, `width`, `height`, `poster` (vidéo).
5. Texte de secours entre les balises ouvrante/fermante.

MÉTHODE Créer un formulaire valide et accessible

1. `<form action="url_traitement" method="POST" novalidate>`.
2. Grouper par `<fieldset>` + `<legend>`.
3. Chaque contrôle : `<label for="id">` + `<input id="id" name="nom" ...>` (id unique, `name` pour le serveur).
4. Typier précisément : `email`, `tel`, `date`, `number`, `file`.
5. Validation native : `required`, `minlength`, `pattern`, `min/max`.
6. Boutons : `<button type="submit">`, `<button type="reset">`.

MÉTHODE Organiser le dossier du site (standard OnBuch+)

1. Racine : `index.html` (seule page à la racine).
2. Dossiers : `/css`, `/js`, `/images`, `/medias`, `/pages`, `/fonts`.
3. Nommage : minuscules, sans accents, tirets (pas d'espaces ni underscores) : `mon-script.js`, `photo-classe.jpg`.
4. Chemins dans HTML : `href="css/style.css"`, `src="../../images/logo.png"` (depuis `/pages`).
5. Tester hors ligne (double-clic `index.html`) et en ligne (déploiement FTP/Netlify/GitHub Pages).

MÉTHODE Appliquer le CSS (séparation des responsabilités)

1. Fichier `css/style.css` lié dans `<head>` par `<link rel="stylesheet" href="css/style.css">`.
2. Reset minimal : `* { box-sizing: border-box; margin: 0; padding: 0; }`.
3. Typographie : `body { font-family: system-ui, sans-serif; line-height: 1.6; color: #333; }`.
4. Mise en page : `display: flex/grid`, `gap`, `max-width: 1200px`; `margin: 0 auto`;
5. Responsive : `@media (max-width: 768px) { ... }` (mobile first).
6. États interactifs : `:hover`, `:focus`, `:valid`, `:invalid`.

À RETENIR Checklist avant le Bac — Leçon N08

- ☐ Je sais écrire la structure complète d'un tableau sémantique (`table`, `caption`, `thead`, `tbody`, `th scope="col|row"`, `td`, `colspan/rowspan`).
- ☐ Je connais par cœur la balise `<img>` avec `src` (chemin relatif), `alt` (obligatoire), `width/height` (évite le CLS).
- ☐ Je sais insérer une vidéo/audio avec `controls`, plusieurs `<source>` (formats de secours) et un message de secours.
- ☐ Je construis un formulaire `<form method="POST">` avec `action` (même si fictif au Pro-

batoire).

- ☐ J'associe systématiquement chaque `<input>/<select>/<textarea>` à un `<label for="id">` (accessibilité).
- ☐ Je choisis le bon type d'input (email, tel, date, file, password, radio, checkbox).
- ☐ J'utilise la validation HTML5 native (required, pattern, min/max, minlength) sans JavaScript.
- ☐ J'organise mon site : `index.html` à la racine, dossiers `/css`, `/js`, `/images`, `/pages`, noms en kebab-case.
- ☐ J'écris des chemins relatifs corrects (`../` pour remonter, dossier/fichier pour descendre).
- ☐ Je lie ma feuille de style externe dans le `<head>` et j'applique le `box-sizing: border-box` global.
- ☐ Je sépare structure (HTML) et présentation (CSS) : pas de `style="..."` ni d'attributs obsolètes (`align`, `bgcolor`, `border` sur table).



Signé • OnBuch+

Ludovic A.
CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+